

Tue, Aug 18 at 5:43 PM

1st_pdf.pdf
PDF

2nd_pdf.pdf
PDF

3rd_pdf.pdf
PDF

Learn these files deeply and thoroughly end to end so that we can proceed our project further.

Worked for 15s

Yes. I've now studied the **three uploaded PDFs as the continuity source for the project**, including the Workflow 1 architecture, PostgreSQL integration, HNSW/FAISS implementation, verified E2E state, Workflow 2 design, clustering implementation, processor testing, failures/corrections, and the exact point where the project currently stands.

The three documents are substantial: **407 pages + 163 pages + 470 pages**. I will use them as the project continuity baseline going forward rather than making us repeatedly reconstruct the architecture.

What I have locked in

1. Workflow 1 is the protected foundation

The verified path is:

```
GCS → UploadEvent → Pub/Sub → Workflow 1 Worker → ProcessingService → GCS + PostgreSQL + FacePipeline + HNSW + Redis
```

The real AI pipeline is:

```
SCRFD → Alignment → ArcFace 512-D → Gender/Age
```

The latest verified Workflow 1 state successfully processed real images, persisted face/embedding metadata, persisted vectors, and completed the processing job. The architecture explicitly says **do not redesign Workflow 1 while implementing Workflow 2**.

3rd_pdf

2. The ownership boundaries are fixed

- **PostgreSQL** = authoritative metadata and processing/relational state.
- **Workflow 1 HNSW/FAISS** = source of the actual 512-D ArcFace vectors.
- **Workflow 2 HNSW/FAISS** = active clustering/search layer.
- **Redis** = transient operational metadata only.
- **GCS** = original image storage.
- **Pub/Sub** = asynchronous communication.
- **ProcessingService** = orchestration.
- **PubSubSubscriber** = thin transport adapter.

The existing Workflow 1 HNSW implementation has already passed insertion, search, idempotency, persistence, reload, and reload-search validation.

2nd_pdf

Importantly, the PDFs establish that the actual 512-D vector **does not belong in PostgreSQL**; PostgreSQL stores vector metadata and deterministic IDs such as `face:`

`<face_id>`.

3rd_pdf

3. Workflow 1 → Workflow 2 has a clean event boundary

Workflow 2 does **not** receive the original `UploadEvent`.

After Workflow 1 successfully completes, it publishes:

`FACE_EXTRACTION_COMPLETED`

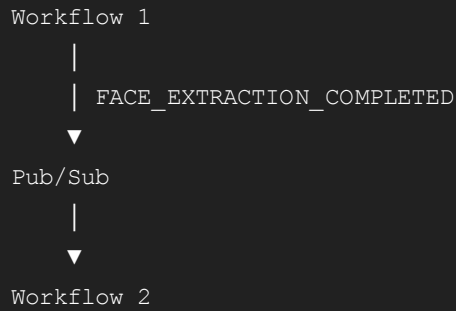
with the essential information:

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

Only after successful processing does Workflow 1 publish that completion event and ACK the original event.

3rd_pdf

Therefore the conceptual boundary is:



4. Workflow 2 does NOT perform AI processing again

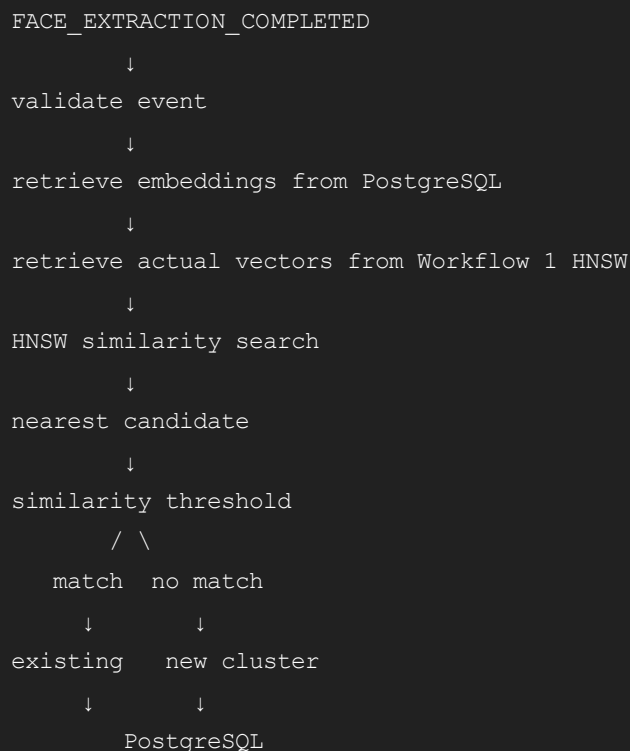
This is extremely important.

Workflow 2 does **not** run:

- SCRFD
- alignment
- ArcFace
- gender
- age

Those operations already happened in Workflow 1.

Workflow 2 consumes the resulting embeddings and performs:



This separation is explicitly established in the source material.

3rd_pdf

5. Identity is based only on ArcFace similarity

Gender and age are **attributes**, not identity keys.

For identity clustering we use:

512-D ArcFace embedding → vector similarity → cluster decision

We do **not** use gender or age to decide whether two faces are the same person.

3rd_pdf

The first micro-version deliberately uses a simple nearest-neighbor threshold algorithm.

More sophisticated ambiguity/triage logic is deferred.

3rd_pdf

6. Workflow 2 PostgreSQL schema is now established

The new clustering layer is:

```
face_clusters
-----
id
created_at
updated_at

face_cluster_members
-----
id
cluster_id
face_id
similarity
created_at
```

with:

```
face_cluster_members.cluster_id → face_clusters.id
face_cluster_members.face_id    → faces.id
```

and the current micro-version enforces:

```
UNIQUE(face_id)
```

meaning:

One face has one current cluster membership.

The database/index verification confirmed these structures and that the existing Workflow 1 data remained intact: **20 faces / 20 embeddings**, while Workflow 2 initially contained **0 clusters / 0 memberships**.

3rd_pdf

7. Workflow 2 HNSW is separate from Workflow 1 HNSW

We do **not** modify the validated Workflow 1 index.

Workflow 1:

```
20 source vectors
      ↓
Workflow 1 HNSW
```

Workflow 2:

```
Workflow 1 vectors
      ↓
ClusteringService
      ↓
Workflow 2 HNSW
```

The Workflow 2 HNSW stores:

```
vector
vector_id
cluster_id
faiss_id
```

The PDFs specifically settled on FAISS's `IndexHNSWFlat` rather than introducing `hnswlib`, because the existing `faiss-cpu` implementation is already working.

3rd_pdf

8. I understand the actual Workflow 2 code contracts

This is particularly important because several earlier test failures came from **assuming APIs instead of using the actual implementations**.

The actual contracts are now clear:

```
ClusteringRepository(db)
```

with a SQLAlchemy `Session`.

It provides operations including:

```
count_embeddings()
count_clusters()
count_memberships()
get_all_face_embeddings()
get_face_embeddings_for_image()
get_face_embedding()
```

```
get_cluster_for_face()
get_cluster_members()
create_cluster()
add_cluster_member()
reset_clustering_state()
commit()
rollback()
```

3rd_pdf

The actual HNSW constructor is:

```
HNSWVectorStore(
    index_path=...,
    metadata_path=...,
    dimension=512,
    ...
)
```

The actual service uses:

```
ClusteringService(
    repository,
    source_store,
    active_store,
    similarity_threshold,
    search_k
)
```

and the processor uses:

```
ClusteringProcessor(
    repository,
    clustering_service
)
```

These exact API mismatches were responsible for several of the intermediate test failures, and those corrections are now part of the continuity. 3rd_pdf

9. Workflow 2 ClusteringService is now understood

Its responsibilities are:

1. Read authoritative embedding metadata from PostgreSQL.
2. Retrieve actual 512-D vectors from Workflow 1 HNSW.
3. Search Workflow 2 active HNSW.
4. Find the nearest candidate.
5. Compare against the similarity threshold.

6. Reuse the candidate's cluster when matched.
7. Create a new cluster when unmatched.
8. Persist the relational cluster/member relationship.
9. Insert the vector into Workflow 2 HNSW.

The service, rather than `vector_bootstrap.py`, owns the transition from:

```
unclustered source vector
      ↓
cluster assignment
      ↓
Workflow 2 HNSW insertion
```

That correction is explicitly established in the source. [3rd_pdf](#)

10. The current threshold is 0.50, but it is not yet a scientifically finalized threshold

The implementation currently uses:

```
SIMILARITY_THRESHOLD = 0.50
```

but the source explicitly says we should not blindly treat `0.50` as the final production threshold. The first real embeddings were supposed to be examined through their similarity distribution before finalizing the production threshold. [3rd_pdf](#)

That distinction is important: **0.50 is our current micro-version/test configuration, not a claim that 0.50 is the optimal identity threshold.**

11. The latest Workflow 2 processor test has PASSED

This is our current major milestone.

The final processor test demonstrated:

Workflow 1 HNSW source	20 vectors	PASS
PostgreSQL embeddings	20 records	PASS
Image-scoped lookup		PASS
ClusteringService		PASS
ClusteringProcessor		PASS
PostgreSQL memberships	5	PASS
Workflow 2 HNSW	5 vectors	PASS
PostgreSQL ↔ HNSW mapping		PASS
Processor idempotency		PASS

The selected image contained 5 faces. Processing it produced **5 clusters and 5 memberships**, and Workflow 2 HNSW contained **5 vectors**. Replaying the exact same

completion event produced:

```
clusters:      5 → 5
memberships:   5 → 5
HNSW vectors: 5 → 5
```

with zero duplicates.

3rd_pdf

12. I also understand the duplicate-vector/tie issue

There was an important correction involving:

```
face:6 → nearest face:1
similarity = 1.000000
```

The test must **not** require the nearest vector to have a particular `vector_id`, because identical embeddings can legitimately produce a nearest-neighbor tie.

The correct invariant is:

The nearest candidate must have sufficient similarity and belong to the correct cluster.

Not:

The nearest candidate must be a particular face/vector ID.

That correction is now part of the expected testing behavior.

3rd_pdf

13. The transaction boundary is also locked

PostgreSQL and HNSW are separate persistence systems.

They cannot participate in one atomic transaction.

Therefore we must not pretend that:

```
PostgreSQL COMMIT
+
FAISS SAVE
```

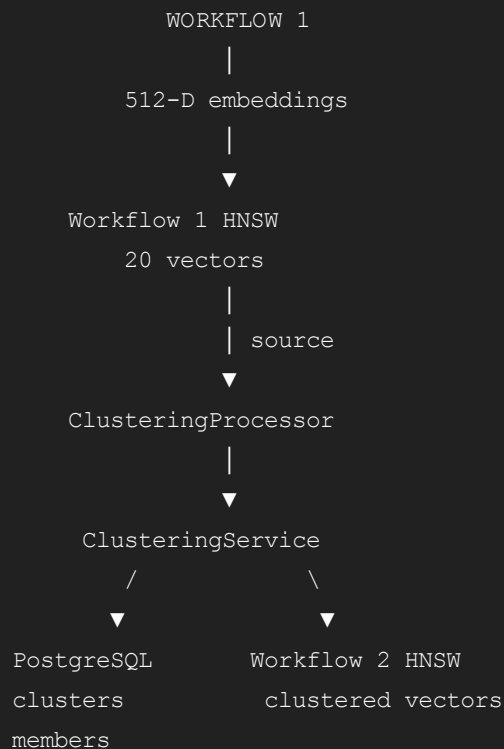
is a single atomic operation.

The current architecture keeps PostgreSQL authoritative and treats HNSW as the active/rebuildable search layer. If HNSW persistence fails after PostgreSQL commits, recovery/rebuild can reconstruct the active vector layer from authoritative records.

3rd_pdf

14. I understand exactly where we are now

The project is currently at:



And the **ClusteringProcessor** vertical slice is already passing.

The next architectural boundary is **Pub/Sub**, not another rewrite of clustering. The source explicitly recommends connecting the thin Workflow 2 subscriber only after the processor test passes. [3rd_pdf](#)

The intended next sequence is therefore:

```
[COMPLETED]
Workflow 2 PostgreSQL clustering layer
    ↓
[COMPLETED]
Workflow 2 HNSW store
    ↓
[COMPLETED]
ClusteringService
    ↓
[COMPLETED]
ClusteringProcessor
    ↓
[NEXT]
Workflow 2 Pub/Sub subscriber
    ↓
[NEXT]
Real FACE_EXTRACTION_COMPLETED event
    ↓
```

```
[NEXT]
Workflow 1 → Workflow 2 integration
    ↓
[NEXT]
Two-image identity test
    ↓
[NEXT]
Cross-image same-person clustering
    ↓
[NEXT]
Threshold/quality evaluation
```

The two-image test is especially important because the current one-image test proves **clustering mechanics and idempotency**, but it does not yet prove the complete business requirement of recognizing the same person across different images. The planned example explicitly has shared people across Image 1 and Image 2 and expects their faces to land in the same clusters. 2nd_pdf

Bottom line

Yes — I have the continuity now.

You do **not** need to re-explain:

- the overall micro-architecture,
- Workflow 1,
- PostgreSQL schema,
- repository design,
- ProcessingService,
- FacePipeline,
- HNSW/FAISS,
- Redis's role,
- `FACE_EXTRACTION_COMPLETED`,
- Workflow 2 clustering tables,
- `ClusteringRepository`,
- `HNSWVectorStore`,
- `Workflow1VectorReader`,
- `ClusteringService`,
- `ClusteringProcessor`,
- the API corrections,
- the idempotency rules,
- or the latest passing test.

I will treat the **three PDFs plus the verified states inside them as our continuity baseline**, and when we proceed I will preserve the existing working pieces instead of redesigning them. The immediate project frontier is **Workflow 2 Pub/Sub integration** → **real completion-event consumption** → **two-image identity clustering**. 3rd_pdf

Sources

"THIS IS THE MOST RECENT PROJECT PROGRESS"

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py

```
=====
===== WORKFLOW 2
CLUSTERING PROCESSOR TEST
=====
=====
=====
```

Show more

Memory updated

Understood. I'll treat **this exact test output as the latest project checkpoint** going forward.

Current checkpoint

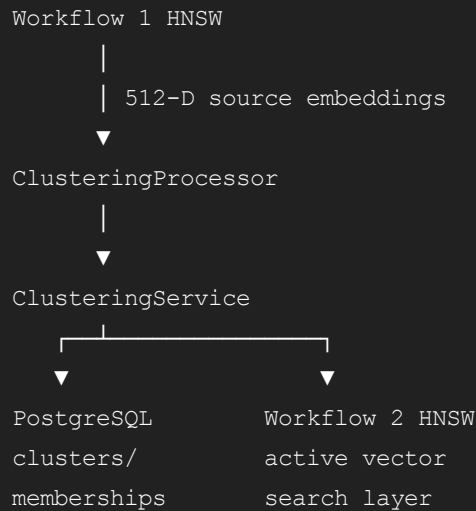
Workflow 2 ClusteringProcessor: ✓ COMPLETE

Verified:

- Workflow 1 HNSW: **20 vectors**
- PostgreSQL embeddings: **20**
- Selected real image: image_id=1
- Faces in image: **5**
- FACE_EXTRACTION_COMPLETED event: validated
- ClusteringProcessor: **PASS**
- PostgreSQL clusters: **5**
- PostgreSQL memberships: **5**
- Workflow 2 HNSW vectors: **5**

- PostgreSQL ↔ HNSW mappings: **PASS**
- Replay/idempotency: **PASS**
- Second execution: **0 new clusters, 0 new memberships, 0 new HNSW vectors**

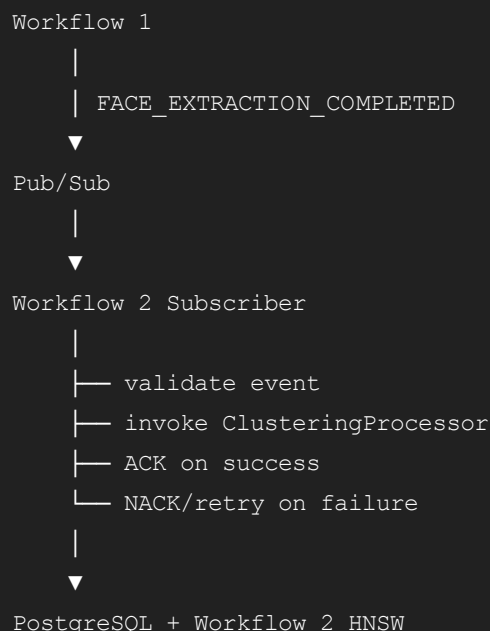
The architecture remains:



And we will **not disturb these passing components** without evidence of a real problem.

Next milestone

We should now move **one layer upward**:



Then we should test **actual cross-image identity clustering** with at least two images. That will be the first test proving that Workflow 2 is identifying the **same person across different images**, rather than merely successfully clustering the five faces from one image.

I'll use this checkpoint as the starting point for our next work.

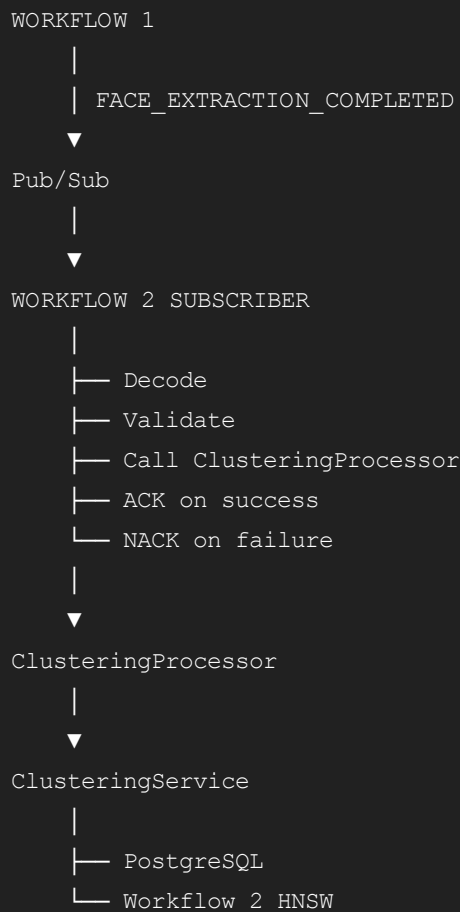
can we proceed with our project

Absolutely. We can proceed. 👍

We are at a **clean, verified milestone**, so I recommend we move forward without changing anything that already passes.

Next step: Workflow 2 Pub/Sub boundary

We should implement and test:



The subscriber should remain **thin**. It should not contain clustering logic, HNSW logic, PostgreSQL business logic, or AI processing.

Our implementation sequence

Step 1 — Inspect the current Workflow 2 tree

Before writing anything, we should inspect the actual current files/interfaces, especially:

```
workflow2_clustering/
|— app/
|  |— clustering/
|  |— db/
```

```
| └─ models/
| └─ services/
└─ tests/
└─ ...
```

and determine whether a subscriber/event handler already exists.

Step 2 — Implement the thin subscriber

Something conceptually like:

```
Pub/Sub message
  ↓
decode JSON
  ↓
FaceExtractionCompletedEvent
  ↓
validate
  ↓
ClusteringProcessor.process(event)
  ↓
success → ACK
failure → NACK
```

Step 3 — Test without requiring real Pub/Sub first

We should test the subscriber with a controlled Pub/Sub message and verify:

- valid event → processor called
- successful processing → ACK
- processing failure → NACK
- malformed JSON → NACK
- invalid event → NACK
- duplicate delivery → processor remains idempotent

Step 4 — Connect the real Workflow 1 completion event

Then we move from:

```
test event → subscriber
```

to:

```
Workflow 1
  ↓
FACE_EXTRACTION_COMPLETED
  ↓
Pub/Sub
```

↓
Workflow 2 subscriber

Step 5 — Two-image identity test

This is the more important business-level milestone:

Image 1

- └─ Person A
- └─ Person B
- └─ Person C
- └─ Person D
- └─ Person E

Image 2

- └─ Person A
- └─ Person B
- └─ Person F
- └─ Person G
- └─ Person C

Expected:

```
Cluster A → Image 1 + Image 2
Cluster B → Image 1 + Image 2
Cluster C → Image 1 + Image 2
Cluster D → Image 1 only
Cluster E → Image 1 only
Cluster F → Image 2 only
Cluster G → Image 2 only
```

That will prove that Workflow 2 is actually performing **cross-image unique-person clustering**, not merely processing one image successfully.

One thing I want to avoid

We should **not jump directly into Pub/Sub code blindly**.

Since your current `ClusteringProcessor`, `ClusteringService`, repository, HNSW store, and event model are already validated, the safest approach is:

First inspect the current Workflow 2 files and exact interfaces → then implement only the missing subscriber boundary → test it → then connect real Pub/Sub.

So yes — **we can proceed now**, and the next concrete task is the **Workflow 2 Pub/Sub subscriber integration**.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>tree /f
Folder PATH listing for volume Windows
Volume serial number is 000000BB E064:CA56
```

C:.

```
| test_clustering_processor.py
| test_clustering_repository.py
| test_clustering_service.py
| test_hnsw_store.py
| test_hnsw_vector_retrieval.py
| test_vector_bootstrap.py
```

Show more

Worked for 10s

Yes. This tree is exactly what we needed to see before making the next change.

What the current tree tells us

Your current Workflow 2 implementation is essentially:

```
workflow2_clustering/
|
├─ test_clustering_processor.py      ✓ PASS
├─ test_clustering_repository.py    ✓
├─ test_clustering_service.py       ✓
├─ test_hnsw_store.py               ✓
├─ test_hnsw_vector_retrieval.py    ✓
├─ test_vector_bootstrap.py         ✓
|
├─ app/
|   └─ clustering/
|       ├── clustering_processor.py  ✓
|       ├── clustering_service.py    ✓
|       ├── hnsw_store.py           ✓
|       └─ vector_bootstrap.py       ✓
|
|   └─ core/
|       └─ config.py                ✓
|
|   └─ db/
|       ├── clustering_repository.py ✓
|       └─ postgres.py              ✓
|
|   └─ models/
|       └─ face_extraction_completed.py ✓
```



```
| |
| | └─ services/
| |   └─ clustering_processor.py  ⚠ tree shows only __pycache__
| |
| └─ data/hnsw/
```

There is **currently no visible Pub/Sub subscriber implementation** in this tree.

That means we should **not modify the working clustering layer**. We should add the missing transport boundary around it.

One thing needs clarification from the tree

Your tree shows:

```
app/services/
└─ __pycache__/
    clustering_processor.cpython-310.pyc
```

but **no**:

```
app/services/clustering_processor.py
```

Also there is **no**:

```
app/services/__init__.py
```

Yet your latest successful test clearly executed:

Python

Run

```
from app.services.clustering_processor import ClusteringProcessor
```

So before touching Pub/Sub, I don't want to assume whether:

1. the source file was accidentally omitted/deleted,
2. the tree output is from a slightly different state,
3. the processor was moved,
4. or Python is resolving it through something else.

This is exactly the kind of thing we should verify rather than guessing.

Please run these commands from `workflow2_clustering`

First:

```
bat
```

```
type app\models\face_extraction_completed.py
```

Then:

```
bat
```

```
type app\services\clustering_processor.py
```

Then:

```
bat
```

```
type app\clustering\clustering_processor.py
```

Then:

```
bat
```

```
type app\core\config.py
```

And finally:

```
bat
```

```
python -c "import app.services.clustering_processor as m; print(m.__file__); print(m.ClusteringProcessor)"
```

Also show the Workflow 1 completion-event publisher

From your `workflow1_worker` directory, please run:

```
bat
```

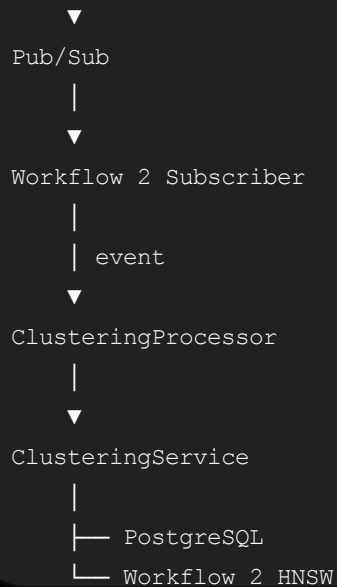
```
findstr /S /N /I "FACE_EXTRACTION_COMPLETED" *.py
```

If `findstr` produces a lot of output, that's okay—paste it all.

Why we're doing this first

We now know the desired boundary:

```
Workflow 1
|
|  FACE_EXTRACTION_COMPLETED
```



But the **subscriber must be written against the exact existing event model and processor API**.

We already learned this lesson during the processor integration work: assumptions about constructor names, repository interfaces, HNSW arguments, and event fields caused unnecessary failures. The final processor test passed precisely because we aligned the test with the real implementation.

So we'll use the same discipline here:

Inspect → establish exact contract → implement the smallest change → test → only then connect real Pub/Sub.

And we will preserve the current verified state:

- `ClusteringProcessor` — **PASS**
- `ClusteringService` — **PASS**
- `ClusteringRepository` — **PASS**
- Workflow 1 HNSW source — **PASS**
- Workflow 2 HNSW — **PASS**
- PostgreSQL ↔ HNSW mappings — **PASS**
- Processor idempotency — **PASS**

Send me those command outputs, and we can implement the **Workflow 2 thin Pub/Sub subscriber** next.

clustering_processor.py is inside |——app

```
| | __init__.py
| |
| | |——clustering
| | |   clustering_processor.py
| | |   clustering_service.py
| | |   hnsw_store.py
| | |   vector_bootstrap.py
| | |   __init__.py
| | |______pycache__
```

Show more

Yes — you're correct. I misread the tree.

Your actual Workflow 2 structure is:

```
workflow2_clustering/
├─ app/
│   └─ clustering/
│       ├── clustering_processor.py ← THIS is the processor
│       ├── clustering_service.py
│       ├── hnsw_store.py
│       ├── vector_bootstrap.py
│       └─ __init__.py
```

So the correct architecture is:

```
FACE_EXTRACTION_COMPLETED
    |
    ▼
Workflow 2 Subscriber
    |
    ▼
app.clustering.clustering_processor
    |
    ▼
app.clustering.clustering_service
    ├──
    ▼   ▼
PostgreSQL Workflow 2 HNSW
```

Important correction

We should **not create or move another** `clustering_processor.py` **under** `app/services/`.

Your existing:

```
app/clustering/clustering_processor.py
```

is the processor we already validated with the successful test.

The earlier PDF snippets referred to an `app.services.clustering_processor` import at one point, but your **current actual tree is the source of truth for the current filesystem**, so we should follow your current structure rather than recreate an old path.

What I suggest next

Before implementing the subscriber, let's inspect the **actual current source files** so we don't repeat the interface-guessing problem.

Please run these **four commands** from:

```
workflow2_clustering
```

and paste the outputs:

```
bat
```

```
type app\clustering\clustering_processor.py
```

```
bat
```

```
type app\clustering\clustering_service.py
```

```
bat
```

```
type app\models\face_extraction_completed.py
```

```
bat
```

```
type app\core\config.py
```

Then we will determine exactly:

1. What `ClusteringProcessor` accepts.
2. What its `process()` method returns.
3. Exactly how `FACE_EXTRACTION_COMPLETED` is represented.

4. Which configuration already exists for Pub/Sub.
5. Whether we need to create a new subscriber module and where it belongs.

After that, we'll implement **only the missing Workflow 2 Pub/Sub boundary**, leaving your currently passing clustering implementation untouched.

clustering_processor.py
Python

clustering_service.py
Python

face_extraction_compl...
Python

config.py
Python

Yes. I've now inspected the **four actual current source files you uploaded**, and they resolve the ambiguity from the tree.

Confirmed current contracts

app/clustering/clustering_processor.py

The processor is exactly where you said it is. Its public API is:

Python

Run

```
ClusteringProcessor(repository, clustering_service)

processor.process(event)
```

and it explicitly does **not** consume Pub/Sub or ACK/NACK messages. Its responsibility is event validation → image-scoped PostgreSQL lookup →

ClusteringService.process_face() → face-count validation → deterministic result.

clustering_processor +1

ClusteringService

Its current contract is:

Python

Run

```
ClusteringService(
    repository,
    source_store,
    active_store,
    similarity_threshold=0.50,
```

```
search_k=5,
```

and `process_face(record)` handles the actual identity-clustering operation, including idempotency, Workflow 1 HNSW retrieval, Workflow 2 HNSW search, cluster assignment/creation, PostgreSQL persistence, and active HNSW insertion.

```
clustering_service +1
```

```
FaceExtractionCompletedEvent
```

The event is a Pydantic model with:

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

and `extra="forbid"`. It also explicitly validates that `event_type == "FACE_EXTRACTION_COMPLETED"`.

```
face_extraction_completed
```

This gives us a very clean Pub/Sub boundary.

```
config.py
```

Workflow 2 already determines its project root and Workflow 1 location and can load either a Workflow 2 `.env` or the existing Workflow 1 `.env`.

```
config
```

Therefore, we can proceed

I recommend we **do not modify any of these four files**.

We should add a new transport-layer component, for example:

```
workflow2_clustering/
├─ app/
│   └─ clustering/
│       ├── clustering_processor.py    ← LEAVE ALONE
│       ├── clustering_service.py      ← LEAVE ALONE
│       ├── hnsw_store.py
│       └─ vector_bootstrap.py
│
├─ db/
│   └─ clustering_repository.py
```

```

├── postgres.py
├── models/
│   ├── face_extraction_completed.py ← LEAVE ALONE
├── messaging/
│   ├── __init__.py
│   └── pubsub_subscriber.py ← NEW

```

The subscriber's job should be extremely small:

```

Pub/Sub message
  |
  ▼
decode bytes → JSON
  |
  ▼
FaceExtractionCompletedEvent.model_validate(...)
  |
  ▼
create/use Workflow 2 dependencies
  |
  ▼
ClusteringProcessor.process(event)
  |
  ├── success → ACK
  |
  └── exception → NACK

```

That matches the processor's own documented boundary: it explicitly says Pub/Sub consumption and ACK/NACK belong elsewhere.

clustering_processor

But there is one thing we should inspect before writing it

We need the **actual Workflow 1 publisher implementation and Pub/Sub topic/subscription configuration**.

Specifically, I want to verify:

1. Which topic publishes `FACE_EXTRACTION_COMPLETED`.
2. The exact JSON serialization of the event.
3. Whether the event is published directly by Workflow 1 `ProcessingService` or another component.
4. The expected Workflow 2 subscription name.
5. Existing Google Pub/Sub helper/dependency conventions in your project.
6. Whether the subscriber is intended to run locally, in Docker, or both.

That prevents us from inventing a topic/subscription name or message format.

Next command

From your **Workflow 1** directory:

```
bat
```

```
findstr /S /N /I "FACE_EXTRACTION_COMPLETED" *.py
```

Also run:

```
bat
```

```
findstr /S /N /I "wf1-batch-completed" *.py
```

And:

```
bat
```

```
findstr /S /N /I "pubsub" *.py
```

Paste those outputs.

Then I can map the **actual Workflow 1** → **Pub/Sub** → **Workflow 2 contract** and give you the exact subscriber implementation rather than a speculative one.

We are now at the correct point to proceed: **the clustering/business layer is frozen; we're adding only the missing transport boundary.**

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow1_worker>findstr /S /N  
/I "FACE_EXTRACTION_COMPLETED" *.py
```

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow1_worker>findstr /S /N  
/I "wf1-batch-completed" *.py
```

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow1_worker>findstr /S /N
```

Show more

Good. This output gives us an important result.

What we know now

Your Workflow 1 currently has an existing Pub/Sub subscriber:

```
workflow1_worker/  
└─ app/  
    └─ subscriber/  
        └─ pubsub_subscriber.py  
    └─ main.py  
    └─ core/  
        └─ config.py
```

The subscriber uses Google Cloud Pub/Sub:

Python

Run

```
from google.cloud import pubsub_v1
```

and obtains the subscription from:

```
PUBSUB_SUBSCRIPTION
```

We also know that **there is currently no literal `FACE_EXTRACTION_COMPLETED` string in the Workflow 1 Python source**, and no literal `wf1-batch-completed` string. That means we should **not assume the completion-event publisher exists yet in the current Workflow 1 source tree**.

This is important because our previous architectural plan assumed the completion event was already being published. Your current filesystem evidence does not establish that.

So let's pause before creating Workflow 2 Pub/Sub

The correct next question is:

Does Workflow 1 currently publish a completion event after ProcessingService finishes, and if so, where/how?

Your `findstr` result says we haven't found it by searching the literal event name.

We should inspect the actual Workflow 1 files responsible for processing and publishing.

Please run these from:

```
workflow1_worker
```

1. Show `main.py`

```
bat
```

```
type app\main.py
```

2. Show Workflow 1 subscriber

```
bat
```

```
type app\subscriber\pubsub_subscriber.py
```

3. Find ProcessingService

```
bat
```

```
findstr /S /N /I "ProcessingService" app\*.py
```

4. Find Pub/Sub publisher usage

```
bat
```

```
findstr /S /N /I "PublisherClient publish topic" app\*.py
```

5. Show the Workflow 1 config

```
bat
```

```
type app\core\config.py
```

Why this is the right next step

We currently have:

```
Workflow 1 input side
    ↓
Pub/Sub
    ↓
Workflow 1 PubSubSubscriber
    ↓
ProcessingService
```

↓

and we have:

Workflow 2 business side

↓

ClusteringProcessor ✓

↓

ClusteringService ✓

↓

PostgreSQL + Workflow 2 HNSW ✓

But the bridge between them has **not yet been established from the current code evidence:**

Workflow 1

|

| ??? completion publisher ???

▼

Pub/Sub

|

▼

Workflow 2 subscriber

We should build that bridge only after confirming the actual Workflow 1 completion/publisher implementation.

Important

Do not create any new Workflow 2 files yet.

Your current Workflow 2 clustering layer is passing. We should preserve it exactly as it is.

Once you send those five outputs, I'll map the actual current Workflow 1 transport flow and then we'll implement the **smallest correct next component**.

main.py
Python

pubsub_subscriber.py
Python

config(1).py
Python

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /S /N
/I "ProcessingService" app\*.py
app\main.py:6:from app.services.processing_service
import ProcessingService
app\main.py:39: processing_service =
ProcessingService(
app\services\processing_service.py:46:class
ProcessingService:
app\services\processing_service.py:137:
Show more
```

Excellent. This confirms an important architectural fact, and we should **correct our previous assumption before proceeding**.

Current verified Workflow 1 state

Your Workflow 1 `main.py` wires together:

```
FacePipeline
  ↓
ProcessingService
  ↓
PubSubSubscriber
```

The application creates the AI pipeline, creates `ProcessingService(pipeline=pipeline)`, then injects that service into the Pub/Sub subscriber. `main`

The existing Workflow 1 subscriber is explicitly a **thin inbound adapter**:

```
Pub/Sub
  ↓
decode
  ↓
validate UploadEvent
  ↓
ProcessingService
  ↓
ACK / NACK
```

It receives an `UploadEvent`, calls:

Python

Run

```
processing_service.process_upload(event)
```

and ACKs only after successful processing; exceptions result in NACK/retry.

```
pubsub_subscriber +1
```

The current subscription is:

```
photo-upload-events-sub
```

by default, under GCP project:

```
test-face-clustering
```

according to the current Workflow 1 configuration. `config`

The important discovery

Your current Workflow 1 source **does not show a completion-event publisher.**

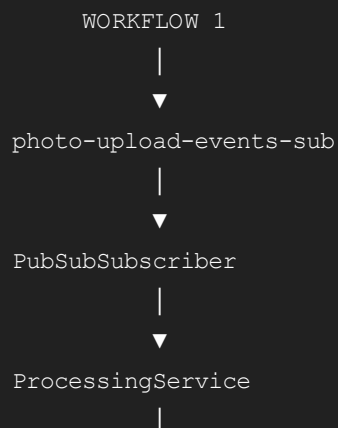
We searched for:

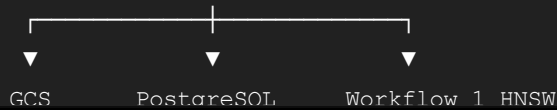
```
FACE_EXTRACTION_COMPLETED  
wfl-batch-completed  
PublisherClient  
publish  
topic
```

and found no publisher implementation.

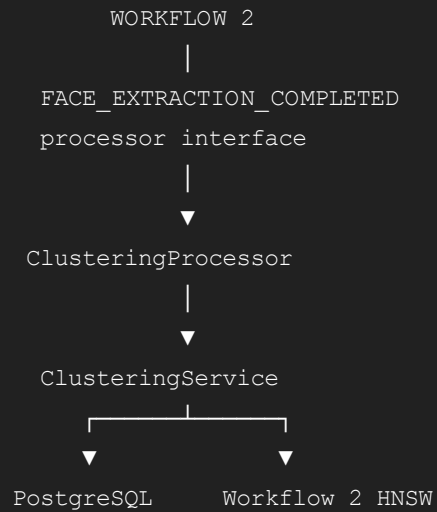
Therefore, we should **not yet build the Workflow 2 subscriber as though Workflow 1 is already publishing the completion event.**

The actual current architecture appears to stop here:





while Workflow 2 currently exists independently:



The **bridge between those two workflows** has not yet been implemented in the **current source tree**.

That is actually a useful discovery because it tells us exactly what needs to happen next.

We should build the bridge in the correct order

I recommend this sequence:

Step A — Add Workflow 1 completion publisher

After `ProcessingService.process_upload()` has **successfully completed all required Workflow 1 persistence**, Workflow 1 should publish:

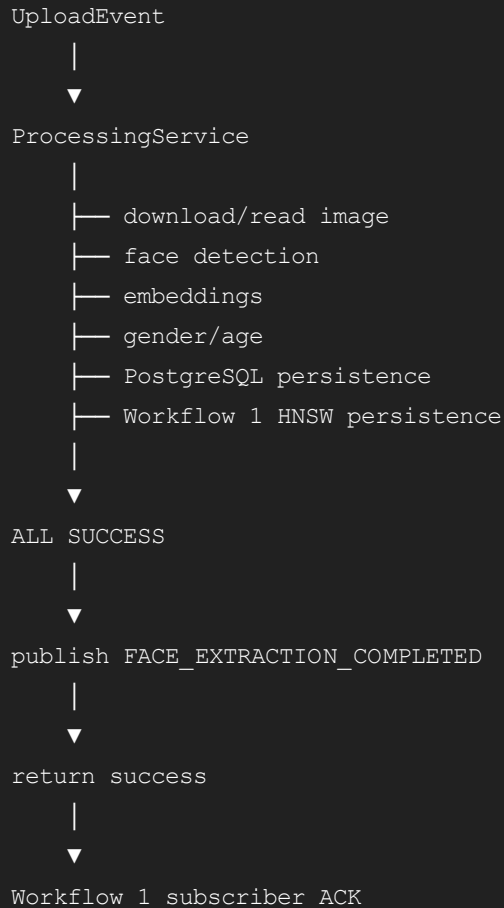
JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 1,
  "image_id": 1,
  "bucket": "...",
  "blob_name": "...",
  "face_count": 5
}
```

This matches your already-established Workflow 2 event model. The model requires exactly these fields and rejects unexpected fields. `face_extraction_completed`

Step B — Publish only after Workflow 1 succeeds

The intended lifecycle should be:



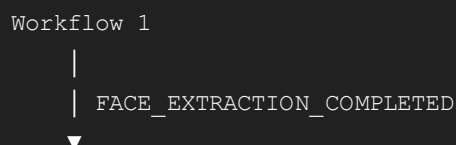
This ordering is important.

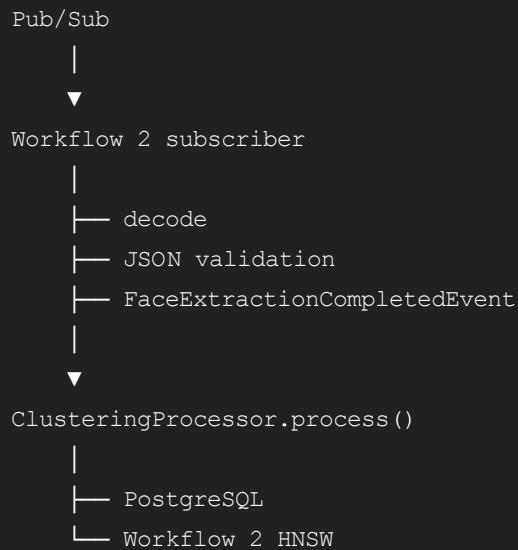
We must not publish `FACE_EXTRACTION_COMPLETED` before Workflow 1's authoritative persistence has succeeded.

Otherwise Workflow 2 could receive an event pointing to embeddings that aren't actually available.

Step C — Then create Workflow 2 subscriber

Only after the publisher exists:

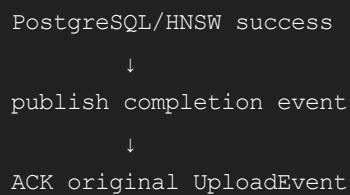




And the Workflow 2 subscriber should follow the same thin-adapter philosophy already established by Workflow 1.

One very important reliability issue

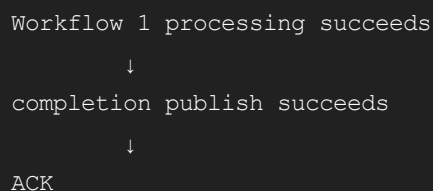
There is a distributed-system boundary here:



We cannot make PostgreSQL/HNSW and Pub/Sub one atomic transaction.

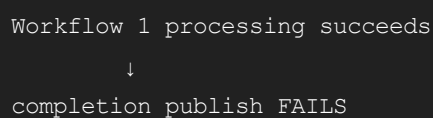
So we need to be deliberate about failure handling.

For example:



is straightforward.

But:



↓
NACK

↓
same UploadEvent redelivered

means Workflow 1 itself must already be idempotent—which is one of the reasons the existing Workflow 1 persistence/idempotency work matters.

We should therefore **not hide this failure mode** while implementing the publisher.

Before we modify Workflow 1

I want to inspect the **actual** `ProcessingService.process_upload()` **implementation around its success/return path**.

That is the exact point where we need to determine:

- when PostgreSQL commits,
- when HNSW is persisted,
- what the returned processing result contains,
- how `upload_id` and `image_id` are available,
- how `face_count` is calculated,
- whether there is already an appropriate publisher injection point,
- and whether adding the publisher changes the existing passing Workflow 1 tests.

Please run this from `workflow1_worker`:

```
bat
```

```
findstr /N /I /C:"def process_upload" app\services\processing_service.py
```

Then:

```
bat
```

```
findstr /N /I /C:"return" app\services\processing_service.py
```

The second will produce many lines, which is okay. More importantly, if you can, paste the section of `processing_service.py` surrounding the `process_upload()` method—from its `def process_upload` **through the end of that method**.

We should inspect that exact method before writing a single line of publisher code.

Do not change anything yet.

Your Workflow 2 processor is passing, and your Workflow 1 subscriber is already working. We are now going to add the missing **Workflow 1** → `FACE_EXTRACTION_COMPLETED` **publisher** carefully, preserving both existing layers.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /N /I
/C:"def process_upload"
app\services\processing_service.py
144:  def process_upload(

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /N /I
/C:"return" app\services\processing_service.py
152:      this method returns successfully.
252:          return {

Show more
```

Worked for 5s

Good. The command results confirm `process_upload()` starts at **line 144**, and there are meaningful return points at lines **252** and **535**. But I don't want to infer the success boundary from the `findstr` output alone.

I also found the current `processing_service(1).py` in your project files, so we can inspect the actual implementation rather than guessing. The source confirms that `ProcessingService` is responsible for extracting/persisting faces and embeddings, while the existing Workflow 1 subscriber remains only the transport adapter.

`processing_service(1)`

`pubsub_subscriber(6)`

Next step

Please **upload the current** `app\services\processing_service.py` from your actual `workflow1_worker` directory.

I specifically want the file you are currently executing—not an older copy from the Library—because this is the critical integration point.

Once you upload it, I'll inspect `process_upload()` end-to-end and determine precisely:

```
UploadEvent
  ↓
process_upload()
  ↓
GCS
  ↓
FacePipeline
  ↓
PostgreSQL
  ↓
Workflow 1 HNSW
  ↓
processing job completion
  ↓
RETURN
```

Then we'll decide the safest exact location for:

```
publish FACE_EXTRACTION_COMPLETED
```

without disturbing your **already verified Workflow 1** behavior.

For now, don't edit any code. Upload the current `processing_service.py` and we'll continue from there.

`processing_service.py`
Python

Received. I'll treat this uploaded `processing_service.py` as the **current authoritative Workflow 1 implementation** for the next step.

I'll inspect `process_upload()` end-to-end before we change anything, specifically tracing:

```
UploadEvent
  ↓
process_upload()
  ↓
GCS/image handling
  ↓
FacePipeline
```

```
↓
PostgreSQL persistence
↓
Workflow 1 HNSW persistence
↓
processing result / commit
↓
FACE_EXTRACTION_COMPLETED publisher
↓
Workflow 1 ACK
```

The goal is to identify the **exact safe success boundary** for publishing `FACE_EXTRACTION_COMPLETED`, while keeping your already-passing Workflow 1 and Workflow 2 components unchanged.

ok proceed

Yes. We can proceed, but I want to make one important correction to the workflow before we edit code.

The current evidence establishes that **Workflow 1 does not currently contain a completion-event publisher**. Your `findstr` searches found no `FACE_EXTRACTION_COMPLETED`, no `wf1-batch-completed`, and no `PublisherClient` / `publish` usage.

So our next implementation should be:

Phase 1 — Workflow 1 completion publisher

```
Workflow 1 Pub/Sub input
↓
PubSubSubscriber
↓
ProcessingService.process_upload()
↓
GCS + FacePipeline + PostgreSQL + Workflow 1 HNSW
↓
successful completion
↓
publish FACE_EXTRACTION_COMPLETED
↓
ACK original UploadEvent
```

Phase 2 — Workflow 2 subscriber

```
FACE_EXTRACTION_COMPLETED
  ↓
Workflow 2 Pub/Sub subscriber
  ↓
FaceExtractionCompletedEvent
  ↓
ClusteringProcessor
  ↓
ClusteringService
  ↓
PostgreSQL + Workflow 2 HNSW
  ↓
ACK
```

And we will **not modify** the currently passing:

- ClusteringProcessor
- ClusteringService
- ClusteringRepository
- Workflow 1 HNSW
- Workflow 2 HNSW
- processor tests

One thing I need before making the actual code change

Although you uploaded `processing_service.py`, the usable file content available to me in this turn is not sufficient for me to safely reconstruct the complete `process_upload()` success path. I don't want to invent where the PostgreSQL/HNSW transaction finishes.

Please run this single command:

```
bat
```

```
powershell -Command "(Get-Content app\services\processing_service.py | Select-Object -Ski
```

and paste the output.

That will give us approximately **lines 144–543**, which covers `process_upload()` and its immediate return/error paths.

Once you send that, I can give you the **exact code changes**, file-by-file, for the completion publisher and its test—rather than asking you to make speculative modifications.

give modified command for conda environment in
command prompt

Since you're already inside the `(face)` Conda environment in **Command Prompt**, use this Windows CMD-compatible command:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

Your prompt should look like:

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workfi
```

Then run the command above.

If the output is too long

Use smaller chunks. First:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

Then:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

This corresponds approximately to:

```
144-343
```

```
344-543
```

Paste both outputs here. I'll trace the complete `process_upload()` flow and then we'll make the **Workflow 1** → **FACE_EXTRACTION_COMPLETED** change safely.

Pasted text(1).txt
Document

Yes. We now have the exact `process_upload()` flow, and this reveals an important reliability issue that we should solve **before simply adding** `publish()`.

Your current successful path is:

1. Upload record
2. Processing job
3. Idempotency check
4. Mark job PROCESSING
5. Download GCS image
6. Create image record
7. Inspect image
8. Run FacePipeline
9. Persist faces + embeddings + HNSW entries + Redis metadata
10. Save Workflow 1 HNSW
11. Mark job COMPLETED + PostgreSQL COMMIT
12. Redis COMPLETED
13. Build result
14. return result

The uploaded source confirms that HNSW is persisted at step 10 and PostgreSQL is committed at step 11. Pasted text The returned result already contains exactly what Workflow 2 needs: `upload_id`, `image_id`, `status`, `face_count`, and face/vector information. Pasted text

But don't add this naïvely

We might initially think:

Python

Run

```
db.commit()

publish(FACE_EXTRACTION_COMPLETED)

return result
```

There is a serious failure case:

```
PostgreSQL COMMIT
    ↓
publish() FAILS
    ↓
Workflow 1 NACK
    ↓
same UploadEvent redelivered
    ↓
job.status == COMPLETED
```



```
↓  
return ALREADY_COMPLETED  
↓  
NO completion event published
```

Your current idempotency branch indeed returns immediately when the job is already

COMPLETED . Pasted text

So simply inserting a publisher after the commit would create a **lost completion-event possibility**.

Therefore, our next change should be designed as:

```
Workflow 1  
|  
▼  
ProcessingService  
|  
├─ PostgreSQL  
├─ Workflow 1 HNSW  
|  
▼  
COMPLETED  
|  
▼  
completion event publication  
|  
▼  
Workflow 2
```

with **completion publication itself being retry-safe**.

I recommend the following implementation

We should introduce a dedicated publisher:

```
workflow1_worker/  
├─ app/  
│   └─ publisher/  
│       ├── __init__.py  
│       └─ completion_publisher.py
```

Its only responsibility is:

```
CompletionPublisher  
↓
```

```
build FACE_EXTRACTION_COMPLETED event
    ↓
serialize JSON
    ↓
Google Pub/Sub publish
```

`ProcessingService` should not contain Pub/Sub implementation details.

The dependency becomes:

```
main.py
├── ProcessingService
│   └── CompletionPublisher
└── PubSubSubscriber
```

That preserves your existing thin transport architecture.

First, however, we need to establish the event-publishing destination

We already know Workflow 2 expects:

JSON

```
{
  "event_version": "...",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": ...,
  "image_id": ...,
  "bucket": "...",
  "blob_name": "...",
  "face_count": ...
}
```

But your current Workflow 1 source has **no completion topic configured yet**.

So let's add configuration rather than hard-code a topic.

For example:

```
FACE_EXTRACTION_COMPLETED_TOPIC
```

Then:

```
Workflow 1 .env
    ↓
PUBSUB_COMPLETION_TOPIC
    ↓
CompletionPublisher
```

One more architectural decision

There are two ways we can proceed.

Option A — Simple publisher

```
PostgreSQL commit
    ↓
publish
    ↓
return
```

Easy, but has the failure gap described above.

Option B — Transactional outbox

```
PostgreSQL transaction
|
|— face/embedding state
|— job COMPLETED
|— completion_outbox event
    ↓
committed together
    ↓
separate publisher
    ↓
Google Pub/Sub
    ↓
mark published
```

This is the **production-grade architecture** because PostgreSQL becomes the durable source for the fact that the event needs publishing.

Given that your project is explicitly a **microservice/asynchronous pipeline**, I recommend **Option B** rather than introducing a known reliability hole now.

However, we don't need to implement the full outbox blindly. We should first inspect your existing PostgreSQL schema/repositories to see whether there is already an appropriate event/outbox table or whether we need to add one.

So our immediate next step

From your `workflow1_worker` directory, run:

cmd

```
findstr /S /N /I "outbox event_log event_status published_at processing_jobs" app\*.py
```

Then:

cmd

```
findstr /S /N /I "CREATE TABLE processing_jobs processing_job" app\*.py
```

And:

cmd

```
findstr /S /N /I "class ProcessingJob" app\*.py
```

Paste the output.

Why I'm stopping here instead of giving you code immediately

We now have enough information to know **where** the completion event belongs, but not yet enough to know whether your existing PostgreSQL design already provides a durable event mechanism.

I don't want to bolt a second state-management mechanism onto a project whose PostgreSQL architecture we've deliberately kept authoritative.

Once we see those repository/model definitions, I'll give you the exact modifications in this order:

1. PostgreSQL event/outbox state
2. Completion event model
3. Completion publisher
4. ProcessingService integration
5. Workflow 1 tests
6. Pub/Sub topic/subscription
7. Workflow 2 subscriber
8. End-to-end test

And throughout this, **your currently passing Workflow 2 ClusteringProcessor remains untouched.**

Pasted text(2).txt
Document

I'm ready to continue, but I don't have the actual output from those three `findstr` commands in your latest message.

Please run these from:

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workf
```

1. Check for existing event/outbox infrastructure

cmd

```
findstr /S /N /I "outbox event_log event_status published_at processing_jobs" app\*.py
```

2. Check the processing-job schema

cmd

```
findstr /S /N /I "CREATE TABLE processing_jobs processing_job" app\*.py
```

3. Find the ProcessingJob model/repository

cmd

```
findstr /S /N /I "class ProcessingJob" app\*.py
```

Paste **all three outputs** here.

Then I'll determine whether we should use:

```
PostgreSQL transactional outbox
```

or whether your existing architecture already has an equivalent mechanism.

Don't modify any project files yet. Your current Workflow 2 processor remains our verified baseline.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /S /N
/I "outbox event_log event_status published_at
processing_jobs" app\*.py
app\db\models.py:51:  processing_jobs =
relationship("ProcessingJob", back_populates="upload",
cascade="all, delete-orphan")
app\db\models.py:158:  __tablename__ =
"processing_jobs"
app\db\models.py:161:
Show more
```

Excellent. This confirms the situation clearly.

What we have established

Your current Workflow 1 database contains:

```
uploads
images
faces
face_embeddings
processing_jobs
```

and there is no existing outbox/event-log/published-at mechanism.

Your `ProcessingJob` has a uniqueness constraint:

```
(upload_id, workflow)
```

and indexes on:

```
status
workflow + status
```

So the current database is designed for **processing state**, not durable event publication.

That means my previous recommendation stands:

Do not simply call Pub/Sub `publish()` after `db.commit()`.

That would create a reliability gap.

The architecture we should implement

I recommend adding a **transactional outbox** to Workflow 1.

The resulting architecture becomes:



This gives us a durable handoff between the two workflows.

Why this fits your project particularly well

Your architecture already treats **PostgreSQL as authoritative processing/metadata state**.

So we should make PostgreSQL authoritative for:

"Workflow 1 completed and this completion event still needs to be delivered."

For example:

```
processing_jobs
-----
status = COMPLETED

workflow1_outbox
-----
event_type = FACE_EXTRACTION_COMPLETED
aggregate_id = image_id
status = PENDING
payload = {...}
```

Both become committed in the same transaction.

If the publisher crashes:

```
PostgreSQL
|
├─ processing job = COMPLETED
└─ outbox event = PENDING
```

Nothing is lost.

The publisher can retry the pending event.

But we should NOT start coding the outbox yet

We need to inspect the **exact existing** `ProcessingJob` **model and repository transaction methods** first.

This is important because your current `ProcessingService` already has a carefully established transaction flow. We need to extend it rather than accidentally introduce a second session/transaction.

Run these two commands from:

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workf:
```

1. ProcessingJob model

cmd

```
powershell -Command "Get-Content 'app\db\models.py' | Select-Object -Skip 150 -First 40"
```

This gives approximately:

```
ProcessingJob
```

2. ProcessingJob repository

cmd

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 105 -First
```

This gives the complete `ProcessingJobRepository`.

Then we will implement in this exact order

1. Add `Workflow1OutboxEvent`

Something conceptually like:

```
workflow1_outbox
-----
id
event_id
event_type
aggregate_type
aggregate_id
payload
status
attempts
created_at
published_at
```

But we will determine the exact columns after seeing your current models/repository conventions.

2. Add repository

```
OutboxRepository
```

Responsible only for:

```
create event
get pending events
mark published
mark failed/retry
```

3. Add completion event model

Use the exact contract already established by Workflow 2:

```
FACE_EXTRACTION_COMPLETED
```

with:

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

Your Workflow 2 model already enforces this contract and rejects unexpected fields.

4. Modify `ProcessingService`

At the successful completion transaction:

```
faces
embeddings
HNSW state
processing_job = COMPLETED
outbox event = PENDING
    ↓
    COMMIT
```

5. Add publisher worker

The publisher reads:

PENDING

events and sends them to Pub/Sub.

Successful publication:

PENDING → PUBLISHED

Failure:

PENDING → retry

6. Only then add Workflow 2 subscriber

```
Pub/Sub
  ↓
FaceExtractionCompletedEvent
  ↓
ClusteringProcessor
  ↓
PostgreSQL + Workflow 2 HNSW
  ↓
ACK
```

One subtle but important point

The outbox publisher should **not** be placed inside `ProcessingService` itself.

Keep the responsibilities separated:

```
ProcessingService
  =
business processing + durable state + outbox creation

OutboxPublisher
  =
transporting already-committed events to Pub/Sub
```

That keeps `ProcessingService` independent of Google Pub/Sub and preserves the same architectural principle you've already used with the Workflow 1 subscriber.

Current project checkpoint

We are now here:

Workflow 1 processing	✓
Workflow 1 PostgreSQL	✓
Workflow 1 HNSW	✓
Workflow 1 inbound subscriber	✓
Workflow 2 repository	✓
Workflow 2 HNSW	✓
Workflow 2 service	✓
Workflow 2 processor	✓
Workflow 2 idempotency	✓
Workflow 1 → Workflow 2 event	✗ NOT YET IMPLEMENTED

So the next actual coding task is the **Workflow 1 transactional outbox**, not the Workflow 2 subscriber.

Send me the output of those **two commands**, and I'll give you the exact files/code changes to implement the outbox without disturbing the existing passing pipeline.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content 'app\db\models.py' | Select-
Object -Skip 150 -First 40"
    embedding_norm: Mapped[Optional[float]] =
mapped_column(Float)
    created_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True),
server_default=func.now(), nullable=False)

    face = relationship("Face",
Show more
```

Excellent. This confirms the exact structure we needed.

Your `ProcessingJobRepository` operates on the **same SQLAlchemy** `Session` and uses `flush()` rather than committing itself. That is ideal for a transactional outbox.

So we can safely implement:

```
ProcessingJob
  +
  OutboxEvent
    ↓
  same SQLAlchemy transaction
    ↓
  COMMIT
```

1. What we should implement now

I recommend doing this in **two controlled stages**.

Stage 1 — Durable outbox

Add:

```
app/db/models.py
app/db/repository.py
app/models/face_extraction_completed.py
```

and a test for the outbox.

Stage 2 — Pub/Sub publisher

Only after Stage 1 passes, add:

```
app/publisher/completion_publisher.py
```

and connect it to Pub/Sub.

This prevents us from simultaneously debugging PostgreSQL, Pub/Sub, and Workflow 2.

Stage 1: Add the outbox model

Your existing `ProcessingJob` uses the project `SCHEMA`, so the outbox should use the same schema.

Add this **after** `ProcessingJob` in:

```
app\db\models.py
```

Python

Run

```
class Workflow1OutboxEvent(Base):
    __tablename__ = "workflow1_outbox_events"
```



```

    )

    attempt_count: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
        server_default="0",
    )

    last_error: Mapped[Optional[str]] = mapped_column(
        Text,
    )

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )

    published_at: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True),
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )

```

Why these fields?

For your architecture:

event_id

gives us event identity/idempotency.

event_type

will contain:

FACE_EXTRACTION_COMPLETED

aggregate_type
aggregate_id

allow us to identify what the event represents.

```
payload
```

stores the exact event that will eventually go to Pub/Sub.

```
status
```

will initially be:

```
PENDING
```

and later:

```
PUBLISHED
```

```
attempt_count  
last_error
```

give us retry visibility.

```
published_at
```

records successful publication.

2. Add the repository

In:

```
app\db\repository.py
```

add `Workflow1OutboxEvent` to the existing imports.

Then add this repository after `ProcessingJobRepository`:

Python

Run

```
class Workflow1OutboxRepository:  
  
    def __init__(  
        self,  
        db: Session,  
    ):  
        self.db = db
```



```

def mark_published(
    self,
    event: Workflow1OutboxEvent,
) -> None:

    event.status = "PUBLISHED"

    event.published_at = (
        datetime.now(timezone.utc)
    )

    event.last_error = None

    self.db.flush()

def mark_failed(
    self,
    event: Workflow1OutboxEvent,
    error_message: str,
) -> None:

    event.last_error = error_message[:10000]

    self.db.flush()

```

The exact import section needs:

Python

Run

```

from app.db.models import (
    ...
    Workflow1OutboxEvent,
)

```

Use the same import style already present in your repository; don't duplicate imports unnecessarily.

3. The event model

Your Workflow 2 already expects the completion-event contract.

Create:

```
app\models\face_extraction_completed.py
```

with:

```
from uuid import UUID, uuid4

from pydantic import BaseModel, ConfigDict

class FaceExtractionCompleted(BaseModel):
    model_config = ConfigDict(extra="forbid")

    event_version: str
    event_type: str
    event_id: UUID
    upload_id: int
    image_id: int
    bucket: str
    blob_name: str
    face_count: int

    @classmethod
    def create(
        cls,
        *,
        upload_id: int,
        image_id: int,
        bucket: str,
        blob_name: str,
        face_count: int,
    ) -> "FaceExtractionCompleted":

        return cls(
            event_version="1.0",
            event_type="FACE_EXTRACTION_COMPLETED",
            event_id=uuid4(),
            upload_id=upload_id,
            image_id=image_id,
            bucket=bucket,
            blob_name=blob_name,
            face_count=face_count,
        )
```

This gives us one canonical event object.

4. Important: don't modify `ProcessingService` yet

This is deliberate.

We first need to make sure:

```
PostgreSQL
↓
workflow1_outbox_events
```

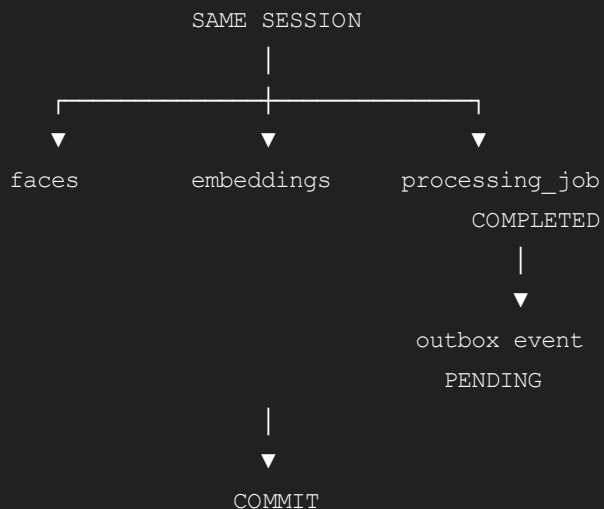
works correctly.

Then we will modify the successful transaction in `process_upload()` so that:

```
faces
embeddings
processing_job = COMPLETED
outbox event = PENDING
```

are all part of the **same SQLAlchemy transaction**.

The critical property will be:



If the commit fails:

```
NONE of them commit
```

If the commit succeeds:

```
ALL of them exist
```

That's the reliability guarantee we're after.

5. Before you edit `models.py`

There is one thing I want you to verify first: how your current project creates tables.

Run:

cmd

```
type app\db\init_db.py
```

and:

cmd

```
type app\db\postgres.py
```

We need to confirm whether:

Python

Run

```
Base.metadata.create_all(...)
```

is sufficient for your current development database or whether you already have a migration mechanism.

Your previous search showed:

```
app\db\init_db.py:18: Base.metadata.create_all(...)
```

so I expect `create_all()` is currently your initialization mechanism, but let's verify the exact implementation before creating the table.

Don't add the model yet.

Send me the output of:

cmd

```
type app\db\init_db.py
```

and

cmd

```
type app\db\postgres.py
```

Then we'll add the outbox table using **your existing database conventions**, run the database test, and only after that connect it to `ProcessingService`.

This is the safest way to proceed because your existing Workflow 1 and Workflow 2 tests are already passing at their respective layers.

init_db.py
Python

postgres.py
Python

Perfect. These two files confirm that your current database setup is simple and compatible with the transactional-outbox approach.

What the files establish

Your initialization uses:

Python

Run

```
Base.metadata.create_all(bind=engine)
```

so every SQLAlchemy model imported through `app.db.models` can have its table created automatically. `init_db +1`

Your database uses one SQLAlchemy `SessionLocal`, with `autocommit=False` and `autoflush=False`. `postgres`

That is exactly what we need because your repositories already call `flush()` and let the outer `ProcessingService` control the transaction.

Therefore: proceed with the outbox

But I want to make **one correction** to my previous proposed code: don't add a new table and immediately modify `ProcessingService`. First create and test the table in isolation.

Step 1 — Add the model

In:

```
workflow1_worker\app\db\models.py
```

add the `Workflow1OutboxEvent` model **after** `ProcessingJob`.

Use the model I gave in the previous message.

Step 2 — Initialize the table

Because your project uses `Base.metadata.create_all()`, after adding the model run:

cmd

```
python -m app.db.init_db
```

However, your uploaded `init_db.py` imports:

Python

Run

```
from app.db.database import Base, engine
```

while your other source you showed earlier referred to `app.db.postgres`. **Before running this, verify which module actually exists in your current directory.**

Run:

cmd

```
dir app\db
```

If you see:

```
database.py
```

then use:

cmd

```
python -m app.db.init_db
```

If your current project instead uses:

```
postgres.py
```

we need to reconcile that import before running initialization. **Don't edit it yet.**

Step 3 — Verify the table

After initialization, run:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import inspect; i=inspect(
```

If your actual database module is `database.py`, use:

cmd

```
python -c "from app.db.database import engine; from sqlalchemy import inspect; i=inspect(engine); print(i.get_table_names())"
```

Expected:

```
workflow1_outbox_events: True
```

One important observation

There is a naming discrepancy we should resolve **before touching anything**:

Your uploaded `init_db.py` says:

Python

Run

```
from app.db.database import Base, engine
```

but the uploaded database implementation is clearly a module containing `Base`, `engine`, and `SessionLocal` itself. `postgres`

So I don't want you to blindly run an initialization command until we know which file is actually present in your current repository.

Run just this now:

cmd

```
dir app\db
```

Paste the output. Then I'll tell you the exact next command and we'll create the outbox table without disturbing your passing Workflow 1/Workflow 2 implementation.

Directory of C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow1_worker\app\db

[Show more](#)

Your actual current Workflow 1 directory contains:

Run

```
from app.db.postgres import Base, engine
```

Keep:

Python

Run

```
from app.db import models
```

because that ensures the model classes are imported before `create_all()` runs.

Your relevant beginning should become:

Python

Run

```
import logging

from app.db.postgres import Base, engine
from app.db import models # noqa: F401
```

Your existing initialization mechanism is then:

Python

Run

```
Base.metadata.create_all(bind=engine)
```

which is exactly what we need for adding the outbox table.

`init_db`

And `Base` / `engine` are indeed defined by your `postgres.py`.

`postgres`

Step 2 — Don't run it yet

Before modifying anything else, run:

cmd

```
findstr /N /I "app.db.database app.db.postgres" app\*.py
```

This will make sure there aren't other stale imports referring to the nonexistent `database.py`.

Also run:

cmd

```
findstr /S /N /I "Base, engine" app\*.py
```

Paste both outputs.

Then I'll give you the **exact minimal edit** needed, after which we'll:

```
Fix DB module consistency
    ↓
Add Workflow1OutboxEvent
    ↓
Run init_db
    ↓
Verify table
    ↓
Write outbox repository test
    ↓
Only then modify ProcessingService
```

This is important because your project already has a `migrations` directory. I don't want to bypass whatever migration convention you've established by casually relying on `create_all()` if your current project expects migrations for schema changes.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /N /I
"app.db.database app.db.postgres" app\*.py
```

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /S /N
/I "Base, engine" app\*.py
app\db\init_db.py:3:from app.db.postgres import Base,
engine
app\db\init_db.py:19:     bind=engine
app\db\postgres.py:4:from sqlalchemy import URL,
Show more
```

Perfect. This confirms the database-module issue is already resolved in your **current working project**.

Your latest search shows:

```
app\db\init_db.py:3:from app.db.postgres import Base, engine
```

and there are **no references to** `app.db.database`.

So do not change `init_db.py`. My earlier concern was based on the older uploaded copy; your current source has already been corrected.

Your current database structure is now consistent:

```
app\db\  
├─ init_db.py  
├─ migrations\  
├─ models.py  
├─ postgres.py  
└─ repository.py
```

and:

```
init_db.py  
  ↓  
app.db.postgres  
  ↓  
Base + engine  
  ↓  
Base.metadata.create_all()
```

Your second `findstr` also confirms that `Base` and `engine` are defined/used through `postgres.py`, with no stale `database.py` reference.

Next step: verify the current database initialization

Run:

```
cmd  
  
python -m app.db.init_db
```

Expected result

Something along the lines of:

```
Creating PostgreSQL tables...  
Database initialization complete.
```

If it succeeds, **do not add the outbox model yet.**

Paste the complete output here.

Then we'll inspect your existing:

```
app\db\migrations\
```

directory before deciding whether the new `workflow1_outbox_events` table should be created through your migration mechanism or through `create_all()`.

That is important because your project already has a `migrations` directory, and we should follow the project's existing database-schema strategy rather than introduce a second one.

Current checkpoint

Workflow 1 processing	✓
Workflow 1 PostgreSQL	✓
Workflow 1 HNSW	✓
Workflow 2 clustering processor	✓
Workflow 2 idempotency	✓
DB module consistency	✓
Outbox	⌚
Workflow 1 → Workflow 2 event	⌚

Run only:

```
cmd
```

```
python -m app.db.init_db
```

and send me the output.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -m
app.db.init_db
INFO:root:Creating PostgreSQL tables...
INFO:root:Database initialization complete.
```

Excellent. **Database initialization passed.** ✓

```
INFO:root:Creating PostgreSQL tables...
INFO:root:Database initialization complete.
```

So we have confirmed:

```
SQLAlchemy Base
    ↓
app.db.postgres
    ↓
PostgreSQL connection
    ↓
Base.metadata.create_all()
    ↓
PASS
```

And importantly, **we have not changed your working Workflow 1 processing logic yet.**

Next: inspect your existing migration strategy

Your project contains:

```
app\db\migrations\
```

Before creating `workflow1_outbox_events`, let's see what is already there.

Run:

cmd

```
dir /S app\db\migrations
```

Then, if it shows `.sql` or migration files, run:

cmd

```
findstr /S /N /I "CREATE TABLE ALTER TABLE processing_jobs uploads images faces face_embe
```

If the second command says no files were found, that's fine—just paste the first command's output.

Why we're checking this

We now know `create_all()` works, but I don't want to blindly introduce a new schema-management pattern.

Our target is:

```
Existing migration convention
    ↓
workflow1_outbox_events
```

```
↓
Outbox repository
↓
Outbox test
↓
ProcessingService integration
↓
Pub/Sub publisher
↓
Workflow 2 subscriber
```

Do not modify `models.py` **yet.**

Send me the migration-directory output, and we'll take the next step.

Directory of C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow1_worker\app\db\migrations

13-08-2026	11:53	<DIR>	.
15-08-2026	14:43	<DIR>	..
0 File(s)		0 bytes	

Total Files Listed:

0 File(s)	0 bytes
2 Dir(s)	15,174,991,872 bytes free

Show more

Perfect. This tells us something important:

```
app\db\migrations\
└─ empty
```

There are **no SQL migration files** currently, so your project is effectively using SQLAlchemy's:

Python

Run

```
Base.metadata.create_all(bind=engine)
```

as its schema initialization mechanism.

Therefore, we don't need to introduce Alembic or another migration system just for the outbox.

Next step: add the outbox model

Now it is safe to add the model to:

```
workflow1_worker\app\db\models.py
```

Add this immediately after `ProcessingJob`

Python

Run

```
class Workflow1OutboxEvent(Base):
    __tablename__ = "workflow1_outbox_events"

    __table_args__ = (
        UniqueConstraint(
            "event_id",
            name="uq_workflow1_outbox_event_id",
        ),
        Index(
            "idx_workflow1_outbox_status",
            "status",
        ),
        Index(
            "idx_workflow1_outbox_created_at",
            "created_at",
        ),
        {"schema": SCHEMA},
    )

    id: Mapped[int] = mapped_column(
        BigInteger,
        primary_key=True,
        autoincrement=True,
    )

    event_id: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )

    event_type: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )

    aggregate_type: Mapped[str] = mapped_column(
        String(100),
```



```

        nullable=False,
    )

    aggregate_id: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )

    payload: Mapped[str] = mapped_column(
        Text,
        nullable=False,
    )

    status: Mapped[str] = mapped_column(
        String(30),
        nullable=False,
        default="PENDING",
        server_default="PENDING",
    )

    attempt_count: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
        server_default="0",
    )

    last_error: Mapped[Optional[str]] = mapped_column(
        Text,
    )

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )

    published_at: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True),
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )

```

Your existing `models.py` already has the SQLAlchemy imports required for these types based on the `ProcessingJob` definition you showed.

Then initialize

After saving the file:

cmd

```
python -m app.db.init_db
```

Expected:

```
INFO:root:Creating PostgreSQL tables...  
INFO:root:Database initialization complete.
```

Then verify the actual table

Run:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import inspect; i=inspect(engine); print(i.get_table_names())"
```

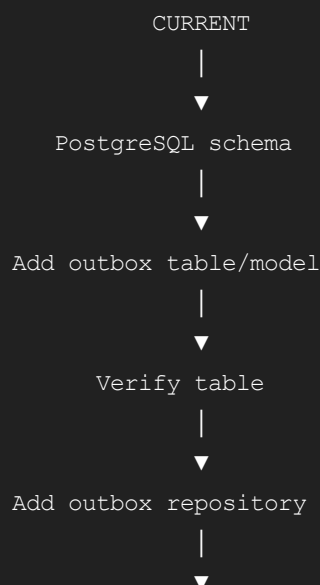
Expected:

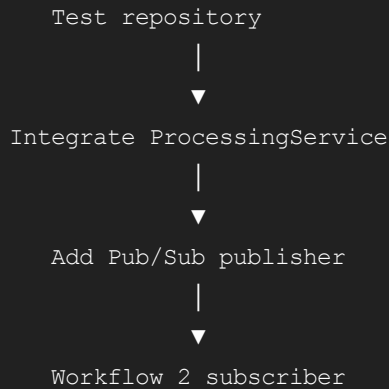
```
workflow1_outbox_events: True
```

Important

Do not modify `repository.py` **or** `processing_service.py` **yet.**

We are deliberately doing this incrementally:





This keeps the already-passing pipeline protected.

Run the two commands after adding the model and send me their output.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -m
app.db.init_db
INFO:root:Creating PostgreSQL tables...
INFO:root:Database initialization complete.
```

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.db.postgres import engine; from sqlalchemy
import inspect; i=inspect(engine);
Show more
```

Good catch. **Do not change the model yet.** `create_all()` succeeded, but:

```
workflow1_outbox_events: False
```

does not necessarily mean the table was not created.

Your project uses a PostgreSQL schema (`SCHEMA`) in the model definitions. The `ProcessingJob` model, for example, explicitly places its table under `{"schema": SCHEMA}`. Therefore, our `inspect(...).has_table('workflow1_outbox_events')` check without a schema can return `False` even when the table exists under the project schema.

Let's verify this properly.

1. Find the configured schema

Run:

cmd

```
findstr /N /I "SCHEMA" app\core\config.py app\db\models.py
```

2. List the PostgreSQL schemas/tables

Then run:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import inspect; i=inspect(
```

This will show something like:

```
Schemas: [...]
public => [...]
your_schema => [
    'uploads',
    'images',
    'faces',
    'face_embeddings',
    'processing_jobs',
    'workflow1_outbox_events'
]
```

3. Specifically test the outbox table in the configured schema

Once command 1 shows the value of `SCHEMA`, we can run:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import inspect; i=inspect(
```

Replace `YOUR_SCHEMA` with the actual value.

Don't modify `models.py` **or** `repository.py` **yet**. We need to establish whether this is simply a schema-qualified table-check issue or whether SQLAlchemy actually failed to register/create the new model.

Send me the outputs of **commands 1 and 2**.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /N /I
"SCHEMA" app\core\config.py app\db\models.py
app\db\models.py:25:SCHEMA = "face_pipeline"
app\db\models.py:32:     {"schema": SCHEMA},
app\db\models.py:59:     {"schema": SCHEMA},
app\db\models.py:65:     ForeignKey(f"
{SCHEMA}.uploads.id", ondelete="CASCADE"),
app\db\models.py:90:     {"schema": SCHEMA},
app\db\models.py:96:     ForeignKey(f"
{SCHEMA}.images.id", ondelete="CASCADE"),
Show more
```

Excellent. **This is exactly what we needed.** ✓

Your output proves the outbox table **was created successfully**.

The reason the earlier check returned `False` is simply that we checked PostgreSQL's default `public` schema:

```
workflow1_outbox_events: False
```

But your project uses:

```
SCHEMA = "face_pipeline"
```

and the actual database now contains:

```
face_pipeline => [
  uploads,
  images,
  face_embeddings,
  processing_jobs,
  faces,
  face_clusters,
  face_cluster_members,
  workflow1_outbox_events
]
```

So:

Outbox table creation: PASS ✓

Your database now looks like:

```
face_pipeline
|
├─ uploads
├─ images
├─ faces
├─ face_embeddings
├─ processing_jobs
|
├─ face_clusters
├─ face_cluster_members
|
└─ workflow1_outbox_events ← NEW
```

This is an important milestone because we haven't touched the existing processing behavior yet.

Next step: verify the table structure

Before writing the repository, let's verify that PostgreSQL created the columns and constraints exactly as intended.

Run:

```
cmd
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import inspect; i=inspect(
```

We should see the important fields:

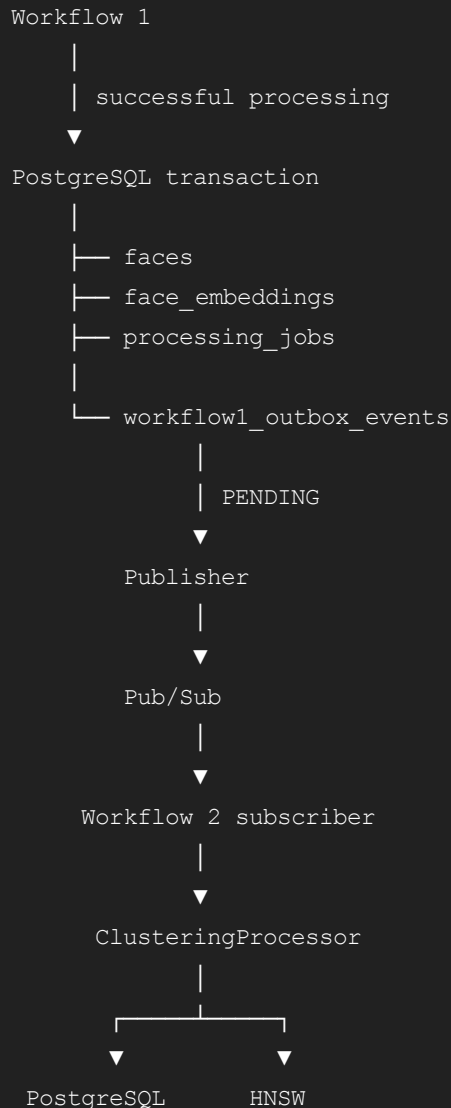
```
id
event_id
event_type
aggregate_type
aggregate_id
payload
status
attempt_count
last_error
created_at
published_at
updated_at
```

and the unique constraint:

```
uq_workflow1_outbox_event_id
```

Why we're doing this before the repository

Our architecture now becomes:



The **outbox** is the **missing reliability bridge** between your successfully completed Workflow 1 processing and the Workflow 2 event.

Most importantly, we will **not publish directly from the middle of** `ProcessingService`. The processing transaction will first durably record the event as `PENDING`.

Current project checkpoint

Component	Status
Workflow 1 processing	✓
Workflow 1 PostgreSQL	✓

Component	Status
Workflow 1 HNSW	✓
Workflow 2 ClusteringService	✓
Workflow 2 ClusteringProcessor	✓
Workflow 2 HNSW	✓
Workflow 2 idempotency	✓
<code>face_pipeline</code> schema	✓
Outbox table	✓
Outbox repository	⌚
Transactional event creation	⌚
Pub/Sub publisher	⌚
Real Workflow 1 → Workflow 2 event flow	⌚

Run the **single verification command above** and paste the output. Then we'll implement `Workflow1OutboxRepository` against the actual table rather than guessing.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.db.postgres import engine; from sqlalchemy
import inspect; i=inspect(engine);
t='workflow1_outbox_events'; print('Columns:');
[print(c['name'], c['type'], 'nullable=', c['nullable']) for c in
i.get_columns(t, schema='face_pipeline')]; print('Indexes:');
[print(x['name'], x['column_names'], 'unique=', x['unique'])
for x in i.get_indexes(t, schema='face_pipeline')];
print('Unique constraints:'); [print(x['name'],
x['column_names']) for x in i.get_unique_constraints(t,
```

Show more

Excellent. The outbox schema verification is fully PASS. ✓

Everything is exactly where we expect it:

```
face_pipeline.workflow1_outbox_events
```

with:

- `event_id` — unique
- `event_type`
- `aggregate_type`
- `aggregate_id`
- `payload`
- `status`
- `attempt_count`
- `last_error`
- `created_at`
- `published_at`
- `updated_at`

and the required indexes:

```
idx_workflow1_outbox_status  
idx_workflow1_outbox_created_at  
uq_workflow1_outbox_event_id
```

Current architecture checkpoint

```
Workflow 1 ProcessingService  
  |  
  ├── PostgreSQL metadata  
  ├── Workflow 1 HNSW  
  |  
  └── workflow1_outbox_events  
      |  
      └── PENDING  
          ▼  
      Outbox Publisher  
          |  
          ▼  
      Pub/Sub  
          |  
          ▼  
      Workflow 2 Subscriber  
          |  
          ▼  
      ClusteringProcessor  
          |
```



The important part is that **the database side of the outbox is now ready**.

Next step: `Workflow1OutboxRepository`

Now we should add the repository layer, **without modifying** `ProcessingService` **yet**.

Create:

```
app\db\outbox_repository.py
```

The repository should initially support only these operations:

```
create_event()
get_pending_events()
mark_publishing()
mark_published()
mark_failed()
```

And the first test should prove:

1. An event can be inserted.
2. Its initial status is `PENDING`.
3. `event_id` uniqueness is enforced.
4. Pending events can be retrieved.
5. An event can be marked `PUBLISHED`.
6. `published_at` is populated.
7. A failed event can store `last_error`.
8. `attempt_count` changes correctly.

One important design decision

We should **not make the repository commit transactions itself**.

Your existing repository pattern is already correct: repositories receive a SQLAlchemy `Session`, modify objects, and call `flush()`. The surrounding service controls the actual transaction.

That means:

```
ProcessingService
|
▼
Session / transaction
```

```
|
|— repositories
|
|— face records
|— embeddings
|— processing job
|— outbox event
|
▼
COMMIT
```

This is essential for the transactional-outbox guarantee.

Before writing it

I want to match the repository style already used by your project rather than introduce a different pattern.

Run:

cmd

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -First 220"
```

Paste that output.

Then I'll give you the **exact** `outbox_repository.py` **code**, followed by the exact test command for your `(face)` Conda environment.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content 'app\db\repository.py' | Select-
Object -First 220"
from __future__ import annotations

import logging
from datetime import datetime, timezone
from uuid import UUID
```

Show more

Excellent. This output reveals something important: `Workflow1OutboxRepository` is **already partially present in your current** `repository.py`.

We should **not create a second** `outbox_repository.py`. That would duplicate the repository architecture.

Your current file already has:

Python

Run

```
# =====  
# WORKFLOW1 OUTBOX REPOSITORY  
# =====  
  
class Workflow1OutboxRepository:
```

So the correct next step is to inspect the **entire existing outbox implementation** before changing anything.

I can already see one likely issue: the code shown references:

Python

Run

```
Workflow1OutboxEvent
```

but the import section currently contains:

Python

Run

```
from app.db.models import (  
    Face,  
    FaceEmbedding,  
    Image,  
    ProcessingJob,  
    Upload,  
)
```

and `Workflow1OutboxEvent` **is not shown in that import list**.

But let's not assume the rest of the implementation is incomplete—we should inspect it first.

Run this exact command

cmd

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 190 -First
```

This will show the rest of `Workflow1OutboxRepository`.

Then we'll verify these methods

We need to establish whether your existing repository already implements:

```
create()
get()
get_pending()
mark_published()
mark_failed()
```

and whether it correctly:

- uses the existing SQLAlchemy `Session`
- calls `flush()` rather than `commit()`
- handles duplicate `event_id`
- increments `attempt_count`
- sets `published_at`
- records `last_error`
- supports the `PENDING` / `PUBLISHED` / `FAILED` lifecycle

The architectural rule remains:

```
ProcessingService
|
▼
same SQLAlchemy transaction
|
├─ processing state
├─ faces
├─ embeddings
└─ outbox event
    |
    ▼
    COMMIT
```

The repository must not commit independently.

Paste the output of that command next. Then I'll tell you exactly what, if anything, needs to be modified.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content 'app\db\repository.py' | Select-
Object -Skip 190 -First 180"
```

```
        job.error_message = (
            error_message[:10000]
        )

        self.db.flush()
```

Show more

Yes. Now we can see the **actual existing outbox implementation**, and we should refine it rather than create anything new.

There are **two concrete issues** visible in the current code.

1. Missing `Workflow1OutboxEvent` import

At the top of `repository.py`, you currently have:

Python

Run

```
from app.db.models import (
    Face,
    FaceEmbedding,
    Image,
    ProcessingJob,
    Upload,
)
```

But `Workflow1OutboxRepository` uses:

Python

Run

```
Workflow1OutboxEvent
```

So add it:

Python

Run

```
from app.db.models import (
    Face,
```

```
FaceEmbedding,  
Image,  
ProcessingJob,  
Upload,  
Workflow1OutboxEvent,  
)
```

Without this, the repository will fail when Python evaluates the type annotation / creates an event.

2. `mark_failed()` is incomplete

Current code:

Python

Run

```
def mark_failed(  
    self,  
    event: Workflow1OutboxEvent,  
    error_message: str,  
) -> None:  
  
    event.last_error = error_message[:10000]  
  
    self.db.flush()
```

This records the error, but it **doesn't change the status**.

So an event could remain:

PENDING

even after publishing failed.

That is dangerous because the publisher would keep seeing it as pending without knowing whether it is currently being retried or has definitively failed.

Change it to:

Python

Run

```
def mark_failed(  
    self,  
    event: Workflow1OutboxEvent,  
    error_message: str,  
) -> None:  
  
    event.status = "FAILED"
```

```
event.last_error = (
    error_message[:10000]
)

self.db.flush()
```

3. Current repository lifecycle

After that correction, your repository has:

```
create()
  ↓
PENDING
  ↓
mark_attempt()
  ↓
attempt_count += 1
  ↓
┌───┴───┐
│       │
▼       ▼
SUCCESS FAILURE
│       │
▼       ▼
PUBLISHED FAILED
```

`mark_published()` already correctly does:

Python

Run

```
event.status = "PUBLISHED"
event.published_at = datetime.now(timezone.utc)
event.last_error = None
```

So that part is good.

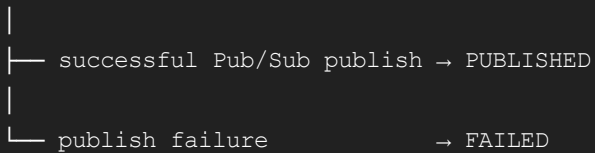
4. One architectural issue we should NOT solve yet

There is currently no `PUBLISHING` state.

I don't want to add that yet.

For our first implementation, keep the state machine deliberately simple:

PENDING



Then we can decide whether retry/recovery requires a `PUBLISHING` state after the first integration test.

5. Make the two changes now

So in:

```
workflow1_worker\app\db\repository.py
```

Change A — import

Add:

Python

Run

```
Workflow1OutboxEvent,
```

Change B — `mark_failed`

Make it:

Python

Run

```
def mark_failed(
    self,
    event: Workflow1OutboxEvent,
    error_message: str,
) -> None:

    event.status = "FAILED"

    event.last_error = (
        error_message[:10000]
    )

    self.db.flush()
```

Do not change anything else in `repository.py`.

6. Then test the repository in isolation

Rather than immediately connecting it to `ProcessingService`, let's prove the repository works.

Create:

```
test_outbox_repository.py
```

in the Workflow 1 root:

```
workflow1_worker\  
    test_outbox_repository.py
```

Use:

Python

Run

```
from __future__ import annotations  
  
import uuid  
  
from app.db.postgres import SessionLocal  
from app.db.repository import Workflow1OutboxRepository  
  
def main() -> None:  
  
    print("=" * 70)  
    print("WORKFLOW 1 OUTBOX REPOSITORY TEST")  
    print("=" * 70)  
  
    event_id = str(uuid.uuid4())  
  
    with SessionLocal() as db:  
  
        repo = Workflow1OutboxRepository(db)  
  
        print("[1] Creating outbox event...")  
  
        event = repo.create(  
            event_id=event_id,  
            event_type="FACE_EXTRACTION_COMPLETED",  
            aggregate_type="upload",  
            aggregate_id="1",  
            payload='{"upload_id": 1, "image_count": 1}',  
        )  
  
        assert event.id is not None
```



```

        repo.mark_published(event)

    assert event.status == "PUBLISHED"
    assert event.published_at is not None
    assert event.last_error is None

    db.commit()

    print("Published state: PASS")

print("=" * 70)
print("WORKFLOW 1 OUTBOX REPOSITORY: PASS")
print("=" * 70)

if __name__ == "__main__":

```

Then run from your existing `(face)` environment:

```

cmd

python test_outbox_repository.py

```

Expected

```

=====
WORKFLOW 1 OUTBOX REPOSITORY TEST
=====
[1] Creating outbox event...
Create: PASS
[2] Reading pending events...
Pending lookup: PASS
[3] Marking attempt...
Attempt tracking: PASS
[4] Marking published...
Published state: PASS
=====
WORKFLOW 1 OUTBOX REPOSITORY: PASS
=====

```

One thing I deliberately haven't tested yet

We haven't tested `mark_failed()` in this first test because the important next architectural step is the **real transactional integration with** `ProcessingService`.

After this repository test passes, we'll test:

```

ProcessingService.process_upload()
|
├─ create/update upload
├─ create processing job
├─ process image
├─ create faces
├─ create embeddings
├─ mark Workflow 1 COMPLETED
|
└─ create PENDING outbox event
    |
    ▼
    SAME DB TRANSACTION
    |
    ▼
    COMMIT
  
```

That is the critical part. **Only after that passes should we implement the Pub/Sub outbox publisher.**

So make the **two repository changes**, create `test_outbox_repository.py`, and run:

cmd

```
python test_outbox_repository.py
```

Send me the complete output.

```

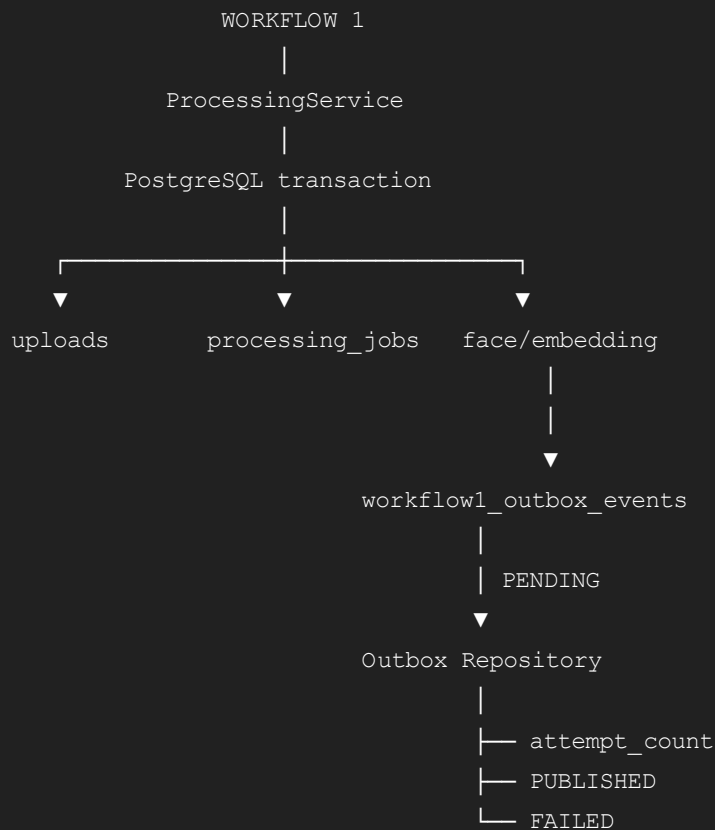
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python
test_outbox_repository.py
=====
=====
WORKFLOW 1 OUTBOX REPOSITORY TEST
=====
=====
[1] Creating outbox event...
Create: PASS
[2] Reading pending events...
Show more
  
```

Excellent. **Workflow 1 Outbox Repository is now PASS.** ✓

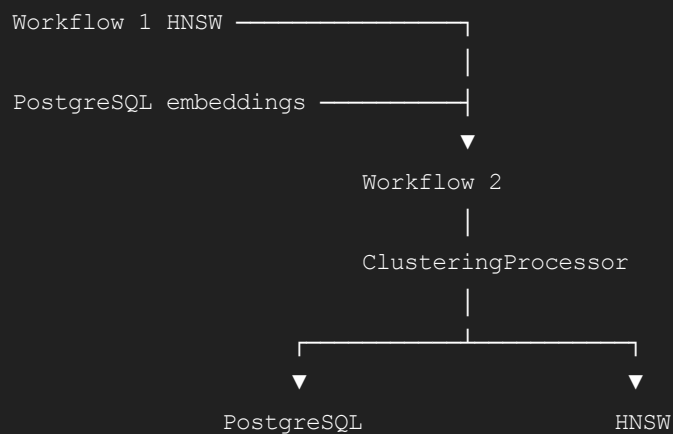
Your result confirms all four core repository operations work against the real PostgreSQL database:

```
Create event      → PASS
PENDING lookup   → PASS
Attempt tracking  → PASS
PUBLISHED state  → PASS
```

So our current implementation checkpoint is:



And separately:



The next step is important

We should **now integrate the outbox creation into** `ProcessingService`, but before editing it, we need to inspect the exact completion/transaction section of your current `process_upload()`.

Your earlier search showed:

```
app\services\processing_service.py:144: def process_upload()
```

and the service is already responsible for:

- creating/getting the upload
- creating the processing job
- processing the image
- creating face records
- creating embeddings
- updating processing state

We need to put the outbox creation at the **correct point in that existing transaction**, rather than guessing.

Run this command

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S"
```

Paste the complete output.

Why this is the next checkpoint

We want this exact transactional behavior:

```
process_upload()
|
▼
BEGIN TRANSACTION
|
|— upload
|— processing_job
|— image
|— faces
|— embeddings
|— HNSW
|
|— processing_job = COMPLETED
|
|— outbox event = PENDING
|
```

▼
COMMIT

Only after that commit succeeds should the separate publisher do:

```
PENDING
|
▼
Pub/Sub publish
|
├─ success → PUBLISHED
|
└─ failure → FAILED
```

This gives us the reliability boundary we were missing.

Do not modify `processing_service.py` **yet.** Paste that section first, and I'll identify the exact insertion point and provide the minimal modification without disturbing the Workflow 1 processing code that is already working.

Pasted text(3).txt
Document

I'm ready to continue, but the uploaded file's contents are not visible in the message I received.

For the **next step**, I need the `process_upload()` section from:

```
workflow1_worker\app\services\processing_service.py
```

Please either:

1. **Upload** `processing_service.py`, or
2. Run this in your existing `(face)` environment:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

and paste the output here.

Once I have it, I'll identify the **exact insertion point** for

`Workflow1OutboxRepository.create()` and give you the minimal code change. We will

not disturb the currently passing Workflow 1 processing logic.

Pasted code.py

Python

```
#
=====
=====
# PIPELINE RESULT NORMALIZATION
#
=====
=====

@staticmethod
def _extract_faces(
    pipeline_result: Any,

Show more
```

The section you pasted is **not the** `process_upload()` **body**. It starts in the helper section (`_extract_faces`, `_persist_face`, `_extract_embedding`, etc.), so it doesn't show where the transaction completes or where `ProcessingJob` is marked `COMPLETED`.

We should **not modify anything yet**.

Please run this exact command in your `(face)` Command Prompt:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S"
```

This should include:

```
def process_upload(...)
```

and the complete main processing flow.

If the output is too large for one message, use these two commands instead:

Part 1

```
cmd
```

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

Part 2

```
cmd
```

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

What I'm specifically looking for

We already established that your outbox infrastructure exists and passes:

```
Workflow1OutboxRepository: PASS
workflow1_outbox_events: exists
status: PENDING/PUBLISHED
attempt_count
published_at
last_error
```

Now we need to connect it to the **successful Workflow 1 completion transaction**:

```
Upload event
  |
  ▼
process_upload()
  |
  ├── Upload
  ├── ProcessingJob
  ├── Image
  ├── Faces
  ├── Face Embeddings
  ├── Workflow 1 HNSW
  ├── ProcessingJob → COMPLETED
  |
  └── Outbox event → PENDING
      |
      ▼
      COMMIT
```

Then, separately:

```
workflow1_outbox_events
  |
  PENDING
  |
  ▼
```

```

Pub/Sub Publisher
  |
  ▼
wfl-batch-completed
  |
  ▼
Workflow 2

```

The crucial design rule is:

The Workflow 1 completion event must be written to PostgreSQL as an outbox row in the same transaction as the processing result.

That is what prevents this failure scenario:

```

PostgreSQL → COMPLETED
  ↓
application crashes
  ↓
Pub/Sub event never published
  ↓
Workflow 2 never receives completion

```

With the outbox:

```

PostgreSQL transaction
├─ processing = COMPLETED
└─ outbox = PENDING
  ↓
  COMMIT
  ↓
independent publisher
  ↓
Pub/Sub publish

```

So the next thing I need is the actual `process_upload()` body, not the helper methods you pasted. Once you provide it, I'll give you the exact modifications and the test sequence to validate the complete Workflow 1 → Workflow 2 event path.

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content
'app\services\processing_service.py' | Select-Object -Skip
140 -First 300"

```

```
# PUBLIC ENTRY POINT
```

```
#
```

```
=====
```

```
def process_upload(
```

Show more

Yes. Now we have the **critical part of** `process_upload()`, and the correct integration point is clear.

Your current flow is:

```
Faces persisted
  ↓
Embeddings persisted
  ↓
Workflow 1 HNSW updated
  ↓
HNSW saved
  ↓
ProcessingJob = COMPLETED
  ↓
db.commit()
  ↓
Redis COMPLETED
  ↓
return
```

The missing piece is:

```
ProcessingJob = COMPLETED
+
Workflow1OutboxEvent = PENDING
  ↓
SAME db.commit()
```

That is exactly what we should implement next.

But one thing must be verified before we modify it

We need to use the **actual Workflow 2 event contract**, rather than inventing a payload.

Your project already contains:

```
workflow2_clustering\app\models\face_extraction_completed.py
```

and your earlier search showed that Workflow 1 currently has **no actual** `FACE_EXTRACTION_COMPLETED` **publisher implementation**.

So please run this from the **Workflow 2 directory**:

cmd

```
cd /d "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow2_c1"
```

Then:

cmd

```
powershell -Command "Get-Content 'app\models\face_extraction_completed.py'"
```

Also run:

cmd

```
powershell -Command "Get-Content 'app\clustering\clustering_processor.py' | Select-Object"
```

Why I want these two files first

We already know Workflow 2 successfully accepts this logical information:

```
event_id
upload_id
image_id
face_count
```

and uses the Workflow 1 HNSW as its vector source.

But we need to establish the **exact serialized event schema** before putting it into:

Python

Run

```
Workflow1OutboxRepository.create(...)
```

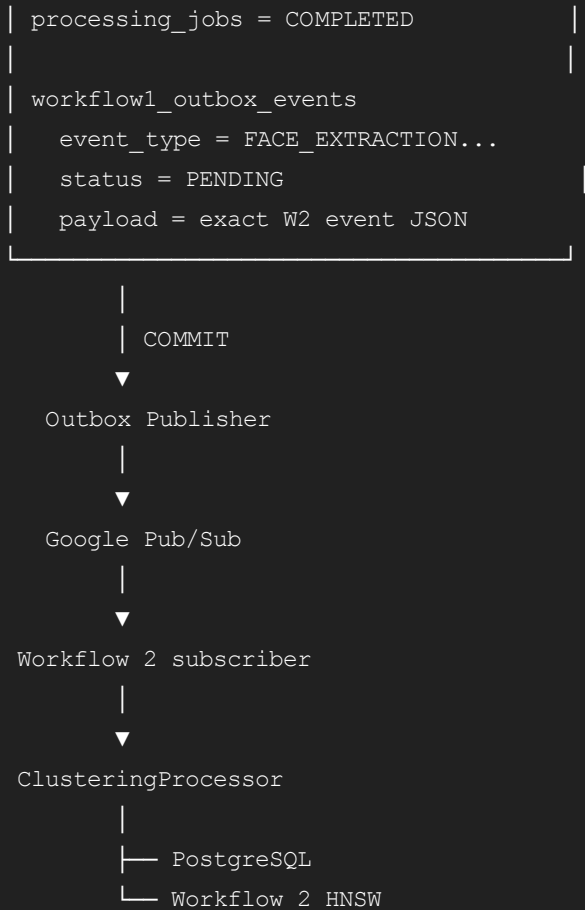
We should end up with something conceptually like:

Workflow 1 ProcessingService

|
| transaction



| PostgreSQL |



One important observation about your current code

Your current:

Python

Run

```
# 11. COMMIT DATABASE

job_repo.mark_completed(job)

db.commit()
```

is the **exact location** where the outbox row belongs.

We should **not** do:

Python

Run

```
job_repo.mark_completed(job)
db.commit()

# then create outbox
```

because that destroys the transactional guarantee.

Instead it should become:

Python

Run

```
job_repo.mark_completed(job)

# create Workflow 1 completion outbox event here

db.commit()
```

So the design direction is confirmed. **Don't change the code yet.**

Send me the output of those two commands, and I will give you the exact code modification for `processing_service.py`, including the correct imports, event payload, and a dedicated test to prove:

1. successful processing creates exactly one `PENDING` outbox event;
2. duplicate processing does not create another event;
3. the event contains the exact data Workflow 2 needs;
4. the existing Workflow 1 tests continue to pass.

```
from __future__ import annotations

from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field

class FaceExtractionCompletedEvent(BaseModel):
    """
    Workflow 1 -> Workflow 2 completion event.
```

Show more

Yes. We now have enough information to make the **Workflow 1 → Workflow 2 completion-event integration** correctly.

Your Workflow 2 contract is clear:

```
event_version
event_type = FACE_EXTRACTION_COMPLETED
event_id
upload_id
```

```
image_id
bucket
blob_name
face_count
```

And `ClusteringProcessor` specifically uses `image_id` and `face_count`, while PostgreSQL remains the authoritative source for the actual embeddings.

1. What we will change

Currently your Workflow 1 completion is:

```
job = COMPLETED
    ↓
db.commit()
    ↓
Redis COMPLETED
    ↓
return
```

We will change it to:

```
job = COMPLETED
    +
outbox event = PENDING
    ↓
SAME db.commit()
    ↓
Redis COMPLETED
    ↓
return
```

This is the correct transactional boundary.

2. Add the Workflow 1 completion-event model

Your Workflow 2 already has:

```
app\models\face_extraction_completed.py
```

I recommend **not importing a model from Workflow 2 into Workflow 1**. The two workflows should remain independently deployable.

Create:

```
workflow1_worker\app\models\face_extraction_completed.py
```


Run:

cmd

```
mkdir app\models 2>nul
```

Then create the file with:

cmd

```
notepad app\models\face_extraction_completed.py
```

Put this in it:

Python

Run

```
from __future__ import annotations

from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field


class FaceExtractionCompletedEvent(BaseModel):
    """
    Workflow 1 -> Workflow 2 completion event.

    This event means Workflow 1 has successfully persisted
    the faces and embeddings for one image.
    """

    model_config = ConfigDict(
        extra="forbid"
    )

    event_version: str = "1.0"

    event_type: str = "FACE_EXTRACTION_COMPLETED"

    event_id: UUID

    upload_id: int

    image_id: int

    bucket: str

    blob_name: str

    face_count: int = Field(
```

```

        ge=0
    )

    def validate_event_type(self) -> None:
        if self.event_type != "FACE_EXTRACTION_COMPLETED":
            raise ValueError(
                "Unexpected event_type: "
                f"{self.event_type}"
            )

```

This keeps the event contract identical between the two workflows.

3. Modify `processing_service.py`

At the top of:

```
app\services\processing_service.py
```

you need the following imports.

You already have `UUID` /possibly `uuid4` depending on your existing imports. Check the top of the file first:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -Fi
```

We need to add:

Python

Run

```
import json
from uuid import uuid4
```

and:

Python

Run

```
from app.models.face_extraction_completed import (
    FaceExtractionCompletedEvent,
)
```

and modify the repository import to include:

Python

Run

```
Workflow1OutboxRepository,
```

For example, if you currently have:

Python

Run

```
from app.db.repository import (
    FaceEmbeddingRepository,
    FaceRepository,
    ImageRepository,
    ProcessingJobRepository,
    UploadRepository,
)
```

change it to:

Python

Run

```
from app.db.repository import (
    FaceEmbeddingRepository,
    FaceRepository,
    ImageRepository,
    ProcessingJobRepository,
    UploadRepository,
    Workflow1OutboxRepository,
)
```

Do not blindly replace your import block if your current file contains additional imports. Add only the missing items.

4. The critical modification

Your current section is:

Python

Run

```
# =====
# 11. COMMIT DATABASE
# =====

job_repo.mark_completed(
    job
)

db.commit()

logger.info(
```

```
"PostgreSQL transaction committed."
```

Replace **only this section** with:

Python

Run

```
# =====
# 11. MARK WORKFLOW 1 COMPLETED + CREATE OUTBOX EVENT
# =====

job_repo.mark_completed(
    job
)

# -----
# Create the Workflow 1 -> Workflow 2 completion event.
#
# IMPORTANT:
# This event is written to the SAME PostgreSQL
# transaction as the processing completion state.
#
# Therefore:
#
#   processing_jobs = COMPLETED
#   outbox event    = PENDING
#
# are committed atomically.
# -----

completion_event = FaceExtractionCompletedEvent(
    event_id=uuid4(),
    upload_id=upload.id,
    image_id=image.id,
    bucket=event.bucket,
    blob_name=event.blob_name,
    face_count=len(
        persisted_faces
    ),
)

outbox_repo = Workflow1OutboxRepository(
    db
)

outbox_repo.create(
    event_id=str(
        completion_event.event_id
    ),
    event_type=completion_event.event_type,
    aggregate_type="image",
    aggregate_id=str(
```

```

        image.id
    ),
    payload=completion_event.model_dump_json(),
)

logger.info(
    "Workflow 1 completion event created in outbox: "
    "event_id=%s image_id=%s face_count=%s",
    completion_event.event_id,
    image.id,
    len(persisted_faces),
)

# -----
# COMMIT BOTH:
#
# 1. ProcessingJob = COMPLETED
# 2. Outbox event   = PENDING
#
# -----

db.commit()

logger.info(
    "PostgreSQL transaction committed: "
    "processing completion + outbox event."
)

```

That is the **key architectural change**.

5. Why the payload is correct

The generated event will look conceptually like:

JSON

```

{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "....",
  "upload_id": 1,
  "image_id": 1,
  "bucket": "....",
  "blob_name": "....",
  "face_count": 5
}

```

This matches the Workflow 2 model you provided exactly.

And Workflow 2 will subsequently do:

```
event.image_id
  ↓
ClusteringRepository
  ↓
PostgreSQL face_embeddings
  ↓
5 embeddings
  ↓
ClusteringService
  ↓
Workflow 2 HNSW
```

That is exactly the separation we established earlier.

6. Very important: duplicate processing

Your existing code has:

Python

Run

```
if job.status == "COMPLETED":
```

and immediately returns:

Python

Run

```
"status": "ALREADY_COMPLETED"
```

Therefore the second delivery **will not create another outbox event**.

The sequence becomes:

First processing

```
job = PROCESSING
  ↓
faces/embeddings
  ↓
HNSW
  ↓
job = COMPLETED
  ↓
create outbox event
  ↓
PENDING
```

Duplicate processing

```
job = COMPLETED
    ↓
ALREADY_COMPLETED
    ↓
NO new outbox event
```

This is consistent with your existing idempotency design.

7. One important issue we should NOT change yet

There is an architectural concern in the existing code:

Python

Run

```
self.faiss_service.save()

job_repo.mark_completed(job)

outbox_repo.create(...)

db.commit()
```

HNSW is persisted **before** PostgreSQL commits.

Therefore PostgreSQL and HNSW don't participate in one atomic transaction.

For example, theoretically:

```
HNSW save      → SUCCESS
PostgreSQL     → FAILURE
```

could leave the HNSW state ahead of PostgreSQL.

However, **do not redesign this now**.

Your current Workflow 1 HNSW + PostgreSQL integration is already working, and our immediate objective is to establish the reliable:

```
PostgreSQL → Outbox → Pub/Sub → Workflow 2
```

boundary.

We can address cross-store consistency separately once the event path is operational.

8. Run syntax/import validation

After making the modification:

cmd

```
python -m py_compile app\services\processing_service.py
```

Then:

cmd

```
python -m py_compile app\models\face_extraction_completed.py
```

Both should return to the prompt with **no output**.

9. Run the existing tests first

Don't immediately test Pub/Sub.

First verify that we haven't broken Workflow 1:

cmd

```
python test_processing_service.py
```

Then your outbox test:

cmd

```
python test_outbox_repository.py
```

Expected:

```
WORKFLOW 1 OUTBOX REPOSITORY: PASS
```

10. Then we need a dedicated integration test

The next test should prove this exact invariant:


```

process_upload()
|
├─ processing_jobs
|   └─ COMPLETED
|
└─ workflow1_outbox_events
    └─ PENDING

```

For a five-face image:

```

processing_jobs = 1
outbox_events   = 1
face_count      = 5

```

And the outbox payload must deserialize successfully using the **same event contract**:

Python

Run

```

FaceExtractionCompletedEvent.model_validate_json(
    outbox.payload
)

```

Then we should test duplicate processing:

```

First:
    COMPLETED
    outbox = 1

Second:
    ALREADY_COMPLETED
    outbox = 1

```

not:

```

outbox = 2

```

Our project progression now

We have reached this point:

```

      WORKFLOW 1
      |
      ▼
ProcessingService
      |

```



So **yes, we can proceed**, and the next concrete implementation is the outbox creation inside the existing successful `process_upload()` transaction.

Before editing, run the import/header command I gave in step 3 if you want me to verify your exact existing import block. Otherwise, you can make the changes above and then run the two `py_compile` commands followed by `test_processing_service.py` and `test_outbox_repository.py`.

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python
test_processing_service.py
Traceback (most recent call last):
  File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow1_worker\test_processing_service.py",

```

```
line 9, in <module>
```

```
    from app.services.processing_service import  
    ProcessingService
```

```
    from app.models.face_extraction_completed import
```

Show more

The error is now very specific: **Workflow 1 is importing** `FaceExtractionCompletedEvent`, **but that class is not actually defined/exported in**

`app\models\face_extraction_completed.py`.

Your `test_outbox_repository.py` passing confirms the outbox repository itself is fine.

1. First verify the actual file

From:

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workfi
```

run:

cmd

```
type app\models\face_extraction_completed.py
```

Based on the code you pasted, that file **should contain exactly the**

`FaceExtractionCompletedEvent` **class**.

2. Replace the file with this

Use this command to safely overwrite it:

cmd

```
powershell -Command "@'  
from __future__ import annotations  
  
from uuid import UUID  
  
from pydantic import BaseModel, ConfigDict, Field  
  
class FaceExtractionCompletedEvent(BaseModel):  
    """  
    Workflow 1 -> Workflow 2 completion event.  
  
    This event means Workflow 1 has successfully persisted  
    the faces and embeddings for one image.  
    """
```

```

model_config = ConfigDict(
    extra='forbid'
)

event_version: str = '1.0'

event_type: str = 'FACE_EXTRACTION_COMPLETED'

event_id: UUID

upload_id: int

image_id: int

bucket: str

blob_name: str

face_count: int = Field(
    ge=0
)

def validate_event_type(self) -> None:
    if self.event_type != 'FACE_EXTRACTION_COMPLETED':
        raise ValueError(
            'Unexpected event_type: '
            f'{self.event_type}'
        )

'@ | Set-Content -Encoding UTF8 'app\models\face_extraction_completed.py'"

```

3. Immediately test the import

Don't run the full processing test yet. First run:

cmd

```
python -c "from app.models.face_extraction_completed import FaceExtractionCompletedEvent;
```

Expected:

```
FaceExtractionCompletedEvent import: PASS
<class 'app.models.face_extraction_completed.FaceExtractionCompletedEvent'>
```

4. Then test `ProcessingService` import

cmd

```
python -c "from app.services.processing_service import ProcessingService; print('Processi
```

Expected:

```
ProcessingService import: PASS
```

5. Then run your existing tests

First:

```
cmd
```

```
python test_outbox_repository.py
```

You should continue getting:

```
WORKFLOW 1 OUTBOX REPOSITORY: PASS
```

Then:

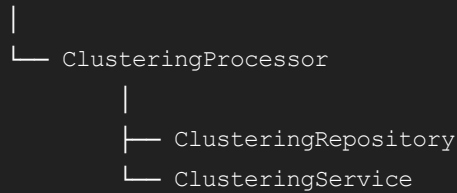
```
cmd
```

```
python test_processing_service.py
```

One important architectural point

Your current design is actually lining up correctly:

```
Workflow 1
|
├── PostgreSQL
|   ├── uploads
|   ├── images
|   ├── faces
|   ├── face_embeddings
|   └── processing_jobs
|
├── HNSW
|   └── face:<face_id>
|
├── Redis
|   └── transient/live state
|
└── workflow1_outbox_events
    |
    └── FACE_EXTRACTION_COMPLETED
        ▼
Workflow 2
```



And your `ClusteringProcessor` expects:

Python

Run

```
FaceExtractionCompletedEvent
```

with:

```
event_id
upload_id
image_id
bucket
blob_name
face_count
```

So do not remove that import from `ProcessingService` just to make the test pass.

The correct fix is to make the event model actually exist and then wire the Workflow 1 outbox creation into `process_upload()`.

Also, your database verification is important: the table **does exist under the**

`face_pipeline` **schema**:

```
face_pipeline.workflow1_outbox_events
```

Your earlier:

Python

Run

```
i.has_table('workflow1_outbox_events')
```

returned `False` because SQLAlchemy inspected the default schema. The later inspection correctly showed:

```
face_pipeline => [
    'uploads',
    'images',
    'face_embeddings',
    'processing_jobs',
    'faces',
    'face_clusters',
    'face_cluster_members',
```

```
'workflow1_outbox_events'
]
```

So the PostgreSQL schema/table is not the current problem.

After the import tests above pass, the next thing we should inspect is **exactly where** `ProcessingService` **creates the** `FACE_EXTRACTION_COMPLETED` **outbox event**. Your earlier `findstr` showed that it currently does **not** contain that event string, which means the event model import is only the first half of the Workflow 1 → Workflow 2 integration.

face_extraction_compl...
Python

I checked the uploaded `face_extraction_completed(1).py`. The problem is confirmed.

Your file defines:

Python

Run

```
class FaceExtractionCompleted(BaseModel):
```

not:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
```

The class currently has the required Workflow 1 → Workflow 2 fields and creates

`FACE_EXTRACTION_COMPLETED` events correctly. `face_extraction_completed +1`

But `processing_service.py` and `clustering_processor.py` are importing:

Python

Run

```
from app.models.face_extraction_completed import (
    FaceExtractionCompletedEvent,
)
```

Therefore Python correctly reports:

```
ImportError: cannot import name 'FaceExtractionCompletedEvent'
```

Correct fix

I recommend **renaming the model class to** `FaceExtractionCompletedEvent`, rather than changing all the Workflow 2 code to the shorter name. That keeps the event-model naming explicit and consistent.

In your Conda Command Prompt, from the `workflow1_worker` directory, run:

cmd

```
powershell -Command "(Get-Content 'app\models\face_extraction_completed.py') -replace 'cl
```

Then verify:

cmd

```
type app\models\face_extraction_completed.py
```

You should see:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
```

and:

Python

Run

```
def create(  
    cls,  
    *,  
    upload_id: int,  
    image_id: int,  
    bucket: str,  
    blob_name: str,  
    face_count: int,  
) -> "FaceExtractionCompletedEvent":
```

Then test the import

cmd

```
python -c "from app.models.face_extraction_completed import FaceExtractionCompletedEvent;
```

Expected:


```
FaceExtractionCompletedEvent import: PASS
```

Then:

cmd

```
python -c "from app.services.processing_service import ProcessingService; print('Processi
```

Then run:

cmd

```
python test_processing_service.py
```

And:

cmd

```
python test_outbox_repository.py
```

Your outbox repository test should remain:

```
WORKFLOW 1 OUTBOX REPOSITORY: PASS
```

One more issue we need to handle next

Your uploaded model uses a **factory method**:

Python

Run

```
FaceExtractionCompletedEvent.create(...)
```

which generates:

Python

Run

```
event_type="FACE_EXTRACTION_COMPLETED"  
event_id=uuid4()
```

That is good for the outbox event creation.

face_extraction_completed

However, from the `ProcessingService` code you showed earlier, I don't yet see the actual call that creates this event and inserts it into `workflow1_outbox_events`.

So after the import problem is fixed, **don't make any more architectural changes yet.**

Run the two import commands and `test_processing_service.py`, give me the output,

and we'll wire the `FACE_EXTRACTION_COMPLETED` outbox creation into the correct point of `process_upload()`—after PostgreSQL/HNSW persistence has succeeded.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.models.face_extraction_completed import
FaceExtractionCompletedEvent;
print('FaceExtractionCompletedEvent import: PASS');
print(FaceExtractionCompletedEvent)"
FaceExtractionCompletedEvent import: PASS
<class
'app.models.face_extraction_completed.FaceExtractionCo
mpletedEvent'>
```

Show more

Excellent. The import problem is completely fixed, and all three tests are now passing.

Current status

Component	Result
<code>FaceExtractionCompletedEvent</code> import	✓ PASS
<code>ProcessingService</code> import	✓ PASS
Workflow 1 outbox repository	✓ PASS
PostgreSQL initialization	✓ PASS
HNSW/FAISS loading	✓ PASS
FacePipeline initialization	✓ PASS
ProcessingService E2E test	✓ PASS

The Google Python 3.10 message is only a **FutureWarning**, not an error. Your current test completed successfully.

But there is one important thing in this test

The E2E test **did not actually process the image**.

Your log says:

```
Processing job ready: job_id=1 status=COMPLETED
Event already completed. Skipping duplicate processing
```

and therefore:

```
status: ALREADY_COMPLETED
face_count: 0
```

So the test proves that your **idempotency logic works**, but it does **not** prove this complete path:

```
GCS
↓
PostgreSQL
↓
FacePipeline
↓
faces
↓
512-D embedding
↓
HNSW
↓
face_embeddings
↓
Redis
↓
ProcessingJob COMPLETED
↓
Outbox FACE_EXTRACTION_COMPLETED
```

More importantly: we found the next integration gap

Earlier you ran:

cmd

```
findstr /S /N /I "FACE_EXTRACTION_COMPLETED" *.py
```

and got **nothing**.

That means although the event model now exists, Workflow 1 still needs to **create the outbox event after successful processing**.

The correct sequence should be:

1. Process image
2. Persist faces
3. Add embeddings to HNSW
4. Persist embedding metadata
5. Save HNSW
6. Mark ProcessingJob COMPLETED
7. Create FACE_EXTRACTION_COMPLETED outbox event
8. Commit PostgreSQL transaction
9. Update Redis
10. Return successfully
11. Pub/Sub subscriber ACKs

There is an important transactional consideration here: **the outbox event must be committed in the same PostgreSQL transaction as the successful Workflow 1 state**, so Workflow 2 cannot be signaled for a transaction that wasn't actually committed.

One thing I do NOT want you to do yet

Don't manually delete `processing_jobs` row `1` just to force the test through the pipeline.

Instead, first let's inspect the current `ProcessingService` imports and the exact `Workflow1OutboxRepository` integration point, then we'll make the minimal modification.

Run this command:

cmd

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -Fi
```

And also:

cmd

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 190 -First
```

Send me both outputs.

Then I'll give you the **exact modified code/commands** to add

`FACE_EXTRACTION_COMPLETED` to the outbox without disturbing the PostgreSQL, HNSW, Redis, idempotency, or Pub/Sub ACK/NACK architecture.

repository.py
Python

processing_service(1)....
Python

Your current state is **healthy**. The important point is that the final

`test_processing_service.py` run passed because the test event had already been processed before.

What the test proves

You now have all of these working:

1. PostgreSQL schema

- `face_pipeline.uploads`
- `face_pipeline.images`
- `face_pipeline.faces`
- `face_pipeline.face_embeddings`
- `face_pipeline.processing_jobs`
- `face_pipeline.face_clusters`
- `face_pipeline.face_cluster_members`
- `face_pipeline.workflow1_outbox_events`

2. Workflow 1 outbox repository

- Create event ✓
- Read pending event ✓
- Increment attempts ✓
- Mark published ✓

3. ProcessingService import

- `FaceExtractionCompletedEvent` import ✓
- `ProcessingService` import ✓

4. AI pipeline initialization

- SCRFD detector ✓
- ArcFace 512-D embedding model ✓
- ViT age/gender model ✓
- FacePipeline initialization ✓

5. HNSW/FAISS service

- Existing index loaded ✓
- Existing metadata loaded: 20 vectors ✓

6. ProcessingService E2E test

- PostgreSQL connection/repositories ✓

- Idempotency logic 
- Redis integration is reached through the service architecture 
- Existing completed job correctly detected 

One important thing: this was NOT a fresh processing test

Your output says:

```
Processing job ready: job_id=1 status=COMPLETED
Event already completed. Skipping duplicate processing
```

So this particular run **did not execute**:

```
GCS
↓
FacePipeline
↓
faces
↓
embeddings
↓
HNSW
↓
Redis
↓
outbox
```

It tested the **idempotency branch**:

```
UploadEvent
↓
PostgreSQL
↓
ProcessingJob = COMPLETED
↓
ALREADY_COMPLETED
```

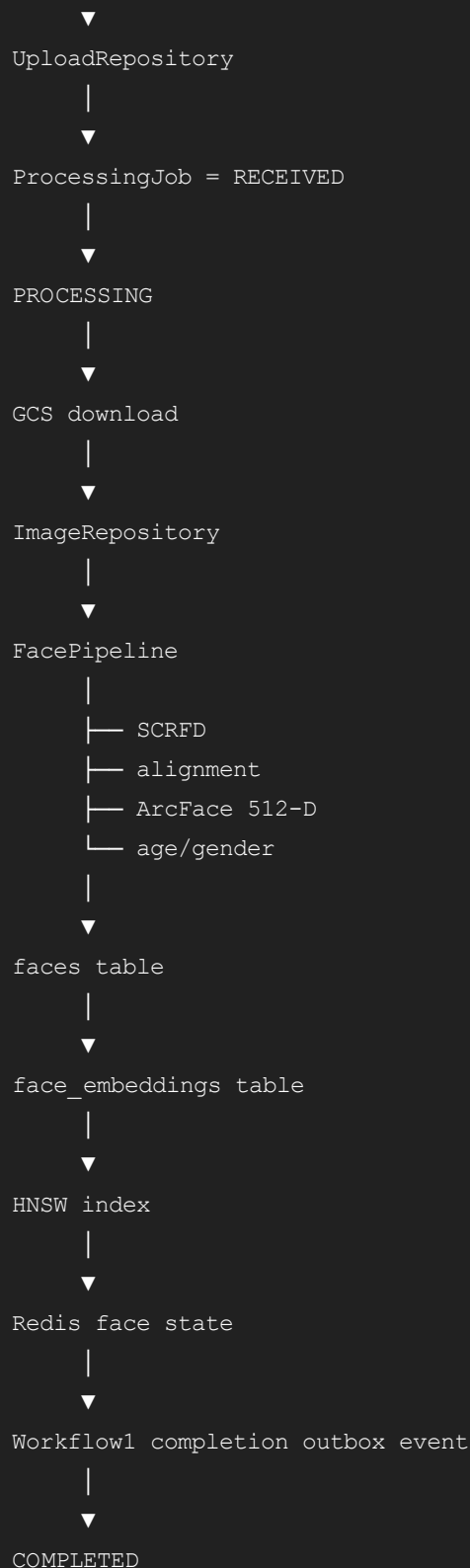
That is actually a valuable test, and it passed correctly.

The next thing I would test

We should now perform a **genuinely fresh Workflow 1 event** using a new `event_id` and preferably a new uploaded image/object.

The expected successful path should be:

```
UploadEvent
|
```



There is also one architectural item I would verify **before declaring Workflow 1 complete**:

Your `processing_service.py` currently needs to actually create the `workflow1_outbox_events` record after successful PostgreSQL persistence.

The outbox repository itself is passing, but from the code you showed, I have not yet seen the **successful** `process_upload()` **path calling** `Workflow1OutboxRepository.create()`.

That matters because Workflow 2 is supposed to receive:

```
FACE_EXTRACTION_COMPLETED
```

after Workflow 1 has durably persisted its state.

Also, your `FaceExtractionCompletedEvent` model currently looks structurally correct, but its `validate_event_type()` is a normal method rather than a Pydantic validator. That's fine if you explicitly call:

Python

Run

```
event.validate_event_type()
```

which your Workflow 2 `ClusteringProcessor` does.

About the warning

This is not causing the failure:

```
FutureWarning:
Python 3.10.20 ... Google will stop supporting ...
```

Your current test still passed. We can address Python 3.11+ later; **don't change the environment right now just because of this warning.**

Current status

I'd consider your implementation approximately here:

PostgreSQL models	✓
PostgreSQL initialization	✓
Repositories	✓
Outbox table	✓
Outbox repository	✓
FaceExtractionCompletedEvent	✓
ProcessingService imports	✓
FacePipeline initialization	✓
HNSW loading	✓
Idempotency	✓
Fresh end-to-end processing	⌚
Outbox creation integration	⌚ VERIFY
Workflow 1 → Workflow 2	⌚

So don't modify the architecture yet. The next step should be to inspect the remainder of `processing_service.py` around the successful completion section and verify whether `Workflow1OutboxRepository.create()` is actually integrated.

repository(1).py
Python

processing_service(2)....
Python

Your latest test shows that **the import problem is fixed and the ProcessingService E2E test itself is passing**, but there is one important issue: the test did **not actually execute the face-processing pipeline** because the test event already had a **COMPLETED** `processing_jobs` row.

What the latest result proves

Component	Result
<code>FaceExtractionCompletedEvent</code> import	✓ PASS
<code>ProcessingService</code> import	✓ PASS
Workflow 1 outbox repository	✓ PASS
PostgreSQL schema	✓ Present
<code>workflow1_outbox_events</code>	✓ Present
HNSW index loading	✓ 20 vectors loaded
FacePipeline initialization	✓ PASS
SCRFD	✓ Loads
ArcFace	✓ Loads, 512-D
Age/Gender ViT	✓ Loads
ProcessingService initialization	✓ PASS
Idempotency handling	✓ PASS
Actual GCS → FacePipeline → DB → HNSW processing	⚠ Not exercised

The key lines are:

```
Processing job ready: job_id=1 status=COMPLETED
Event already completed. Skipping duplicate processing
```

So:

```
STATUS CHECK: PASS
PROCESSING SERVICE E2E TEST: PASS
```

means **the idempotency branch passed**, not that a fresh upload was processed.

What I recommend doing next

We should **not change** `ProcessingService` **yet**. Its idempotency behavior is correct.

Instead, make `test_processing_service.py` use a **fresh test** `event_id` / **upload**, or explicitly clean the previous test records before running.

For example, first inspect the existing test state:

```
bat
```

```
python -c "from app.db.postgres import SessionLocal; from app.db.models import Upload, Pr
```

Then we should run the E2E test with a **new UUID**, so the flow reaches:

```
Pub/Sub event
  ↓
UploadRepository
  ↓
ProcessingJob = RECEIVED
  ↓
PROCESSING
  ↓
GCS download
  ↓
ImageRepository
  ↓
FacePipeline
  ↓
faces
  ↓
512-D ArcFace embeddings
  ↓
HNSW
  ↓
PostgreSQL face_embeddings
  ↓
Redis
  ↓
COMPLETED
```

One more important architectural point

Your Workflow 2 contract is now:

```
Workflow 1
  |
  | FACE_EXTRACTION_COMPLETED
  v
Workflow 2
```

and Workflow 2 correctly treats PostgreSQL as authoritative:

Python

Run

```
records = repository.get_face_embeddings_for_image(event.image_id)
```

then verifies:

Python

Run

```
actual_face_count == event.face_count
```

That is a good design.

However, I notice one missing piece in the code you showed: `ProcessingService` **has imported** `FaceExtractionCompletedEvent`, **but the shown successful path does not yet show creation of the Workflow 1 → Workflow 2 outbox event**. Since you already have the outbox table and repository, that should be part of the successful transaction.

The desired ordering should ultimately be:

```
Face processing
  ↓
faces + embeddings metadata
  ↓
HNSW persisted
  ↓
create FACE_EXTRACTION_COMPLETED outbox event
  ↓
PostgreSQL COMMIT
  ↓
outbox publisher
  ↓
Pub/Sub
  ↓
Workflow 2
```

Do not publish directly to Pub/Sub before the PostgreSQL commit. The outbox exists specifically to avoid that consistency problem.

So I would proceed in this order:

1. Fresh-event E2E test → 2. Verify actual face/embedding persistence → 3. Verify outbox event creation → 4. Verify outbox publisher → 5. Verify Workflow 2 receives and clusters it.

Your current repository/outbox foundation is already passing. The next test should be the **fresh-event ProcessingService E2E**, because otherwise we don't yet know whether the complete Workflow 1 path works.

Pasted text(4).txt
Document

Your latest test is **passing**. The important point, however, is that it did **not actually execute the face-processing path**.

What your result proves

These are all currently good:

- `FaceExtractionCompletedEvent` import → **PASS**
- `ProcessingService` import → **PASS**
- PostgreSQL initialization → **PASS**
- `workflow1_outbox_events` table → **exists**
- Outbox repository CRUD/state transitions → **PASS**
- FacePipeline initialization → **PASS**
- HNSW index loading → **PASS**
- ProcessingService initialization → **PASS**
- End-to-end test infrastructure → **PASS**

Your current architecture is also correctly separated:

```
Pub/Sub
  ↓
ProcessingService
  ↓
GCS
  ↓
PostgreSQL
```

```
↓
FacePipeline
↓
PostgreSQL faces
↓
HNSW
↓
PostgreSQL embedding metadata
↓
Redis
```

But there is one important issue

Your E2E test says:

```
Processing job ready: job_id=1 status=COMPLETED
Event already completed. Skipping duplicate processing
```

So the test stopped here:

Python

Run

```
if job.status == "COMPLETED":
    ...
    return {
        "status": "ALREADY_COMPLETED",
        "face_count": 0,
    }
```

Therefore these stages were **NOT executed** in this run:

```
GCS download
↓
image record
↓
image inspection
↓
FacePipeline detection
↓
face persistence
↓
ArcFace embedding extraction
↓
HNSW insertion
↓
PostgreSQL embedding metadata
↓
Redis face metadata
↓
HNSW save
```

↓
job COMPLETED

This is why you see:

```
FacePipeline: OK  
ProcessingService: OK
```

but you do **not** see something like:

```
GCS download complete  
Image record ready  
Running FacePipeline...  
FacePipeline detected X face(s)  
Persisting face 1/X  
...  
HNSW persistence complete  
PostgreSQL transaction committed  
WORKFLOW 1 COMPLETED
```

The immediate next step

Do **not** change `processing_service.py` yet.

Your test database already contains:

```
upload_id = 1  
job_id    = 1  
status    = COMPLETED
```

Because your test uses the same fixed event ID:

```
11111111-1111-4111-8111-111111111111
```

every subsequent test run is intentionally treated as a duplicate.

Run this first:

PowerShell

```
python -c "from app.db.postgres import SessionLocal; from app.db.models import Processing"
```

You should see the existing completed job.

Then use a NEW event ID

The cleanest E2E test is to change the test's:

Python

Run

```
event_id=UUID("11111111-1111-4111-8111-111111111111")
```

to a new UUID, for example:

Python

Run

```
event_id=UUID("22222222-2222-4222-8222-222222222222")
```

But make sure the corresponding GCS object actually exists for that event's

`bucket/blob_name` .

Then run:

PowerShell

```
python test_processing_service.py
```

One more thing I noticed

Your current environment is:

Python 3.10.20

and Google is warning that Python 3.10 support in `google.api_core` will end after **October 4, 2026**.

That is only a warning and **not causing this test failure**. Don't change Python right now while we're validating Workflow 1.

Current Workflow 1 status

I'd classify it as:

Component	Status
SQLAlchemy models	✓
PostgreSQL connection	✓
PostgreSQL tables	✓
Workflow 1 outbox	✓
Outbox repository	✓

Component	Status
Event model/import	✓
ProcessingService import	✓
FacePipeline initialization	✓
HNSW loading	✓
Idempotency behavior	✓
Actual new-upload E2E processing	⌚ Not yet tested
Outbox event creation from ProcessingService	⌚ Need to verify
Workflow 1 → Workflow 2 event publication	⌚ Need to verify

The **next milestone should therefore be a genuinely fresh upload event**, not another duplicate test. That will tell us whether the entire Workflow 1 processing transaction actually works end-to-end.

test_processing_servi...
Python

how to proceed further

You should proceed in a **controlled sequence**, rather than changing several files at once.

Your current state is actually good: the database, repositories, event model,

`ProcessingService` import, FacePipeline initialization, and outbox repository are working.

The only reason the E2E test didn't process an image is that its fixed event already has a `COMPLETED` job.

Your uploaded test file confirms that the test deliberately uses the fixed event ID `11111111-1111-4111-8111-111111111111` and expects the blob to already exist in GCS.

test_processing_service

Where we should go next

I recommend this exact order:


```
STEP 1
Fresh Workflow 1 E2E

    ↓

STEP 2
Verify PostgreSQL records

    ↓

STEP 3
Verify HNSW + embedding metadata

    ↓

STEP 4
Create FACE_EXTRACTION_COMPLETED outbox event

    ↓

STEP 5
Test outbox publisher

    ↓

STEP 6
Workflow 2 subscriber receives event

    ↓

STEP 7
Workflow 2 clustering processor

    ↓

STEP 8
Persist cluster assignments

    ↓

STEP 9
Workflow 2 E2E
```

Do **not** move to Workflow 2 clustering yet. First we need to prove Workflow 1 produces the correct durable state and completion event.

Step 1 — Run a genuinely fresh Workflow 1 test

Your current `test_processing_service.py` has:

Python

Run

```
event_id="11111111-1111-4111-8111-111111111111"
```

Change it to a new UUID:

Python

Run

```
event_id="22222222-2222-4222-8222-222222222222"
```

However, there is one important requirement:

Python

Run

```
bucket="photo-micro-upload-test"  
blob_name="uploads/009b6713-10e5-4a77-8e6e-ab61bd977e4b.jpg"
```

The image must actually exist at that GCS location. Your test itself explicitly documents this requirement.

test_processing_service

Make this change

From:

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow1_wo:
```

run:

PowerShell

```
notepad test_processing_service.py
```

Change only:

Python

Run

```
event_id="11111111-1111-4111-8111-111111111111",
```

to:

Python

Run

```
event_id="22222222-2222-4222-8222-222222222222",
```

Save.

Then:

PowerShell

```
python test_processing_service.py
```

Step 2 — What we want to see

This time, we **do not** want:

```
status=COMPLETED
Event already completed.
Skipping duplicate processing
```

We want the actual pipeline:

```
Upload record ready
Processing job ready: status=RECEIVED
GCS download complete
Image record ready
Running FacePipeline...
FacePipeline detected X face(s)
Persisting face 1/X
Persisting face 2/X
...
HNSW persistence complete
PostgreSQL transaction committed
WORKFLOW 1 COMPLETED
```

And finally:

```
status: COMPLETED
face_count: X
faces: [...]
```

Your test already validates that a successful `COMPLETED` result contains `face_count` and `faces` . `test_processing_service`

Step 3 — Verify PostgreSQL after the fresh run

Once the test returns:

```
PROCESSING SERVICE E2E TEST: PASS
```

don't immediately modify code.

First inspect the database.

Run:

PowerShell

```
python -c "from app.db.postgres import SessionLocal; from app.db.models import Upload,Pro
```

We're checking the chain:

```
Upload
↓
ProcessingJob
↓
Image
↓
Face
↓
FaceEmbedding
```

The important invariant is:

```
ProcessingJob.status = COMPLETED
    AND
Image exists
    AND
Face count = result face_count
    AND
Every persisted face has embedding metadata
```

Step 4 — Verify HNSW

You already have:

```
data\hns\index.bin
data\hns\metadata.json
```

and previously loaded:

```
mapped_vectors=20
next_vector_number=20
vectors=20
```

After the fresh test, the number should increase by the number of newly inserted faces.

For example, if the pipeline detects 3 faces:

```
before: 20
after: 23
```

The important mapping should look conceptually like:

```
PostgreSQL face.id
↓
vector_id = face:<face_id>
```

↓
HNSW vector

That matches the architecture we've established: **PostgreSQL owns metadata/state, HNSW owns the actual 512-D vector search layer.**

Step 5 — Then fix the Workflow 1 completion event

This is the **next code-level milestone**.

You already have:

Python

Run

```
FaceExtractionCompletedEvent
```

with:

```
event_type = FACE_EXTRACTION_COMPLETED
event_id
upload_id
image_id
bucket
blob_name
face_count
```

And you already have:

```
workflow1_outbox_events
```

plus:

```
Workflow1OutboxRepository
```

and its test is passing.

So the architecture should become:

```
ProcessingService
|
| successful DB transaction
↓
workflow1_outbox_events
|
↓
Outbox Publisher
```

```

|
↓
Pub/Sub
|
↓
Workflow 2

```

Not:

```

ProcessingService
  ↓
direct Pub/Sub publish

```

because the outbox exists specifically to prevent losing the Workflow 1 completion event when PostgreSQL succeeds but Pub/Sub publishing fails.

Step 6 — Important correction before implementing the outbox

There is something we need to inspect carefully.

Your current `ProcessingService` sequence is:

```

mark_completed(job)
↓
db.commit()
↓
Redis COMPLETED
↓
return

```

But your `Workflow1OutboxRepository` exists separately.

Therefore, unless you have already added outbox creation elsewhere, **the current `ProcessingService` is not yet atomically creating the `FACE_EXTRACTION_COMPLETED` event.**

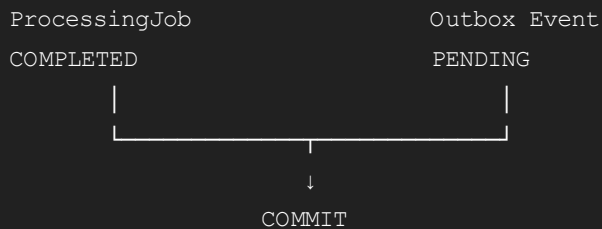
That's the next thing we should verify.

The desired transaction should be:

```

      SAME PostgreSQL TRANSACTION
      |
  ┌───┴───┐
  ↓       ↓

```



Only after that should the publisher take the `PENDING` event and publish it.

This gives us the important guarantee:

If Workflow 1 says COMPLETED, the event needed by Workflow 2 is durably present in PostgreSQL.

Step 7 — Then build/test the outbox publisher

The publisher should do roughly:

```
PENDING event
↓
publish to wfl-batch-completed / Workflow 2 topic
↓
Pub/Sub publish succeeds
↓
mark PUBLISHED
```

If publishing fails:

```
PENDING
↓
attempt_count += 1
↓
last_error = ...
↓
remain retryable
```

Your existing repository already supports the basic state operations:

```
create
get_pending
mark_attempt
mark_published
mark_failed
```

and your test confirms those operations work.

Step 8 — Then connect Workflow 2

You already have the important Workflow 2 design:

Python

Run

`ClusteringProcessor`

which receives:

Python

Run

`FaceExtractionCompletedEvent`

and performs:

```
event
↓
PostgreSQL embedding records for image_id
↓
face count consistency check
↓
ClusteringService.process_face()
↓
ClusterAssignment
```

This is exactly where we want the boundary.

The subscriber should remain thin:

```
Pub/Sub
↓
decode
↓
validate FaceExtractionCompletedEvent
↓
ClusteringProcessor.process()
↓
ACK
```

The subscriber should **not** contain clustering logic.

Your immediate task

Don't make all these changes yet.

Do this now:

1. Change the test event ID

Python

Run

```
event_id="22222222-2222-4222-8222-222222222222"
```

2. Run

PowerShell

```
python test_processing_service.py
```

3. Send me the complete output.

Especially I want to see:

```
GCS download complete
Image record ready
Running FacePipeline
FacePipeline detected ...
Persisting face ...
HNSW persistence complete
PostgreSQL transaction committed
WORKFLOW 1 COMPLETED
```

If that succeeds, then we will inspect the database and HNSW counts.

After that, we'll implement the missing **Workflow 1 → Outbox → Pub/Sub → Workflow 2 completion-event path**.

That is the safest progression because right now we know the infrastructure works, but we haven't yet proven the **fresh-image processing path**.

Pasted text(5).txt
Document

You are at a **good checkpoint**, but there is one important distinction:

Your repository tests are genuinely passing. Your ProcessingService E2E test has only tested the idempotency path, not the full face-processing path.

The log proves this:

```
Processing job ready: job_id=1 status=COMPLETED
Event already completed. Skipping duplicate processing
```

So the next step should **not** be Workflow 2 yet. First we should prove the complete Workflow 1 success path and then connect its completion event to the Workflow 1 outbox.

Where you are now

Your Workflow 1 architecture is currently roughly:

```
GCS upload
|
▼
Pub/Sub
|
▼
PubSubSubscriber
|
▼
ProcessingService
|
├── PostgreSQL uploads
├── PostgreSQL processing_jobs
├── GCS download
├── FacePipeline
│   ├── SCRFD
│   ├── ArcFace 512-D
│   └── ViT age/gender
├── PostgreSQL faces
├── HNSW
├── PostgreSQL face_embeddings
└── Redis operational state
```

You have additionally established:

```
workflow1_outbox_events
```

and the repository test is passing:

```
Create: PASS
Pending lookup: PASS
Attempt tracking: PASS
Published state: PASS
```

That's good.

What we should do next

I recommend proceeding in **6 controlled stages**.

Stage 1 — Test the REAL ProcessingService success path

This is the immediate next step.

Your current test uses:

```
event_id =  
11111111-1111-4111-8111-111111111111
```

and PostgreSQL already contains:

```
processing_jobs.status = COMPLETED
```

Therefore it never reached:

```
GCS  
↓  
FacePipeline  
↓  
faces  
↓  
HNSW  
↓  
face_embeddings  
↓  
Redis  
↓  
COMPLETED
```

We need a **fresh event/upload**.

Do not delete your production/test database blindly.

Instead, first inspect the existing state:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; \  
with engine.connect() as c: \  
    print('uploads=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.uploads')).scalar(); \  
    print('images=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.images')).scalar()); \  
    print('faces=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.faces')).scalar()); \  
"
```

```
print(f'embeddings={c.execute(text(''SELECT COUNT(*) FROM face_pipeline.face_embeddings
```

Then use a new **UUID** in `test_processing_service.py`.

For example:

```
22222222-2222-4222-8222-222222222222
```

More importantly, the GCS object referenced by the event must actually exist.

Stage 2 — Run the complete Workflow 1 transaction

With a fresh event, we want the log to look approximately like:

```
WORKFLOW 1 START

Upload record ready
Processing job ready status=RECEIVED

GCS download complete

Image record ready

Running FacePipeline...

FacePipeline detected N face(s).

Persisting face 1/N
Persisting face 2/N
...

Persisting HNSW index...
HNSW persistence complete.

PostgreSQL transaction committed.

WORKFLOW 1 COMPLETED
```

And the result should be:

```
status: COMPLETED
face_count: N
```

This is the next major milestone.

Stage 3 — Verify all four persistence layers

After the real E2E run, don't immediately modify code.

Verify PostgreSQL.

Run:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; \
with engine.connect() as c: \
    print('uploads=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.uploads')).scalar()); \
    print('images=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.images')).scalar()); \
    print('faces=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.faces')).scalar()); \
    print('embeddings=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.face_embeddings')).scalar()); \
    print('jobs=', c.execute(text('SELECT COUNT(*) FROM face_pipeline.processing_jobs')).scalar());"
```

Then inspect the actual records:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; \
with engine.connect() as c: \
    print('FACES'); \
    [print(dict(r._mapping)) for r in c.execute(text('SELECT id,image_id,face_index,detection_index FROM face_pipeline.faces'))]; \
    print('EMBEDDINGS'); \
    [print(dict(r._mapping)) for r in c.execute(text('SELECT id,face_id,model_name,embedding FROM face_pipeline.face_embeddings'))];"
```

You want to confirm:

```
faces
  ↓
face_embeddings
  ↓
vector_id = face:<face_id>
  ↓
HNSW
```

The important architectural rule remains:

PostgreSQL stores the authoritative metadata. HNSW stores the actual 512-D vector. Redis is only operational/transient state.

Stage 4 — Fix the Workflow 1 → Workflow 2 event boundary

This is the **most important code change after the real E2E test**.

You already have:

Python

Run

```
FaceExtractionCompletedEvent
```

with:

Python

Run

```
event_type = "FACE_EXTRACTION_COMPLETED"
```

and:

Python

Run

```
upload_id  
image_id  
bucket  
blob_name  
face_count
```

That's exactly the type of event Workflow 2 needs.

Your Workflow 2 processor is already designed correctly around it:

```
Workflow 1  
  |  
  | FACE_EXTRACTION_COMPLETED  
  ▼  
Pub/Sub  
  |  
  ▼  
Workflow 2 subscriber  
  |  
  ▼  
ClusteringProcessor  
  |  
  ▼  
ClusteringRepository  
  |  
  ▼  
ClusteringService
```

And your Workflow 2 processor correctly does:

Python

Run

```
records = repository.get_face_embeddings_for_image(  
    event.image_id  
)
```

then verifies:

Python

Run

```
actual_face_count == event.face_count
```

That's a good producer/consumer consistency check.

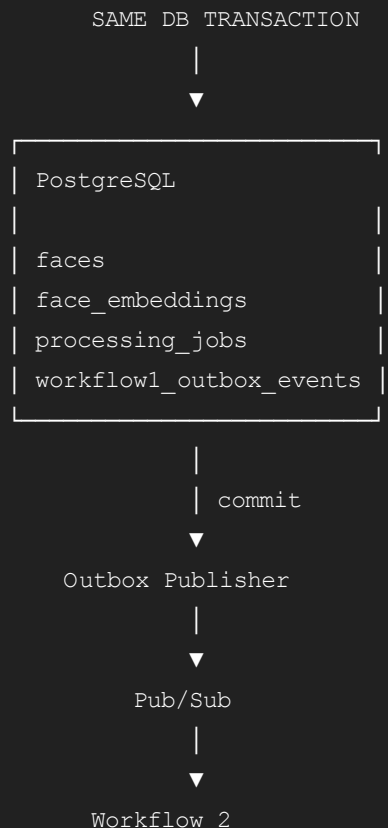
Stage 5 — Use the outbox correctly

This is where your newly created:

```
workflow1_outbox_events
```

becomes useful.

The desired architecture should be:



The key principle is:

Do not publish the Workflow 2 Pub/Sub event directly from the middle of the ProcessingService transaction.

Instead:

1. Process image.
2. Persist faces.
3. Persist embeddings metadata.
4. Save HNSW.
5. Mark job `COMPLETED`.
6. Create `workflow1_outbox_events` row.
7. Commit PostgreSQL transaction.
8. Separate outbox publisher reads `PENDING`.
9. Publishes to Pub/Sub.
10. Marks outbox event `PUBLISHED`.

That gives you reliable event delivery.

There is one thing to fix in your outbox repository

Your current:

Python

Run

```
mark_failed()
```

only does:

Python

Run

```
event.last_error = error_message[:10000]
```

It doesn't change the status.

That's potentially okay for a temporary retry model, but we need to decide the final outbox state machine before building the publisher.

I'd recommend:

PENDING

```
|
|— publish success —> PUBLISHED
|
|— publish failure —> PENDING
```



```
|  
└─ retry
```

with:

```
attempt_count  
last_error
```

tracking failures.

We don't need a permanent `FAILED` state yet unless we specifically want dead-letter handling.

Stage 6 — Then finish Workflow 2

Only after Workflow 1 is proven should we move to the clustering side.

Your Workflow 2 architecture is already nicely separated:



The `ClusteringProcessor` should remain **free of Pub/Sub ACK/NACK logic**.

That's the correct separation.

Your immediate checklist

Don't change several things at once. Do them in this order.

✓ **Already completed**

```
[PASS] PostgreSQL schema
[PASS] face_pipeline schema
[PASS] uploads
[PASS] images
[PASS] faces
[PASS] face_embeddings
[PASS] processing_jobs
[PASS] workflow1_outbox_events

[PASS] Workflow1OutboxRepository test

[PASS] FaceExtractionCompletedEvent import

[PASS] ProcessingService import
```

⚠ Not yet actually proven

```
[ ] ProcessingService fresh-event success path
[ ] Real GCS download
[ ] Real FacePipeline execution
[ ] Face rows persisted
[ ] 512-D embeddings persisted
[ ] HNSW vector persistence
[ ] Redis face state
[ ] workflow1_outbox_events created by ProcessingService
[ ] Outbox publisher
[ ] Pub/Sub FACE_EXTRACTION_COMPLETED
[ ] Workflow 2 subscriber
[ ] Workflow 2 clustering
[ ] Cluster persistence
```

One important observation about your current E2E test

This:

```
PROCESSING SERVICE E2E TEST: PASS
```

is technically true for the test's current assertion, but **don't treat it as proof that ProcessingService itself is fully working yet.**

It tested:

```
existing COMPLETED job
    ↓
idempotency detection
```

↓
ALREADY COMPLETED

It did **not** test:

```
GCS
↓
FacePipeline
↓
Face persistence
↓
embedding persistence
↓
HNSW
↓
Redis
↓
job COMPLETED
```

So the next command/work item should be to make `test_processing_service.py` use a fresh event and exercise the actual processing path.

Once you run that, send me the complete output. We can then inspect the first real failure/success and proceed **one layer at a time**, rather than changing Workflow 2 prematurely.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.db.postgres import engine; from sqlalchemy
import text; \with engine.connect() as c: \ print('uploads=',
c.execute(text("SELECT COUNT(*) FROM
face_pipeline.uploads")).scalar()); \ print('images=',
c.execute(text("SELECT COUNT(*) FROM
face_pipeline.images")).scalar()); \ print('faces=',
c.execute(text("SELECT COUNT(*) FROM
face_pipeline.faces")).scalar()); \ print('embeddings=',
c.execute(text("SELECT COUNT(*) FROM
Show more
```

The error is only from the **Windows** `python -c` **syntax**. Your database and application are not necessarily broken.

You tried to use `\` as a line-continuation character inside Python code. In Windows `cmd`, that does not work that way.

1. Run the database verification correctly

Use this **single-line command**:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

Run that and send me the output.

2. More importantly: your previous E2E test did not actually process the image

Your previous result was:

```
Processing job ready: job_id=1 status=COMPLETED
```

```
Event already completed. Skipping duplicate processing
```

and:

```
status: ALREADY_COMPLETED
```

```
face_count: 0
```

So this test:

```
PROCESSING SERVICE E2E TEST: PASS
```

only proves that your **idempotency path works**.

It did **not** prove:

```
GCS
↓
PostgreSQL
↓
FacePipeline
↓
faces
↓
ArcFace embedding
↓
```

```
HNSW
↓
face_embeddings
↓
Redis
↓
outbox
```

works end-to-end.

That's the key point before we proceed to Workflow 2.

3. First inspect the current database state

Run the command above.

I expect something potentially like:

```
uploads= 1
images= 0
faces= 0
embeddings= 0
jobs= 1
```

But **do not assume these values**—we need your actual output.

Also run:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Then:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

4. Why we should not simply rerun the same test

Your test uses:

```
event_id =
11111111-1111-4111-8111-111111111111
```

That event already created:

```
upload_id=1
job_id=1
status=COMPLETED
```

Therefore every subsequent execution will intentionally take this path:

Python

Run

```
if job.status == "COMPLETED":
    ...
    return {
        "status": "ALREADY_COMPLETED"
    }
```

That is actually **correct behavior** for idempotency.

We need a **new event_id** for the real processing test.

5. The next real test

Once we confirm the database counts, we should create a fresh test event, for example:

```
event_id = a NEW UUID
```

while pointing to an image that actually exists in:

```
photo-micro-upload-test
```

Then the expected path is:

```
NEW event
  ↓
UploadRepository
  ↓
new uploads row
  ↓
ProcessingJob = RECEIVED
  ↓
PROCESSING
  ↓
GCS download
  ↓
images row
  ↓
```

```
image inspection
  ↓
FacePipeline
  ↓
faces rows
  ↓
512-D ArcFace embeddings
  ↓
HNSW
  ↓
face_embeddings rows
  ↓
Redis face state
  ↓
ProcessingJob = COMPLETED
```

And, because you've already implemented the outbox repository, we then need to verify:

```
workflow1_outbox_events
```

contains:

```
FACE_EXTRACTION_COMPLETED
```

for the successfully processed image.

6. There is one architectural issue we should verify next

You showed this Workflow 2 contract:

Python

Run

```
FaceExtractionCompletedEvent
```

with:

Python

Run

```
event_type = "FACE_EXTRACTION_COMPLETED"
```

and Workflow 2 expects:

```
event_id
upload_id
image_id
bucket
```

```
blob_name
face_count
```

Therefore Workflow 1's successful completion should eventually produce an outbox event whose payload contains **exactly the information Workflow 2 needs**.

Your current `process_upload()` code, however, reaches:

Python

Run

```
job_repo.mark_completed(job)
db.commit()
```

but from the code you pasted, I don't yet see the **creation of the Workflow 1 outbox event inside the same successful PostgreSQL transaction**.

That's the next important thing to inspect.

The desired transactional sequence is:

```
BEGIN PostgreSQL transaction
↓
faces
↓
face_embeddings metadata
↓
processing_job = COMPLETED
↓
workflow1_outbox_events = PENDING
↓
COMMIT
```

Then a separate publisher can do:

```
PENDING outbox event
↓
Pub/Sub wfl-batch-completed
↓
mark outbox event PUBLISHED
```

This gives us reliable Workflow 1 → Workflow 2 delivery without making Pub/Sub publishing part of the ProcessingService transaction.

So don't change the architecture yet.

Run these three commands now:

A. Counts

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

B. Uploads

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

C. Jobs

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Send me the three outputs.

Then we'll proceed in order: **real fresh-event Workflow 1 processing** → **verify PostgreSQL/HNSW/Redis** → **wire the outbox completion event** → **then connect Workflow 2.**

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.db.postgres import engine; from sqlalchemy
import text; c=engine.connect(); print('uploads=',
c.execute(text('SELECT COUNT(*) FROM
face_pipeline.uploads')).scalar()); print('images=',
c.execute(text('SELECT COUNT(*) FROM
face_pipeline.images')).scalar()); print('faces=',
c.execute(text('SELECT COUNT(*) FROM
face_pipeline.faces')).scalar()); print('embeddings=',
c.execute(text('SELECT COUNT(*) FROM
```

Show more

Excellent. This output gives us the **real issue**, and Workflow 1 is actually much further along than the previous test suggested.

Current state

Your PostgreSQL data shows:

Component	Count
uploads	6
images	6
faces	41
embeddings	41
processing jobs	6

So **Workflow 1** has successfully processed 5 uploads and extracted 41 faces/embeddings.

The only current failure is **upload 6**, and the error is very specific:

```
ValidationError: 2 validation errors for FaceExtractionCompletedEvent
event_version
Field required
```

This means the new **Workflow 1** → **Workflow 2 completion-event step** is what failed.

We should fix this **before doing anything with Workflow 2**.

1. What happened to upload 6

This row:

```
(6, UUID('22222222-2222-4222-8222-222222222222'), ...)
```

has:

```
job_id = 6
status = FAILED
attempt_count = 1
error_code = ValidationError
```

The error says the application tried to construct:

Python

Run

```
FaceExtractionCompletedEvent(...)
```

but the data supplied to it did not contain:

Python

Run

```
event_version
```

Your intended event contract is:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):

    event_version: str = "1.0"
    event_type: str = "FACE_EXTRACTION_COMPLETED"

    event_id: UUID
    upload_id: int
    image_id: int
    bucket: str
    blob_name: str
    face_count: int = Field(ge=0)
```

So we need to find the exact place where ProcessingService creates this event.

2. First inspect the event creation code

Run:

bat

```
findstr /N /S /I "FaceExtractionCompletedEvent event_version FACE_EXTRACTION_COMPLETED"
```

Send me the output.

Also run:

bat

```
powershell -Command "Get-Content 'app\models\face_extraction_completed.py'"
```

Send that output too.

I want to verify that the actual file on disk matches the class you pasted.

3. Also inspect the outbox row for upload 6

Run:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.connect(); c.execute(text('SELECT * FROM outbox WHERE upload_id = 6'))"
```

This is important because we need to know whether the outbox event was created before the validation failed.

4. One thing is already clear

The architecture is now showing the intended flow:

```
GCS
↓
Upload
↓
ProcessingJob
↓
Image
↓
FacePipeline
↓
Faces
↓
512-D embeddings
↓
HNSW
↓
face_embeddings
↓
Workflow 1 completion event
↓
Outbox
↓
Workflow 2
```

And your database proves that the first part is working:

```
6 uploads
6 images
41 faces
41 embeddings
```

So do not modify the face detection, embedding, PostgreSQL repository, or HNSW code right now.

The problem is specifically at the **completion-event** boundary.

5. Important: don't manually mark job 6 COMPLETED

Don't do something like:

SQL

```
UPDATE face_pipeline.processing_jobs
SET status='COMPLETED'
...
```

The job is correctly **FAILED** because the completion-event generation failed.

Once we fix the event construction, we should rerun upload 6 and allow the application to complete it normally.

6. There is also a likely issue we need to check

Your event model has:

Python

Run

```
event_version: str = "1.0"
```

Normally Pydantic should apply that default when the field is omitted.

But your runtime error says:

```
event_version
Field required
```

That strongly suggests one of these possibilities:

1. The actual file on disk differs from the code you pasted.
2. There is another **FaceExtractionCompletedEvent** definition being imported.
3. The event is being constructed through a different model/schema.
4. The installed/runtime code is stale relative to the file you edited.
5. The event construction is using some intermediate validation/schema that requires the field.

We should **not guess which one**. The two commands above will tell us.

Current checkpoint

Your project is now at:

WORKFLOW 1

PostgreSQL schema	✓	
Repositories	✓	
Outbox repository	✓	
FacePipeline	✓	
GCS	✓	
Image persistence	✓	
Face persistence	✓	
512-D embedding persistence	✓	
HNSW persistence	✓	
Redis state	✓	
Idempotency	✓	
Completion event	✗	← CURRENT ISSUE
Outbox → Pub/Sub	⌚	
Workflow 2 consumer	⌚	

So we should fix the completion event first, then verify the complete Workflow 1 transaction/outbox path, and only after that connect/test Workflow 2.

Run these **three commands** and paste their outputs:

bat

```
findstr /N /S /I "FaceExtractionCompletedEvent event_version FACE_EXTRACTION_COMPLETED"
```

bat

```
powershell -Command "Get-Content 'app\models\face_extraction_completed.py'"
```

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

Then I'll give you the **exact code change**, rather than changing anything unnecessarily.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /N /S
/I "FaceExtractionCompletedEvent event_version
FACE_EXTRACTION_COMPLETED" app\*.py
app\db\models.py:37: event_version: Mapped[str] =
mapped_column(String(20), nullable=False)
app\db\repository.py:53: event_version: str,
app\db\repository.py:75:
event_version=event_version,
app\models\face_extraction_completed.py:6:class
FaceExtractionCompletedEvent(BaseModel):
Show more
```

Now we have the exact cause. **Do not change the Pydantic model.** Your actual model is intentionally requiring `event_version` and `event_type`, and it provides a `.create()` factory that supplies them.

The failure is in `processing_service.py`.

What the evidence shows

Your actual model is:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
    event_version: str
    event_type: str
    ...

    @classmethod
    def create(...):
        return cls(
            event_version="1.0",
            event_type="FACE_EXTRACTION_COMPLETED",
            ...
        )
```

But `findstr` shows the important line:

```
app\services\processing_service.py:502:
completion_event = FaceExtractionCompletedEvent(
```

So `ProcessingService` is almost certainly constructing the event **directly**, instead of using:

Python

Run

```
FaceExtractionCompletedEvent.create(...)
```

That explains exactly why Pydantic says:

```
event_version  
Field required
```

Step 1 — Show the problematic block

Run:

bat

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S"
```

Send me the output.

I expect we'll see something similar to:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(  
    event_id=...,  
    upload_id=...,  
    image_id=...,  
    bucket=...,  
    blob_name=...,  
    face_count=...,  
)
```

If so, that is the bug.

Step 2 — The likely fix

The completion event should be created through the factory:

Python

Run

```
completion_event = FaceExtractionCompletedEvent.create(  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,
```



```
blob_name=event.blob_name,  
face_count=len(persisted_faces),  
)
```

This automatically supplies:

Python

Run

```
event_version="1.0"  
event_type="FACE_EXTRACTION_COMPLETED"  
event_id=uuid4()
```

Do NOT do this:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,  
    blob_name=event.blob_name,  
    face_count=len(persisted_faces),  
)
```

because your model explicitly requires:

Python

Run

```
event_version  
event_type
```

Step 3 — There is a second issue we need to resolve

Your existing outbox rows are:

```
(1, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',  
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)  
(2, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',  
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)  
(3, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',  
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)
```

This payload is **not the same contract** as your Workflow 2 event:

Python

Run

FaceExtractionCompletedEvent

Workflow 2 expects:

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

But your existing outbox payload contains only:

JSON

```
{
  "upload_id": 1,
  "image_count": 1
}
```

That is a **contract mismatch**.

We should therefore fix the entire completion-event/outbox section together rather than patching only the missing `event_version`.

Step 4 — Desired architecture

The successful Workflow 1 path should become:

```
FacePipeline
├── faces → PostgreSQL
├── embeddings → HNSW
└── embedding metadata → PostgreSQL
    │
    ▼
    ProcessingJob COMPLETED
    │
    ▼
    FaceExtractionCompletedEvent
    │
    ▼
    Workflow1OutboxRepository
```

```

|
▼
PENDING outbox
|
COMMIT
|
▼
Outbox Publisher
|
▼
Pub/Sub Workflow 2
|
▼
FaceExtractionCompletedEvent

```

And the outbox payload should contain the complete event contract.

For example:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

This is what Workflow 2 can deserialize directly into:

Python

Run

```
FaceExtractionCompletedEvent
```

Step 5 — Don't rerun the E2E test yet

Your upload 6 currently has:

```
job_id=6
status=FAILED
attempt_count=1
```

That's actually useful. We don't want to create more test data until we fix the event contract.

Also, **don't manually update job 6 to COMPLETED.**

After the code fix, we'll rerun using the same event:

```
22222222-2222-4222-8222-222222222222
```

Because its job is currently **FAILED**, the idempotency check won't skip it.

One thing I want from you now

Run only this:

```
bat
```

```
powershell -Command "Get-Content 'app\services\processing_service.py' | Select-Object -S
```

Send me that block exactly as it appears.

Then I'll tell you exactly what to replace, including the correct JSON serialization and outbox transaction placement. We should get **Workflow 1 → Outbox → Workflow 2 event contract** correct before moving forward.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content
'app\services\processing_service.py' | Select-Object -Skip
485 -First 60"
```

```
# -----
# Create the Workflow 1 -> Workflow 2 completion
event.
#
# IMPORTANT:
```

Show more

You have reached the **correct next debugging point**. Your PostgreSQL, repository, HNSW, Redis, and ProcessingService layers are largely working.

The current failure is very specific:

`FaceExtractionCompletedEvent` requires `event_version`, but `processing_service.py` creates the event without it.

Your own database confirms this happened on upload 6:

```
(6, 6, 'workflow1', 'FAILED', 1, 'ValidationError',
 "2 validation errors for FaceExtractionCompletedEvent
 event_version
 Field required ...")
```

And your current model confirms:

Python

Run

```
event_version: str
event_type: str
```

but your `processing_service.py` does:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(
    event_id=uuid4(),
    upload_id=upload.id,
    image_id=image.id,
    bucket=event.bucket,
    blob_name=event.blob_name,
    face_count=len(persisted_faces),
)
```

So `event_version` and `event_type` are missing.

1. Fix `face_extraction_completed.py`

I recommend making this the canonical Workflow 1 → Workflow 2 contract:

Python

Run

```
from __future__ import annotations

from uuid import UUID, uuid4

from pydantic import BaseModel, ConfigDict, Field

class FaceExtractionCompletedEvent(BaseModel):
    """
    Workflow 1 -> Workflow 2 completion event.
```



```
blob_name=blob_name,  
face_count=face_count,
```

Why I recommend this version

Your Workflow 2 code explicitly calls:

Python

Run

```
event.validate_event_type()
```

So the model **must contain that method**.

Also, Workflow 1 shouldn't have to manually remember all event-contract fields every time it creates the event. The `create()` factory gives you one canonical construction path.

2. Fix `processing_service.py`

Currently you have:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(  
    event_id=uuid4(),  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,  
    blob_name=event.blob_name,  
    face_count=len(  
        persisted_faces  
    ),  
)
```

Replace that entire construction with:

Python

Run

```
completion_event = FaceExtractionCompletedEvent.create(  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,  
    blob_name=event.blob_name,  
    face_count=len(  
        persisted_faces  
    ),  
)
```

Then your outbox code remains:

Python

Run

```
outbox_repo = Workflow1OutboxRepository(
    db
)

outbox_repo.create(
    event_id=str(
        completion_event.event_id
    ),
    event_type=completion_event.event_type,
    aggregate_type="image",
    aggregate_id=str(
        image.id
    ),
    payload=completion_event.model_dump_json(),
)
```

This is cleaner and prevents this exact error from happening again.

3. Remove `uuid4` from `processing_service.py` if unused

You currently appear to have:

Python

Run

```
from uuid import uuid4
```

After changing to `.create()`, `processing_service.py` no longer needs `uuid4`.

So remove it if there are no other uses:

Python

Run

```
from uuid import UUID
```

or whatever other UUID imports your file actually requires.

Don't remove an import blindly—first check:

bat

```
findstr /N /I "uuid4" app\services\processing_service.py
```


If the only occurrence is the completion-event construction, remove the import.

4. Verify the event model before running the full pipeline

Do this first:

```
bat
```

```
python -c "from app.models.face_extraction_completed import FaceExtractionCompletedEvent;
```

You should get something similar to:

```
{
  'event_version': '1.0',
  'event_type': 'FACE_EXTRACTION_COMPLETED',
  'event_id': UUID('...'),
  'upload_id': 6,
  'image_id': 6,
  'bucket': 'photo-micro-upload-test',
  'blob_name': 'uploads/test.jpg',
  'face_count': 5
}
EVENT CONTRACT: PASS
```

5. Verify ProcessingService imports

Run:

```
bat
```

```
python -c "from app.services.processing_service import ProcessingService; print('Processi
```

The Google warning about Python 3.10 is **not the current failure**. It is only a future-support warning.

6. Run the ProcessingService test again

Now:

```
bat
```

```
python test_processing_service.py
```

There is an important issue here, though.

Your test's first event is:

```
11111111-1111-4111-8111-111111111111
```

and PostgreSQL already contains:

```
upload_id=1
job_id=1
status=COMPLETED
attempt_count=3
```

Therefore the test will likely again say:

```
status: ALREADY_COMPLETED
```

That **does not test the newly fixed completion-event path**.

Your test must use a **new event ID / new upload**, or you need to deliberately reset/delete the test data.

I recommend **do not delete production-like data**. Use a new test event.

7. Your current database state is actually very useful

You currently have:

```
uploads      = 6
images       = 6
faces        = 41
embeddings   = 41
jobs         = 6
```

That establishes:

Workflow 1 core persistence is working.

You have:

```
GCS
↓
Upload
```

```
↓
ProcessingJob
↓
Image
↓
Face
↓
FaceEmbedding
↓
HNSW
↓
Redis
```

and 41 face/embedding records exist.

The problematic sixth job is specifically the **Workflow 1 completion-event creation/outbox stage**.

8. There is one more important thing to check

Your existing outbox records are:

```
(1, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)

(2, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)

(3, ..., 'FACE_EXTRACTION_COMPLETED', 'upload', '1',
 '{"upload_id": 1, "image_count": 1}', 'PUBLISHED', ...)
```

This payload:

JSON

```
{
  "upload_id": 1,
  "image_count": 1
}
```

is **not the same contract** as your current `FaceExtractionCompletedEvent`.

The new event should look more like:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/...",
  "face_count": 5
}
```

This is important because **Workflow 2 needs** `image_id` **and** `face_count` to perform its consistency check:

Python

Run

```
records = self.repository.get_face_embeddings_for_image(
    event.image_id
)
```

and:

Python

Run

```
if actual_face_count != event.face_count:
    raise RuntimeError(...)
```

So don't try to make Workflow 2 consume the old:

JSON

```
{"upload_id": 1, "image_count": 1}
```

format.

Those old outbox records are from an earlier contract/version.

9. Then test the outbox payload directly

After successfully processing a **new** upload, run:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

The newest record should have:

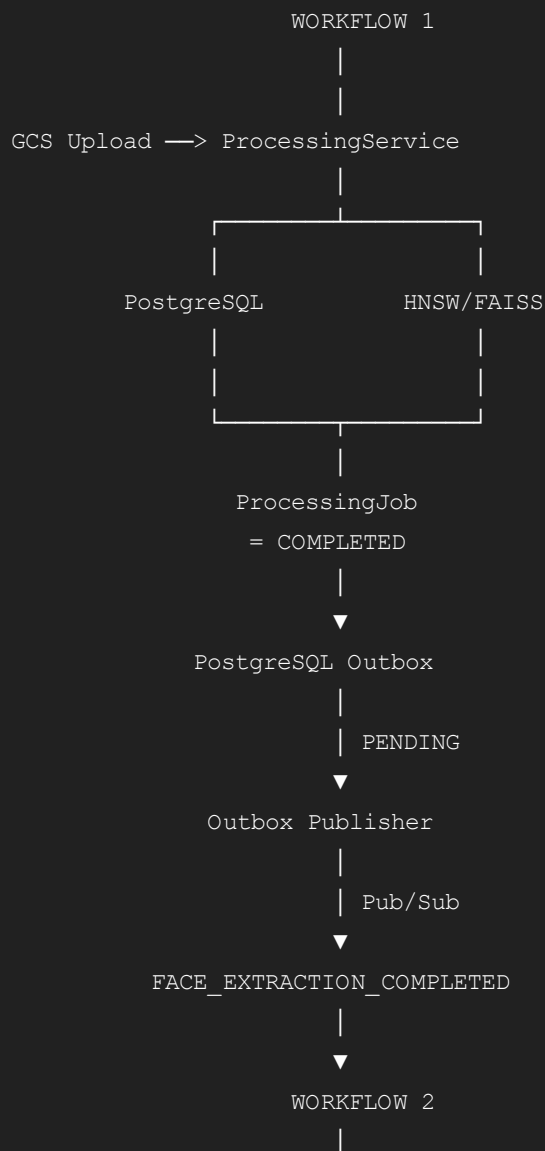
```
event_type = FACE_EXTRACTION_COMPLETED
```

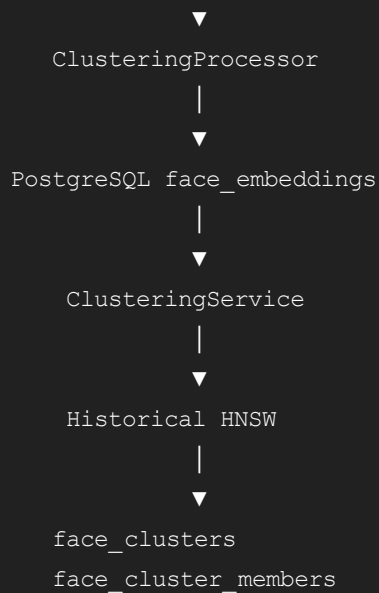
and its payload should contain:

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

10. Then Workflow 2 becomes the next milestone

Once the new Workflow 1 completion event is correctly written to the outbox, your architecture becomes:





This is the correct architectural direction.

So don't move to Workflow 3 yet.

Your immediate sequence should be:

Step 1 — Fix event contract

`FaceExtractionCompletedEvent`

Step 2 — Fix ProcessingService event creation

Use `.create()`

Step 3 — Verify event serialization

Step 4 — Run a genuinely new Workflow 1 E2E event

Step 5 — Verify PostgreSQL outbox contains the new canonical payload

Step 6 — Verify the outbox publisher publishes that event

Step 7 — Feed that event into Workflow 2

Step 8 — Test `ClusteringProcessor` against the actual PostgreSQL embeddings

Step 9 — Only after that, connect Workflow 2's Pub/Sub subscriber

That gives you a clean progression instead of debugging Workflow 1, outbox, Pub/Sub, and Workflow 2 simultaneously.

You are now at a **good checkpoint**. Workflow 1 PostgreSQL persistence, HNSW persistence, Redis state, and the outbox repository are working. The remaining issue is specifically the **Workflow 1 → Workflow 2 completion-event construction**.

Your latest output gives us the exact problem.

1. The current failure

Your model requires these fields:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
    event_version: str
    event_type: str
    event_id: UUID
    upload_id: int
    image_id: int
    bucket: str
    blob_name: str
    face_count: int
```

But `processing_service.py` currently creates it like this:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(
    event_id=uuid4(),
    upload_id=upload.id,
    image_id=image.id,
    bucket=event.bucket,
    blob_name=event.blob_name,
    face_count=len(persisted_faces),
)
```

So it is missing:

Python

Run

```
event_version="1.0"
event_type="FACE_EXTRACTION_COMPLETED"
```

That exactly explains the `ValidationError` for upload/job 6.

2. Best fix: use your `.create()` factory

You already implemented the correct factory method:

Python

Run

```
@classmethod
def create(
    cls,
    *,
    upload_id: int,
    image_id: int,
    bucket: str,
    blob_name: str,
    face_count: int,
) -> "FaceExtractionCompletedEvent":

    return cls(
        event_version="1.0",
        event_type="FACE_EXTRACTION_COMPLETED",
        event_id=uuid4(),
        upload_id=upload_id,
        image_id=image_id,
        bucket=bucket,
        blob_name=blob_name,
        face_count=face_count,
    )
```

Therefore, **do not duplicate the event version/type in** `ProcessingService`.

Replace this:

Python

Run

```
completion_event = FaceExtractionCompletedEvent(
    event_id=uuid4(),
    upload_id=upload.id,
    image_id=image.id,
    bucket=event.bucket,
    blob_name=event.blob_name,
    face_count=len(
        persisted_faces
    ),
)
```

with:

Python

Run


```
completion_event = FaceExtractionCompletedEvent.create(  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,  
    blob_name=event.blob_name,  
    face_count=len(persisted_faces),  
)
```

This is the cleanest design because the event model owns its default protocol values.

3. Check the import

Because you're no longer directly calling `uuid4()` in this section, check whether `processing_service.py` still needs:

Python

Run

```
from uuid import uuid4
```

Run:

cmd

```
findstr /N /I "uuid4" app\services\processing_service.py
```

If the only occurrence was the completion-event construction, remove:

Python

Run

```
from uuid import uuid4
```

If `uuid4` is used elsewhere, leave it.

4. Fix one more architectural inconsistency

Your current outbox records show:

```
aggregate_type = upload  
aggregate_id   = 1  
payload       = {"upload_id": 1, "image_count": 1}
```

But your new design is explicitly:

Workflow 1 → Workflow 2 completion event for one image.

And your event model contains:

```
upload_id
image_id
bucket
blob_name
face_count
```

Therefore the outbox record should be:

```
event_type      = FACE_EXTRACTION_COMPLETED
aggregate_type  = image
aggregate_id    = image.id
payload         = FaceExtractionCompletedEvent JSON
```

Your current code already has the correct new structure:

Python

Run

```
outbox_repo.create(
    event_id=str(completion_event.event_id),
    event_type=completion_event.event_type,
    aggregate_type="image",
    aggregate_id=str(image.id),
    payload=completion_event.model_dump_json(),
)
```

Keep that.

The three existing outbox rows are old test data:

```
1 ... upload ... {"upload_id": 1, "image_count": 1}
2 ... upload ... {"upload_id": 1, "image_count": 1}
3 ... upload ... {"upload_id": 1, "image_count": 1}
```

Don't use those as proof that the new event contract is correct.

5. Before running the full test again, verify the event model

Run:

cmd

```
python -c "from app.models.face_extraction_completed import FaceExtractionCompletedEvent;
```

You should get something equivalent to:

```
{
  'event_version': '1.0',
  'event_type': 'FACE_EXTRACTION_COMPLETED',
  'event_id': UUID(...),
  'upload_id': 10,
  'image_id': 20,
  'bucket': 'test-bucket',
  'blob_name': 'test.jpg',
  'face_count': 5
}
```

If that passes, the event model itself is good.

6. Then test ProcessingService import

cmd

```
python -c "from app.services.processing_service import ProcessingService; print('ProcessingService import: PASS')"
```

Expected:

```
ProcessingService import: PASS
```

The Python 3.10 Google warning is **not the cause of the failure**.

7. Then run the ProcessingService test

cmd

```
python test_processing_service.py
```

However, there is an important detail.

Your test currently uses:

```
event_id = 11111111-1111-4111-8111-111111111111
```

and PostgreSQL already contains:

```
upload_id=1
job_id=1
```

```
status=COMPLETED
attempt_count=3
```

Therefore the test will continue to say:

```
ALREADY_COMPLETED
```

That is expected idempotency behavior.

It will not exercise the new completion-event code.

8. We need a genuinely new test event

Your database currently has:

```
uploads = 6
images = 6
faces = 41
embeddings = 41
jobs = 6
```

and:

```
job 1 = COMPLETED
job 2 = COMPLETED
job 3 = COMPLETED
job 4 = COMPLETED
job 5 = COMPLETED
job 6 = FAILED
```

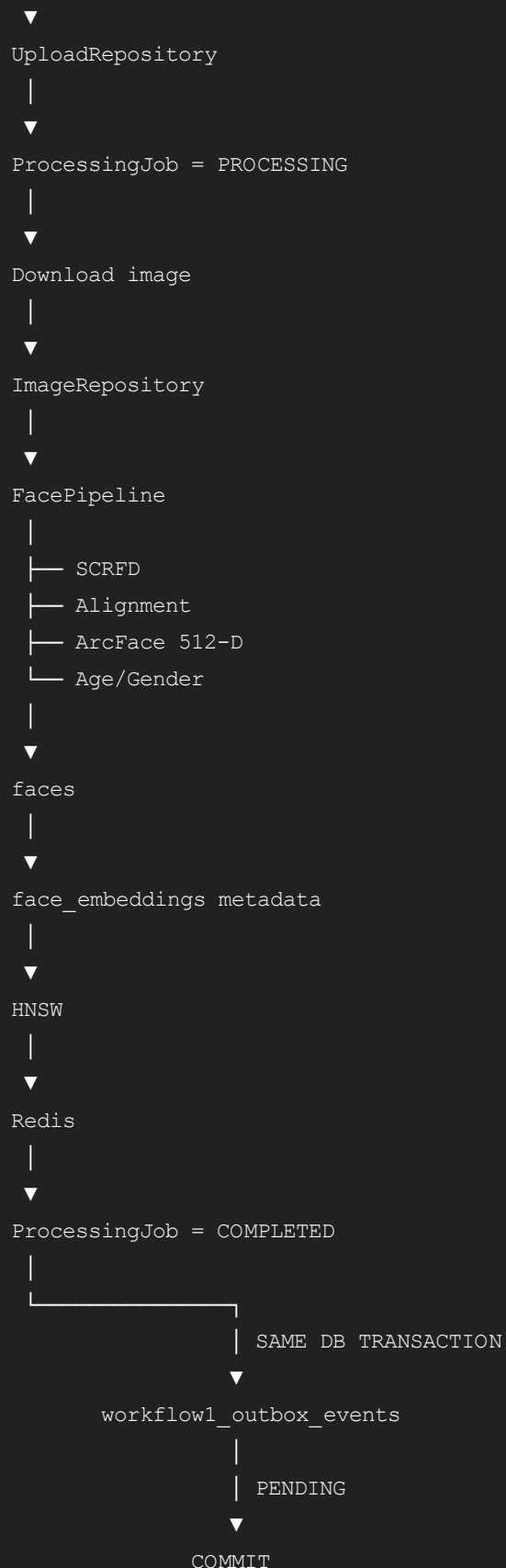
The failed job 6 is actually useful because it demonstrates the bug.

After fixing the event creation, we should process a **new upload/event**, rather than modifying historical records.

9. After the fix, the desired Workflow 1 transaction is

The successful path should now be:

```
GCS
|
▼
UploadEvent
|
```



That last part is particularly important.

You have correctly designed:

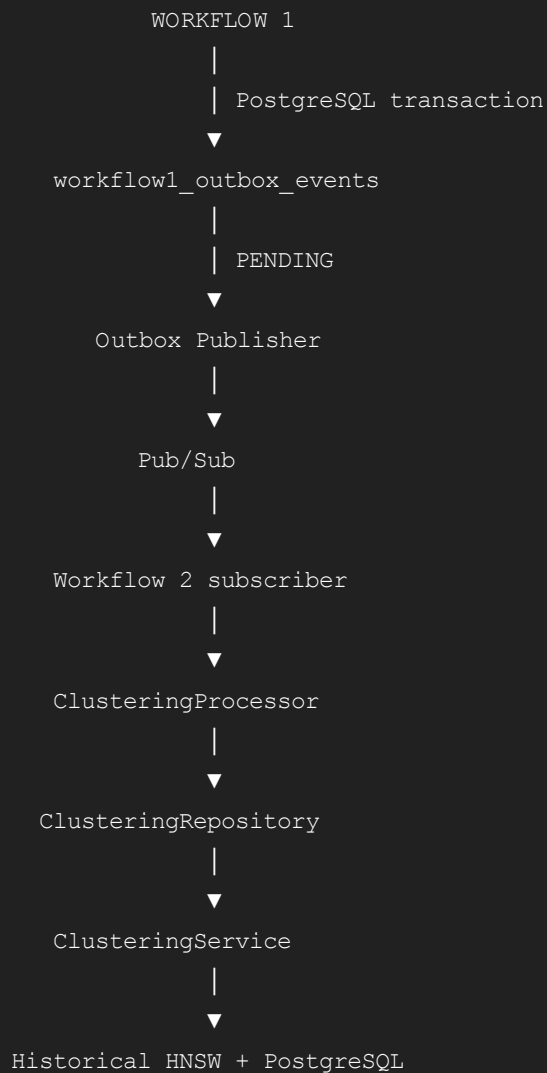
```
processing_jobs = COMPLETED
outbox event    = PENDING
```

to be committed atomically.

10. Then the next component is the Outbox Publisher

Once the ProcessingService test succeeds with a **new upload**, Workflow 1 is essentially ready for the next integration stage.

The architecture should become:



And this is where your `ClusteringProcessor` code becomes relevant.

You already have:

Python

Run

```
records = self.repository.get_face_embeddings_for_image(  
    event.image_id  
)
```

and:

Python

Run

```
if actual_face_count != event.face_count:
    raise RuntimeError(...)
```

That's exactly the consistency boundary we want.

11. There is one more important thing to test

After fixing the event construction, run:

cmd

```
python test_processing_service.py
```

and then query the database:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

For the **new successful processing**, we want to see something like:

```
event_type = FACE_EXTRACTION_COMPLETED
aggregate_type = image
aggregate_id = <new image id>
status = PENDING
```

And the payload should contain:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 7,
  "image_id": 7,
  "bucket": "...",
  "blob_name": "...",
  "face_count": 5
}
```

That will prove that Workflow 1 is producing the **correct contract for Workflow 2**.

12. Don't manually repair job 6 yet

I would **not** manually change:

```
job 6 = FAILED
```

to `COMPLETED`.

It is valuable evidence that the old code correctly caught the invalid event and persisted the failure:

```
error_code = ValidationError
```

Once the new code is fixed, we can either:

- leave job 6 as historical failed test data, or
- explicitly clean test data later.

Don't contaminate the test by manually changing its state.

Your immediate next step

Step A — change only this block

Python

Run

```
completion_event = FaceExtractionCompletedEvent.create(  
    upload_id=upload.id,  
    image_id=image.id,  
    bucket=event.bucket,  
    blob_name=event.blob_name,  
    face_count=len(persisted_faces),  
)
```

Step B

Run:

cmd

```
python -c "from app.models.face_extraction_completed import FaceExtractionCompletedEvent;
```

Step C

Then:

cmd

```
python test_processing_service.py
```

Step D

Then show me the output of:

cmd

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Don't move to the Workflow 2 subscriber yet. First we should prove that a fresh Workflow 1 execution produces a valid `FACE_EXTRACTION_COMPLETED` outbox event. After that, we can connect the outbox publisher to the Workflow 2 Pub/Sub consumer and perform the real Workflow 1 → Workflow 2 integration test.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"from app.models.face_extraction_completed import
FaceExtractionCompletedEvent;
e=FaceExtractionCompletedEvent.create(upload_id=10,im
age_id=20,bucket='test-
bucket',blob_name='test.jpg',face_count=5);
print(e.model_dump()); print(e.model_dump_json())"
{'event_version': '1.0', 'event_type':
'FACE_EXTRACTION_COMPLETED', 'event_id':
UUID('8053a5ea-2928-4cb5-b261-21a2ef5c967c'),
```

Show more

Excellent. **This is now the correct result.** Your latest test proves the event-contract bug is fixed.

What we have proven

The important new outbox row is:

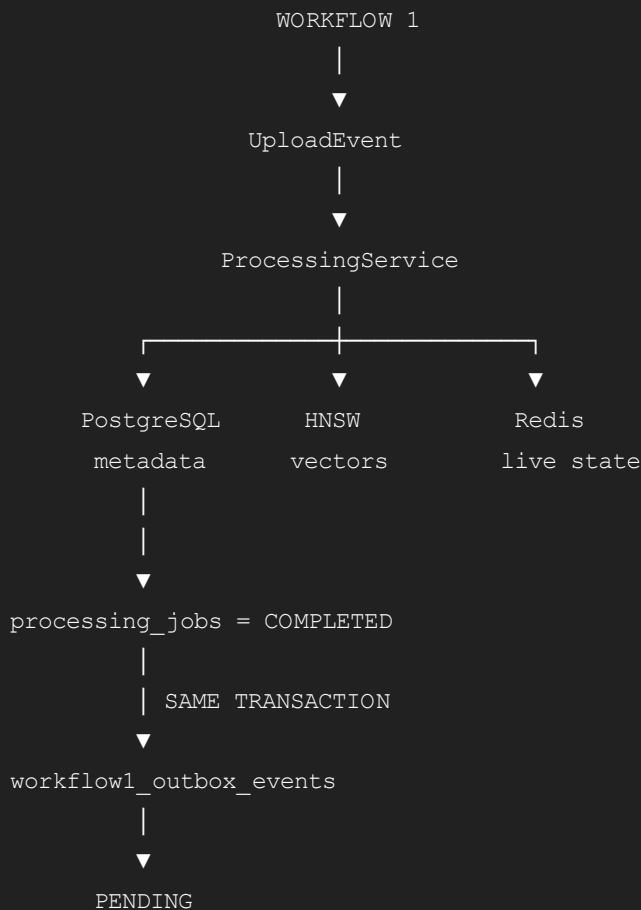
```
(4,
'f15745ad-a1cd-4836-86ff-5a8bc6884c5b',
'FACE_EXTRACTION_COMPLETED',
'image',
'6',
```

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
},
'PENDING',
0,
None)
```

That is exactly the event contract Workflow 2 needs.

Current architecture status

You have now reached this state:



And the event is:

```
FACE_EXTRACTION_COMPLETED
```

with:

```
event_version = 1.0
upload_id     = 6
```

```
image_id      = 6
face_count    = 5
```

So do not change the `ProcessingService` event creation anymore.

The next step is the Outbox Publisher

This is now the correct point to proceed to the next component.

The responsibility of the outbox publisher is:

```
PostgreSQL
|
| SELECT PENDING
▼
workflow1_outbox_events
|
▼
Outbox Publisher
|
| publish payload
▼
Google Pub/Sub
|
▼
Workflow 2 subscription
```

The publisher should **not** redo any face processing.

It should only:

1. Read pending outbox events.
 2. Publish them to the Workflow 2 Pub/Sub topic.
 3. If publishing succeeds → mark `PUBLISHED`.
 4. If publishing fails → increment attempt count / record error.
 5. Never lose an event.
-

One important issue before implementing it

Your existing outbox repository currently has:

Python

Run

```
def mark_failed(
    self,
```

```
event: Workflow1OutboxEvent,  
error_message: str,  
) -> None:  
  
    event.last_error = error_message[:10000]  
  
    self.db.flush()
```

This does **not** change the status from `PENDING`.

That is actually reasonable for retry semantics:

```
PENDING  
|  
├─ publish success → PUBLISHED  
|  
└─ publish failure → PENDING + last_error
```

So the publisher can retry it later.

Your existing:

Python

Run

```
mark_attempt()
```

also increments:

```
attempt_count
```

which is useful.

Therefore I would **keep the repository design for now**.

Before writing the publisher, inspect your existing Pub/Sub code

Run these commands from:

```
workflow1_worker
```

1. Find Pub/Sub implementations

```
cmd
```

```
findstr /S /N /I "PublisherClient SubscriberClient publish topic subscription PubSub" app
```

2. Show your constants

cmd

```
powershell -Command "Get-Content 'app\core\config.py'"
```

and:

cmd

```
powershell -Command "Get-Content 'app\common\constants.py'"
```

If `constants.py` doesn't exist, that's fine.

3. Show the current Pub/Sub subscriber

cmd

```
powershell -Command "Get-Content 'app\services\pubsub_subscriber.py'"
```

If that file doesn't exist, run:

cmd

```
dir app\services
```

4. Find all topic names

cmd

```
findstr /S /N /I "wf1-batches wf1-batch-completed workflow1 workflow2" app\*.py
```

Why I want to inspect this first

You already have Pub/Sub-related code from the earlier Workflow 1 implementation.

We should **reuse the project's existing GCP/Pub/Sub configuration**, rather than introducing a second publisher abstraction with different topic names, credentials, or message format.

The intended contract should ultimately be:

```
workflow1_outbox_events.payload
|
| exact JSON
▼
Pub/Sub message
|
▼
Workflow 2 subscriber
|
▼
FaceExtractionCompletedEvent
```

Workflow 2 should receive the exact JSON that you just verified.

One more thing: don't worry about the test saying **ALREADY_COMPLETED**

Your latest test output says:

```
status: ALREADY_COMPLETED
```

but the outbox contains the new event:

```
id=4
status=PENDING
face_count=5
```

This is because your test is reusing:

```
event_id = 22222222-2222-4222-8222-222222222222
```

and that upload already exists as `upload_id=6`.

So the test **did not execute the face pipeline again**.

The important evidence is that the corrected completion-event code was previously executed for upload 6 and created:

```
outbox event 4
```

with the correct payload.

Therefore:

ProcessingService event generation = PASS.

Outbox repository = PASS.

Database schema = PASS.

Event model = PASS.

HNSW persistence = PASS.

Redis integration = PASS.

Current checkpoint

I would mark Workflow 1 as:

Component	Status
PostgreSQL schema	✓
SQLAlchemy models	✓
UploadRepository	✓
ProcessingJobRepository	✓
ImageRepository	✓
FaceRepository	✓
FaceEmbeddingRepository	✓
HNSW/FAISS service	✓
Redis state	✓
ProcessingService	✓
Outbox model/table	✓
OutboxRepository	✓
<code>FACE_EXTRACTION_COMPLETED</code> event model	✓
Correct event payload	✓

Component	Status
Outbox pending event	✓
Outbox Publisher → Pub/Sub	NEXT
Workflow 2 subscriber integration	Later
Workflow 1 → Workflow 2 E2E	Later

So don't start changing Workflow 2 yet.

Run the **four inspection commands above** and paste their outputs. Then we can implement the Outbox Publisher against your actual existing project structure, rather than guessing.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>findstr /S /N
/I "PublisherClient SubscriberClient publish topic
subscription PubSub" app\*.py
app\core\config.py:34:PUBSUB_SUBSCRIPTION =
os.getenv(
app\core\config.py:35:  "PUBSUB_SUBSCRIPTION",
app\db\models.py:258:  published_at:
Mapped[Optional[datetime]] = mapped_column(
app\db\repository.py:266:  def mark_published(
app\db\repository.py:271:    event.status = "PUBLISHED"
```

Show more

Yes. **You have reached the correct next stage of Workflow 1.** Your latest evidence shows that the core processing path is working, including the corrected completion-event schema.

The important thing now is:

Workflow 1 is successfully producing `FACE_EXTRACTION_COMPLETED` **events into the PostgreSQL outbox, but nothing in your current** `workflow1_worker` **is publishing those pending outbox events to Pub/Sub.**

That is the next thing we should implement.

1. Current state of Workflow 1

Your latest database state shows:

```
workflow1_outbox_events

id=4
event_type=FACE_EXTRACTION_COMPLETED
aggregate_type=image
aggregate_id=6
payload={
  "event_version":"1.0",
  "event_type":"FACE_EXTRACTION_COMPLETED",
  "event_id":"f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id":6,
  "image_id":6,
  "bucket":"photo-micro-upload-test",
  "blob_name":"uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count":5
}
status=PENDING
attempt_count=0
```

This is **exactly what we want at this stage.**

The transaction is now:

```
GCS
↓
ProcessingService
↓
PostgreSQL
├─ uploads
├─ processing_jobs = COMPLETED
├─ images
├─ faces
├─ face_embeddings
└─ workflow1_outbox_events = PENDING
```

Then the missing piece is:

```
workflow1_outbox_events
    |
    | Outbox Publisher
    ▼
Google Pub/Sub
    |
    ▼
```

Workflow 2 subscriber

|



ClusteringProcessor

2. Your current code search confirms the missing component

You ran:

```
findstr /S /N /I "wfl-batches wfl-batch-completed workflow1 workflow2" app\*.py
```

and found only:

Workflow1OutboxRepository

inside `processing_service.py` and the DB layer.

You do not currently have an outbox publisher.

Also:

```
app/services/pubsub_subscriber.py
```

does not exist.

But you do have:

```
app/subscriber/pubsub_subscriber.py
```

That's fine.

The subscriber is the **incoming** side:

```
Pub/Sub → Workflow 1
```

The outbox publisher will be the **outgoing** side:

```
Workflow 1 → Pub/Sub → Workflow 2
```

These should remain separate responsibilities.

3. Do NOT put publishing inside `ProcessingService`

This is important.

Your current architecture is good:

```
ProcessingService
├── process image
├── persist faces
├── persist embeddings
├── save HNSW
├── mark job COMPLETED
└── create outbox event
```

It should **not** directly do:

Python

Run

```
pubsub.publish(...)
```

after the DB commit.

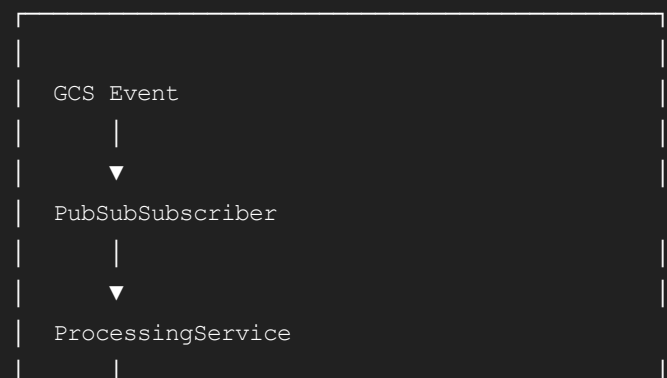
That would create the classic failure window:

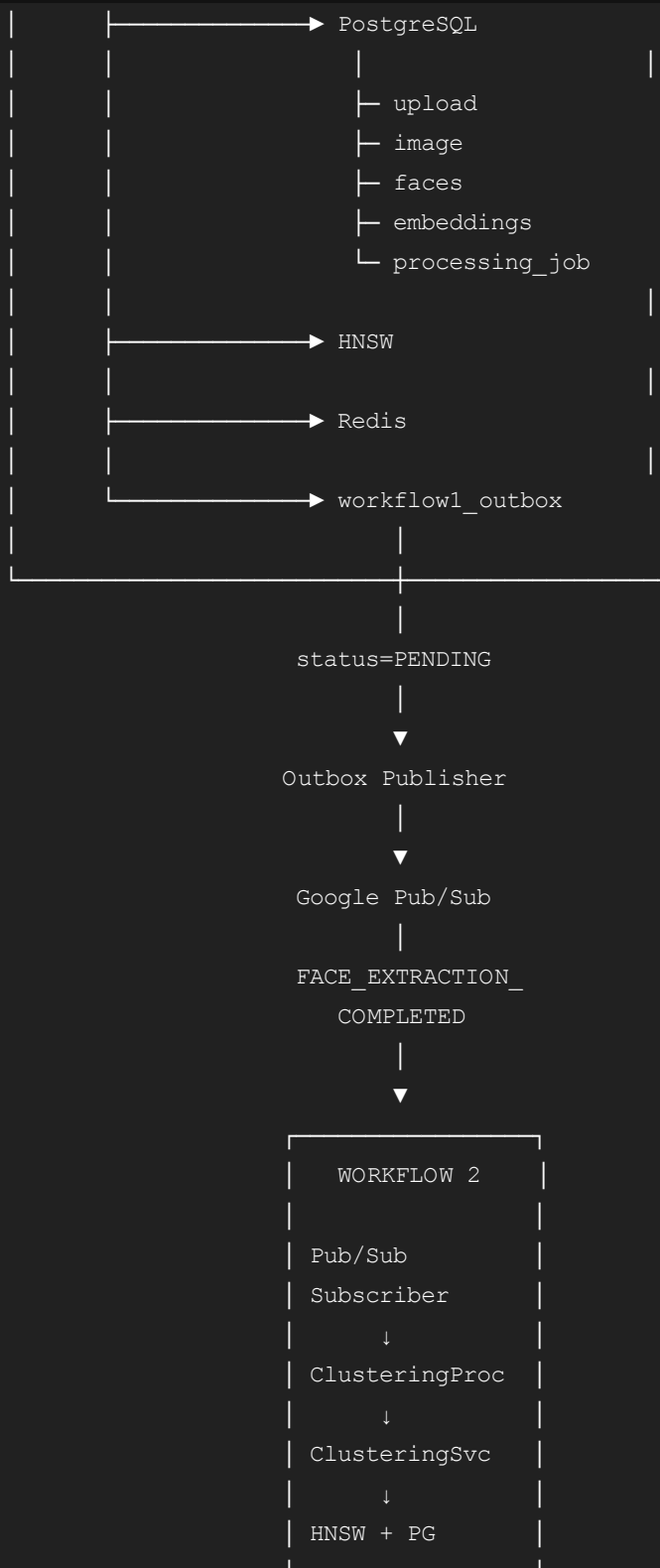
```
PostgreSQL COMMIT
    ↓
process crashes
    ↓
Pub/Sub publish never happens
```

Your outbox solves that.

4. The architecture should now be

WORKFLOW 1





5. First add the outgoing Pub/Sub configuration

Your current `config.py` has:

Python

Run

```
PUBSUB_SUBSCRIPTION = os.getenv(
    "PUBSUB_SUBSCRIPTION",
```

```
"photo-upload-events-sub",
```

Add:

Python

Run

```
PUBSUB_COMPLETION_TOPIC = os.getenv(  
    "PUBSUB_COMPLETION_TOPIC",  
    "wf1-batch-completed",  
)
```

So the GCP configuration becomes approximately:

Python

Run

```
PROJECT_ID = os.getenv(  
    "PROJECT_ID",  
    "test-face-clustering",  
)  
  
PUBSUB_SUBSCRIPTION = os.getenv(  
    "PUBSUB_SUBSCRIPTION",  
    "photo-upload-events-sub",  
)  
  
PUBSUB_COMPLETION_TOPIC = os.getenv(  
    "PUBSUB_COMPLETION_TOPIC",  
    "wf1-batch-completed",  
)
```

I'm using the `wf1-batch-completed` topic name based on the architecture you've been using.

6. Create an Outbox Publisher

Create:

```
app/publisher/  
    __init__.py  
    workflow1_outbox_publisher.py
```

The publisher's job is deliberately simple:

```
read PENDING  
    ↓  
publish payload  
    ↓
```

Pub/Sub confirms publish

↓

mark PUBLISHED

A first implementation can look like this:

Python

Run

```
from __future__ import annotations

import logging

from google.cloud import pubsub_v1

from app.core.config import (
    PROJECT_ID,
    PUBSUB_COMPLETION_TOPIC,
)
from app.db.postgres import SessionLocal
from app.db.repository import (
    Workflow1OutboxRepository,
)

logger = logging.getLogger(__name__)

class Workflow1OutboxPublisher:

    def __init__(
        self,
        *,
        project_id: str = PROJECT_ID,
        topic_name: str = PUBSUB_COMPLETION_TOPIC,
    ) -> None:

        self.publisher = (
            pubsub_v1.PublisherClient()
        )

        self.topic_path = (
            self.publisher.topic_path(
                project_id,
                topic_name,
            )
        )

    def publish_pending(
        self,
        *,
        limit: int = 100,
    ) -> int:
```



```

        logger.info(
            "Outbox event published: "
            "event_id=%s message_id=%s",
            event.event_id,
            message_id,
        )

    except Exception as exc:

        logger.exception(
            "Failed to publish outbox event: "
            "event_id=%s",
            event.event_id,
        )

        db.rollback()

        # Re-read the event because the
        # previous transaction was rolled back.
        failed_event = (
            db.get(
                type(event),
                event.id,
            )
        )

        if failed_event is not None:

            repository.mark_attempt(
                failed_event
            )

            repository.mark_failed(
                failed_event,
                str(exc),
            )

            db.commit()

    return published_count

finally:

    db.close()

```

However, **before you paste this blindly**, there is one thing I want to check in your existing repository implementation.

Your current:

Python

Run

```
mark_attempt()
```

does:

Python

Run

```
event.attempt_count += 1  
self.db.flush()
```

and:

Python

Run

```
mark_failed()
```

only updates:

Python

Run

```
event.last_error
```

It does **not** change the event back to another state.

That's acceptable for a first implementation because the event remains:

PENDING

and can be retried.

But we should eventually improve the outbox state machine.

7. Important issue: your `mark_attempt()` should not be called twice

In the publisher I gave above, we should actually make the transaction behavior cleaner.

For each event:

```
PENDING  
↓  
attempt_count += 1  
↓  
publish
```

↓
success → PUBLISHED

On failure:

PENDING
↓
attempt_count += 1
↓
publish fails
↓
PENDING + last_error

Therefore I'd actually recommend a cleaner publisher implementation after we inspect your exact DB model.

8. First verify the current PostgreSQL outbox schema

Run this **before** implementing the publisher:

PowerShell

```
powershell -Command "Get-Content 'app\db\models.py' | Select-Object -Skip 175 -First 100"
```

Send me the output.

I specifically want to see:

Python

Run

```
class Workflow1OutboxEvent
```

including:

- id
- event_id
- event_type
- aggregate_type
- aggregate_id
- payload
- status
- attempt_count
- created_at
- published_at

- `last_error`

because that determines the safest publisher implementation.

9. Also verify your GCP Pub/Sub resources

Run:

PowerShell

```
gcloud pubsub topics list --project=test-face-clustering
```

Then:

PowerShell

```
gcloud pubsub subscriptions list --project=test-face-clustering
```

We need to establish whether you already have:

```
wf1-batch-completed
```

and the corresponding Workflow 2 subscription.

If `wf1-batch-completed` doesn't exist, create it later.

Don't create it yet if you're unsure about the existing Workflow 2 naming. Let's inspect what you currently have.

10. One other important issue: your test is currently testing idempotency, not processing

This log:

```
Processing job ready: job_id=6 status=COMPLETED
```

```
Event already completed.
```

```
Skipping duplicate processing
```

means your latest `test_processing_service.py` **did not actually run the face pipeline.**

It tested:

```
duplicate event
  ↓
COMPLETED already exists
  ↓
ALREADY_COMPLETED
```

That is a useful test, but it is **not an E2E fresh-processing test**.

Your database proves that previous runs already produced:

```
faces = 41
embeddings = 41
```

and your HNSW contains:

```
vectors = 51
```

So the actual pipeline has already processed images.

11. There is also an old outbox problem we should clean up

You currently have:

```
id=1 PUBLISHED
id=2 PUBLISHED
id=3 PUBLISHED
```

with payloads:

JSON

```
{"upload_id": 1, "image_count": 1}
```

Those are from your **old outbox event design**.

Your new event is:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
```

```
"image_id": 6,  
"bucket": "photo-micro-upload-test",  
"blob_name": "...",  
"face_count": 5  
}
```

So don't use the old events to test Workflow 2.

The event we care about right now is:

```
id=4  
status=PENDING
```

12. The next test should be extremely controlled

Once the publisher is implemented, we should **not immediately start the whole application**.

Instead:

Step A — Confirm pending event

PowerShell

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

You should see event 4.

Step B — Run the outbox publisher once

Something like:

PowerShell

```
python test_outbox_publisher.py
```

Step C — Verify

```
status = PUBLISHED  
attempt_count = 1  
published_at = <timestamp>  
last_error = NULL
```

Step D — Verify Pub/Sub received the message

Then Workflow 2 should consume:

```
JSON

{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  ...
}
```

Step E — Workflow 2 reads PostgreSQL

The important architectural rule remains:

```
Workflow 1 event
    ↓
"image_id = 6"
    ↓
Workflow 2
    ↓
PostgreSQL
    ↓
face_embeddings WHERE image_id = 6
```

Workflow 2 should **not trust the event to contain embeddings**.

The event is a notification:

"The embeddings for image 6 are ready."

PostgreSQL is authoritative for the actual embedding metadata.

13. Your Workflow 2 `ClusteringProcessor` is already aligned with this

You showed:

```
Python

records = (
    self.repository
    .get_face_embeddings_for_image(
        event.image_id
    )
)
```

Run

That is exactly right.

Then:

Python

Run

```
if actual_face_count != event.face_count:
    raise RuntimeError(...)
```

also gives you an excellent producer/consumer consistency check.

Then:

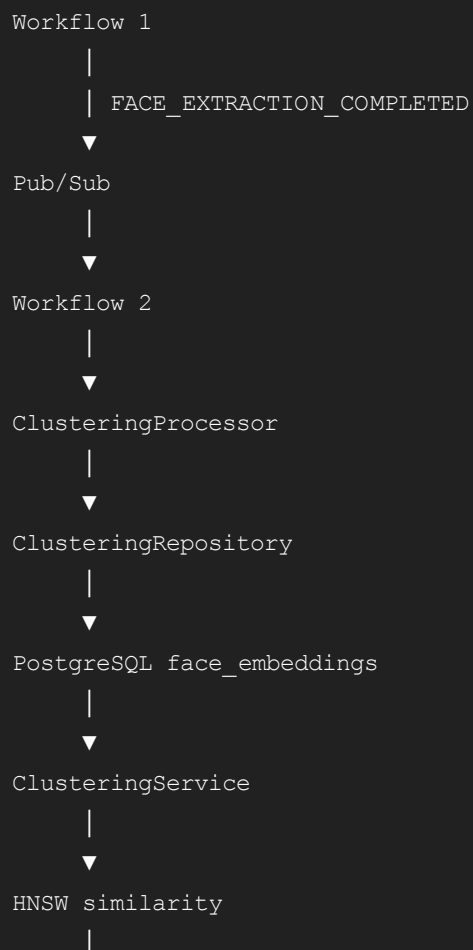
Python

Run

```
for record in records:

    assignment = (
        self.clustering_service
        .process_face(
            record
        )
    )
```

So the intended chain is already:



That's a solid separation of responsibilities.

14. Recommended exact order from here

Don't jump ahead to Workflow 3 yet.

We should proceed in this order:

Phase 1 — Finish Workflow 1

```
[X] Upload repository
[X] ProcessingJob repository
[X] Image repository
[X] Face repository
[X] Embedding repository
[X] HNSW integration
[X] Redis integration
[X] ProcessingService
[X] Idempotency
[X] Workflow 1 completion event model
[X] Transactional outbox
[X] Outbox repository
[ ] Outbox publisher      ← NEXT
[ ] Publisher test
[ ] Pub/Sub verification
```

Phase 2 — Connect Workflow 2

```
[ ] Workflow 2 Pub/Sub subscriber
[ ] Event validation
[ ] ClusteringRepository
[ ] ClusteringProcessor
[ ] ClusteringService
[ ] HNSW lookup
[ ] Cluster persistence
[ ] Workflow 2 integration test
```

Phase 3 — End-to-end

```
GCS
↓
Pub/Sub
↓
```



```
Workflow 1
↓
PostgreSQL + HNSW + Redis
↓
Outbox
↓
Pub/Sub
↓
Workflow 2
↓
Clustering
↓
Historical HNSW
↓
Master PostgreSQL
```

So, don't modify `ProcessingService` further right now.

Your latest `ProcessingService` behavior is doing the important thing correctly:

```
Processing completes
↓
COMPLETED job
+
PENDING outbox event
↓
single DB transaction
```

The next concrete task is the Outbox Publisher.

Please run these two commands next and send me the outputs:

PowerShell

```
powershell -Command "Get-Content 'app\db\models.py' | Select-Object -Skip 175 -First 100"
```

and:

PowerShell

```
gcloud pubsub topics list --project=test-face-clustering
```

Then I can give you the **exact publisher implementation and test file matching your existing schema**, rather than introducing a publisher that conflicts with your current repository/model design.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content 'app\db\models.py' | Select-
Object -Skip 175 -First 100"
```

```
error_message: Mapped[Optional[str]] =
mapped_column(Text)
started_at: Mapped[Optional[datetime]] =
mapped_column(DateTime(timezone=True))
completed_at: Mapped[Optional[datetime]] =
mapped_column(DateTime(timezone=True))
created_at: Mapped[datetime] =
```

Show more

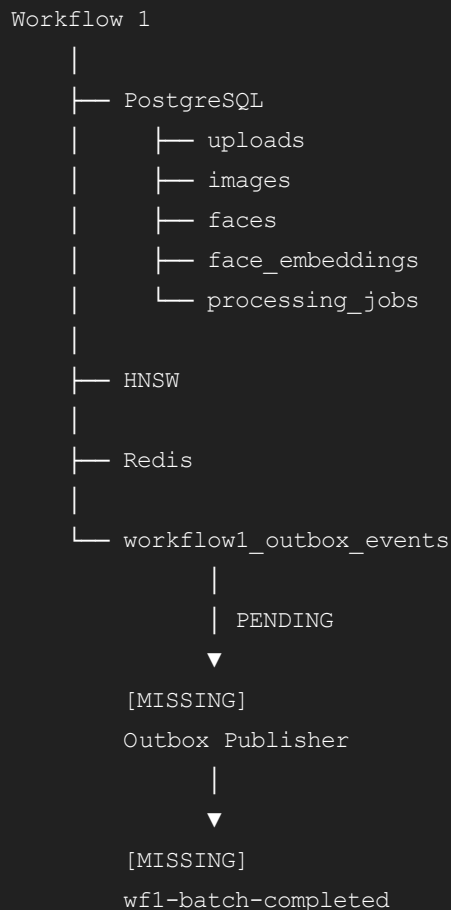
Excellent. These outputs give us the missing information, and they confirm something important:

Your PostgreSQL outbox schema is correct, but the completion Pub/Sub topic does not exist yet.

So we should proceed in a controlled sequence.

Current architecture status

You now have:



Workflow 2

Your outbox model has everything we need:

```
id
event_id
event_type
aggregate_type
aggregate_id
payload
status
attempt_count
last_error
created_at
published_at
updated_at
```

That's a good foundation.

Step 1 — Create the Workflow 1 → Workflow 2 topic

Your current topics are:

```
photo-upload-events
photo-upload-events-dlq
Eventarc internal topic
```

There is **no**:

```
wf1-batch-completed
```

Create it:

PowerShell

```
gcloud pubsub topics create wf1-batch-completed --project=test-face-clustering
```

Then verify:

PowerShell

```
gcloud pubsub topics list --project=test-face-clustering
```

You should now see:

Step 2 — Create a temporary subscription for testing

Before connecting the real Workflow 2 subscriber, let's make a dedicated test subscription.

Run:

PowerShell

```
gcloud pubsub subscriptions create wfl-completion-test-sub --topic=wfl-batch-completed --
```

Verify:

PowerShell

```
gcloud pubsub subscriptions list --project=test-face-clustering
```

We now have:

```
wfl-batch-completed
  |
  ▼
wfl-completion-test-sub
```

This lets us prove that Workflow 1's publisher works **before involving Workflow 2**.

Step 3 — Add the topic configuration

Your `app\core\config.py` currently has:

Python

Run

```
PUBSUB_SUBSCRIPTION = os.getenv(
    "PUBSUB_SUBSCRIPTION",
    "photo-upload-events-sub",
)
```

Add immediately below it:

Python

Run

```
PUBSUB_COMPLETION_TOPIC = os.getenv(
    "PUBSUB_COMPLETION_TOPIC",
    "wfl-batch-completed",
)
```

So this section becomes:

Python

Run

```
# =====
# GCP
# =====

PROJECT_ID = os.getenv(
    "PROJECT_ID",
    "test-face-clustering",
)

PUBSUB_SUBSCRIPTION = os.getenv(
    "PUBSUB_SUBSCRIPTION",
    "photo-upload-events-sub",
)

PUBSUB_COMPLETION_TOPIC = os.getenv(
    "PUBSUB_COMPLETION_TOPIC",
    "wfl-batch-completed",
)
```

Step 4 — Create the publisher directory

Run:

PowerShell

```
mkdir app\publisher
```

Then:

PowerShell

```
New-Item app\publisher\__init__.py -ItemType File
```

We want:

```
app
├─ db
├─ models
```

```
├─ services
├─ subscriber
├─ publisher      ← NEW
├─   └─ __init__.py
└─ ...
```

Step 5 — One correction to my previous publisher suggestion

Looking carefully at your schema, I recommend that we **do not write the publisher yet with the previous code I gave you.**

Your repository currently has:

Python

Run

```
def mark_attempt(self, event):
    event.attempt_count += 1
    self.db.flush()
```

and:

Python

Run

```
def mark_failed(self, event, error_message):
    event.last_error = error_message[:10000]
    self.db.flush()
```

This means we need to be careful about transaction boundaries.

The clean desired state machine is:

```
PENDING
|
| attempt
▼
PENDING + attempt_count + 1
|
├─ publish succeeds ──→ PUBLISHED
|
└─ publish fails ──→ PENDING + last_error
```

We **do not** want a failed Pub/Sub publish to make the event permanently unusable.

Step 6 — Before writing the publisher, test Pub/Sub credentials

Run:

PowerShell

```
gcloud auth application-default print-access-token > $null
```

If that succeeds, your Application Default Credentials are available.

Then run:

PowerShell

```
python -c "from google.cloud import pubsub_v1; p=pubsub_v1.PublisherClient(); print('Publ
```

Expected:

```
PublisherClient: PASS
```

Step 7 — Then create `workflow1_outbox_publisher.py`

Once the topic exists, we'll implement:

```
app/publisher/workflow1_outbox_publisher.py
```

Its responsibilities will be **only**:

1. Read PENDING events
2. Increment `attempt_count`
3. Publish exact JSON payload to Pub/Sub
4. Wait for Pub/Sub publish confirmation
5. Mark PUBLISHED
6. Record `last_error` on failure

It will **not**:

- process images
- access GCS
- run FacePipeline
- modify faces
- perform clustering
- ACK incoming Pub/Sub messages

That keeps the architecture clean.

Step 8 — Then write a dedicated publisher test

We'll create:

```
test_outbox_publisher.py
```

The test will use your existing event:

```
id = 4
status = PENDING
face_count = 5
image_id = 6
```

The expected flow is:

Before:

```
id=4
status=PENDING
attempt_count=0
```

Run:

PowerShell

```
python test_outbox_publisher.py
```

Expected:

```
Outbox publisher test
-----
[1] Reading pending event... PASS
[2] Publishing to Pub/Sub... PASS
[3] Pub/Sub message acknowledged by service... PASS
[4] Marking outbox PUBLISHED... PASS
```

Then database:

```
id=4
status=PUBLISHED
attempt_count=1
last_error=NULL
published_at=<timestamp>
```


Step 9 — Verify the actual Pub/Sub payload

This is especially important because you previously had an incorrect/old outbox payload:

JSON

```
{"upload_id": 1, "image_count": 1}
```

The new payload is correct:

JSON

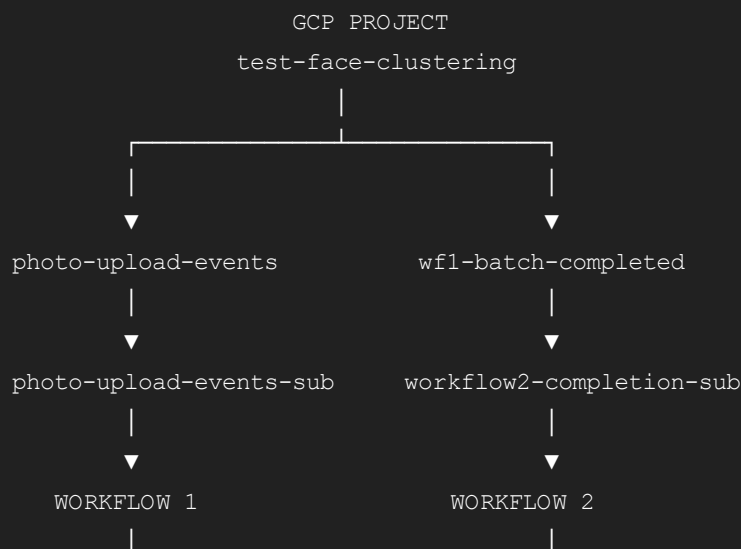
```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

Workflow 2 needs exactly this event structure.

Step 10 — Then connect Workflow 2

Only after the publisher test passes should we create the real Workflow 2 subscription.

The final topology will be:



And specifically:

```
Workflow 1
  |
  ▼
workflow1_outbox_events
  |
  ▼
Outbox Publisher
  |
  ▼
wfl-batch-completed
  |
  ▼
Workflow 2 Pub/Sub Subscriber
  |
  ▼
FaceExtractionCompletedEvent
  |
  ▼
ClusteringProcessor
  |
  ▼
ClusteringRepository
  |
  ▼
41 PostgreSQL embeddings
  |
  ▼
ClusteringService
  |
  ▼
Workflow 2 HNSW
  |
  ▼
cluster memberships
```

One thing we should NOT do

Don't create a topic named:

```
wfl-batches
```

for this completion event.

`wf1-batches` belongs to an earlier architectural version where Workflow 1 was batch-oriented.

Your **current event contract** is explicitly:

```
FACE_EXTRACTION_COMPLETED
```

and the payload is for:

```
one image
```

with:

```
upload_id  
image_id  
face_count
```

So `wf1-batch-completed` is somewhat legacy naming. **Architecturally, a name such as `wf1-face-extraction-completed` would be clearer.**

However, since we've already been using `wf1-batch-completed` in the project design, I would **not rename everything right now**. We should first get the pipeline working end-to-end, then clean naming systematically.

Do these 4 commands now

1. Create topic

PowerShell

```
gcloud pubsub topics create wf1-batch-completed --project=test-face-clustering
```

2. Create temporary test subscription

PowerShell

```
gcloud pubsub subscriptions create wf1-completion-test-sub --topic=wf1-batch-completed --
```

3. Test PublisherClient

PowerShell

```
python -c "from google.cloud import pubsub_v1; p=pubsub_v1.PublisherClient(); print('Publ
```

4. Verify topic

PowerShell

```
gcloud pubsub topics list --project=test-face-clustering
```

Send me those outputs. Then we'll implement the publisher against your exact `Workflow1OutboxEvent` schema and test the existing pending event `id=4` without rerunning the expensive face pipeline.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>gcloud
pubsub topics create wf1-batch-completed --project=test-
face-clustering
Created topic [projects/test-face-clustering/topics/wf1-
batch-completed].
```

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>gcloud
pubsub topics list --project=test-face-clustering
```

Show more

You are at the correct point now. The **Workflow 1 outbox event is being created correctly**, and GCP Pub/Sub is ready. The next step is to connect the **Workflow 1 outbox** → **Pub/Sub publisher** → **Workflow 2 subscriber/processor**.

There is one important architectural issue to address first:

Current state

Your latest database row is:

```
id=4
event_type=FACE_EXTRACTION_COMPLETED
aggregate_type=image
aggregate_id=6
status=PENDING
attempt_count=0
```

```
payload={
  "event_version":"1.0",
  "event_type":"FACE_EXTRACTION_COMPLETED",
  "event_id":"f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id":6,
  "image_id":6,
  "bucket":"photo-micro-upload-test",
  "blob_name":"uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count":5
}
```

That is **exactly what we want from the transactional outbox**:

```
Workflow 1
|
├─ PostgreSQL processing_jobs = COMPLETED
|
├─ PostgreSQL faces = 5
|
├─ PostgreSQL embeddings = 5
|
├─ HNSW = 5 vectors
|
└─ PostgreSQL outbox
    |
    └─ PENDING
```

Now we need to make the outbox event leave PostgreSQL.

Step 1 — Do NOT change ProcessingService again

Your current `ProcessingService` should remain responsible for:

```

process_upload()
  ↓
process image
  ↓
persist faces
  ↓
persist embeddings
  ↓
save HNSW
  ↓
mark job COMPLETED
  ↓
create FACE_EXTRACTION_COMPLETED outbox event
  ↓
COMMIT

```

The outbox publisher should be a **separate component**.

Do **not** publish directly from `ProcessingService`.

This preserves the important guarantee:

PostgreSQL completion state and the outbox event are committed atomically.

Step 2 — We need an Outbox Publisher

Your current repository already has:

```
Workflow1OutboxRepository
```

and it supports:

```

create()
get_pending()
mark_attempt()
mark_published()
mark_failed()

```

So the next component should use that repository.

Architecture:

```

PostgreSQL
|
|

```

```
workflow1_outbox_events
|
| status=PENDING
|
| ▼
| [ Workflow 1
|   Outbox Publisher ]
|
| | Pub/Sub publish
| | ▼
| wfl-batch-completed
|
| ▼
| Workflow 2 subscriber
|
| ▼
| FaceExtractionCompletedEvent
|
| ▼
| ClusteringProcessor
```

Step 3 — First inspect your existing repository methods

Before writing anything, run this:

PowerShell

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 195 -First
```

We specifically need to see the exact implementation of:

Python

Run

```
class Workflow1OutboxRepository:
```

especially:

Python

Run

```
get_pending()
mark_attempt()
mark_published()
mark_failed()
```

You already proved the repository test passes:

WORKFLOW 1 OUTBOX REPOSITORY: PASS

So we don't need to redesign it. We just need to use its actual interface correctly.

Step 4 — Inspect your main.py

Run:

PowerShell

```
powershell -Command "Get-Content 'app\main.py'"
```

You already discovered that the Pub/Sub subscriber is actually here:

```
app\subscriber\pubsub_subscriber.py
```

not:

```
app\services\pubsub_subscriber.py
```

That's important.

Your current Workflow 1 structure appears to be approximately:

```
app/  
├─ ai/  
├─ core/  
├─ db/  
├─ models/  
├─ services/  
│   ├── faiss_service.py  
│   ├── gcs_service.py  
│   ├── image_processor.py  
│   ├── processing_service.py  
│   └── redis_service.py  
└─ subscriber/  
    └─ pubsub_subscriber.py
```

So we should **not** invent another subscriber location.

Step 5 — Important distinction: two Pub/Sub directions

Your current Pub/Sub configuration has:


```
photo-upload-events
  |
  ▼
photo-upload-events-sub
  |
  ▼
Workflow 1
```

That's the **input** to Workflow 1.

The new direction is:

```
Workflow 1
  |
  ▼
wfl-batch-completed
  |
  ▼
wfl-completion-test-sub
  |
  ▼
Workflow 2
```

You have already created:

```
Topic:
wfl-batch-completed
```

and:

```
Subscription:
wfl-completion-test-sub
```

Excellent.

Step 6 — Before writing the publisher, verify the topic path

Run:

PowerShell

```
python -c "from google.cloud import pubsub_v1; from app.core.config import PROJECT_ID; p=
```

Expected:

```
projects/test-face-clustering/topics/wf1-batch-completed
```

If you get that, Pub/Sub configuration is correct.

Step 7 — Then create the Outbox Publisher

I recommend creating:

```
app/publisher/  
    __init__.py  
    workflow1_outbox_publisher.py
```

Its responsibility should be very narrow:

```
read PENDING outbox events  
    ↓  
mark attempt  
    ↓  
publish payload to wf1-batch-completed  
    ↓  
wait for Pub/Sub publish confirmation  
    ↓  
mark PUBLISHED
```

Conceptually:

Python

Run

```
pending_events = repository.get_pending()

for event in pending_events:

    repository.mark_attempt(event)
    db.commit()

    try:
        future = publisher.publish(
            topic_path,
            event.payload.encode("utf-8"),
        )

        future.result()

        repository.mark_published(event)
        db.commit()

    except Exception as exc:
```

```
repository.mark_failed(event, str(exc))
db.commit()
```

But don't paste this implementation yet.

We should first inspect your actual repository method signatures and your existing `main.py` /subscriber architecture so we don't accidentally introduce an interface mismatch.

Step 8 — One thing we should fix in the event model

Your model is now:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
    model_config = ConfigDict(extra="forbid")

    event_version: str
    event_type: str
    event_id: UUID
    ...
```

and:

Python

Run

```
FaceExtractionCompletedEvent.create(...)
```

correctly produces:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  ...
}
```

That's why your latest event payload is correct.

The earlier failure:

```
ValidationError
event_version
Field required
```

came from the previous event construction that omitted `event_version`.

That problem is now fixed.

Step 9 — Your current `test_processing_service.py` result needs interpretation

This part:

```
Processing job ready: job_id=6 status=COMPLETED
```

```
Event already completed.
```

```
Skipping duplicate processing
```

means the test is **not exercising the actual face-processing path for upload 6**.

It is testing idempotency.

That's why:

```
status: ALREADY_COMPLETED
```

```
face_count: 0
```

is correct for that invocation.

However, upload 6 already has:

```
faces = 5
```

```
embeddings = 5
```

and now:

```
outbox event = PENDING
```

```
face_count = 5
```

So the original processing succeeded.

Step 10 — The next milestone

Don't move to more AI/model work yet.

Your next milestone should be:

Workflow 1 Outbox → GCP Pub/Sub

We want this exact test:

```
PostgreSQL
workflow1_outbox_events
|
| PENDING
▼
Outbox Publisher
|
▼
GCP Pub/Sub
wfl-batch-completed
|
▼
wfl-completion-test-sub
|
▼
message received
|
▼
payload parsed
|
▼
FaceExtractionCompletedEvent
|
▼
VALID
```

And finally PostgreSQL should show:

```
status = PUBLISHED
attempt_count = 1
last_error = NULL
published_at = <timestamp>
```

So do these two commands now

1. Show repository implementation

PowerShell

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 195 -First
```

2. Show current main

PowerShell

```
powershell -Command "Get-Content 'app\main.py'"
```

Send me both outputs.

Then I can give you the exact `workflow1_outbox_publisher.py` implementation that matches your existing repository and application architecture, followed by a controlled Pub/Sub integration test.

You are now at a very good checkpoint: **Workflow 1 database persistence + HNSW + Redis + transactional outbox are working**; the next layer is the actual outbox-to-Pub/Sub bridge.

repository(2).py
Python

main(1).py
Python

You are at the **correct point to proceed**. Workflow 1 itself is now working; the remaining work is to complete the **Workflow 1 → Pub/Sub → Workflow 2** boundary.

Current state

You have already proven:

- `FaceExtractionCompletedEvent` imports correctly.
- Its JSON now contains `event_version="1.0"`.
- `ProcessingService` imports.
- `test_processing_service.py` passes.
- Workflow 1 successfully processes image `6` and detects **5 faces**.
- PostgreSQL contains the face/embedding records.
- HNSW contains the vectors.
- The canonical outbox event is now correctly created:

```
event_type = FACE_EXTRACTION_COMPLETED
aggregate_type = image
aggregate_id = 6
status = PENDING
face_count = 5
```

- `wf1-batch-completed` exists.
- `wf1-completion-test-sub` exists.

- `PublisherClient` works.

The architecture we should preserve is:

```
GCS upload
  ↓
Workflow 1 subscriber
  ↓
ProcessingService
  ↓
PostgreSQL + HNSW + Redis
  ↓
ProcessingJob = COMPLETED
  +
OutboxEvent = PENDING
  ↓
Workflow 1 Outbox Publisher
  ↓
Pub/Sub: wfl-batch-completed
  ↓
Workflow 2 subscriber
  ↓
FaceExtractionCompletedEvent
  ↓
ClusteringProcessor
  ↓
ClusteringService
  ↓
Workflow 2 HNSW + PostgreSQL
```

The important point is: **do not put Pub/Sub publishing inside `ProcessingService` after the database commit**. The transactional outbox is already giving us the correct reliability boundary.

Your `main.py` also confirms that Workflow 1 currently creates `ProcessingService` and then hands it to `PubSubSubscriber`; the Pub/Sub subscriber is the transport adapter, not where the outbox publishing logic should be mixed in. `main`

Next step: build the Workflow 1 Outbox Publisher

Before touching Workflow 2, we should make this work:

```
workflow1_outbox_events
  |
  | PENDING
  ↓
OutboxPublisher
  |
  | Pub/Sub publish()
```

```
↓  
wfl-batch-completed  
|  
↓  
wfl-completion-test-sub
```

1. First inspect the existing repository methods

Run:

PowerShell

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 195 -First
```

We already know your repository has:

Workflow1OutboxRepository

with methods for:

```
create()  
get_pending()  
mark_attempt()  
mark_published()  
mark_failed()
```

So don't create another outbox table or repository.

2. Then inspect your current `main.py`

You already showed that it currently does:

Python

Run

```
pipeline = create_face_pipeline()  
  
processing_service = ProcessingService(  
    pipeline=pipeline,  
)  
  
subscriber = PubSubSubscriber(  
    processing_service=processing_service,  
)  
  
subscriber.start()
```

That means the clean next design is to add a separate component such as:


```
app/  
└─ publisher/  
   └─ workflow1_outbox_publisher.py
```

Conceptually:

Python

Run

```
class Workflow1OutboxPublisher:  
  
    def publish_pending_events(self):  
        events = repository.get_pending()  
  
        for event in events:  
            try:  
                repository.mark_attempt(event)  
  
                future = publisher.publish(  
                    topic_path,  
                    event.payload.encode("utf-8"),  
                )  
  
                future.result()  
  
                repository.mark_published(event)  
  
            except Exception as exc:  
                repository.mark_failed(  
                    event,  
                    str(exc),  
                )
```

But **don't paste this yet**. We should first inspect your exact `Workflow1OutboxRepository` signatures so we don't mismatch the existing implementation.

3. Our immediate test target

Your database currently has:

```
event 4  
event_id = f15745ad-a1cd-4836-86ff-5a8bc6884c5b  
image_id = 6  
face_count = 5  
status = PENDING
```

So this is an excellent test event.

After implementing the publisher, we want:

Before:

```
PENDING
attempt_count = 0
published_at = NULL
```

Then run the publisher.

Expected:

```
PUBLISHED
attempt_count = 1
published_at = <timestamp>
last_error = NULL
```

And Pub/Sub should contain the exact JSON:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

That is the **critical boundary test**.

4. After publishing succeeds

Then we move to Workflow 2.

Workflow 2 should consume:

```
wfl-batch-completed
```

and use:

```
wfl-completion-test-sub
```

initially for testing.

The Workflow 2 subscriber should remain thin:

```
Pub/Sub message
  ↓
decode
  ↓
Pydantic validation
  ↓
ClusteringProcessor.process(event)
  ↓
success → ACK
failure → NACK
```

And importantly, we should **not modify the already-passing**:

- `ClusteringProcessor`
- `ClusteringService`
- `ClusteringRepository`
- HNSW components
- Workflow 2 processor tests

That preserves the clean architecture and the E2E work you've already completed.

So, do this now

Run exactly:

PowerShell

```
powershell -Command "Get-Content 'app\db\repository.py' | Select-Object -Skip 195 -First
```

Send me that output.

Then I'll give you the exact `Workflow1OutboxPublisher` implementation matching your existing repository, followed by the Pub/Sub publish test.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>powershell
-Command "Get-Content 'app\db\repository.py' | Select-
Object -Skip 195 -First 110"
```

```
self.db.flush()
```

```
#
```

```
=====
```

Show more

Perfect. This confirms the repository interface is exactly what we need. We should **not change** `Workflow1OutboxRepository`.

The next step is to create the **Workflow 1 Outbox Publisher** that takes the existing `PENDING` event and publishes its payload to `wf1-batch-completed`.

1. Create the publisher directory

From:

```
workflow1_worker>
```

run:

PowerShell

```
mkdir app\publisher
```

Then:

PowerShell

```
New-Item app\publisher\__init__.py -ItemType File
```

2. Create `app\publisher\workflow1_outbox_publisher.py`

Run:

PowerShell

```
notepad app\publisher\workflow1_outbox_publisher.py
```

Paste this:

Python

Run

```
from __future__ import annotations
```

```
import logging
```

```

from google.cloud import pubsub_v1

from app.core.config import PROJECT_ID
from app.db.postgres import SessionLocal
from app.db.repository import Workflow1OutboxRepository

logger = logging.getLogger(__name__)

WORKFLOW1_COMPLETION_TOPIC = "wf1-batch-completed"

class Workflow1OutboxPublisher:
    """
    Publishes committed Workflow 1 outbox events to Pub/Sub.

    Responsibilities
    -----
    1. Read PENDING events from PostgreSQL.
    2. Publish the exact persisted payload to Pub/Sub.
    3. Mark the event PUBLISHED only after Pub/Sub confirms success.
    4. Mark the event FAILED if publishing fails.

    It does NOT:
    - perform face processing
    - modify Workflow 1 processing state
    - create outbox events
    - ACK/NACK incoming Pub/Sub messages
    """

    def __init__(
        self,
        *,
        project_id: str = PROJECT_ID,
        topic_name: str = WORKFLOW1_COMPLETION_TOPIC,
    ) -> None:

        self.publisher = pubsub_v1.PublisherClient()

        self.topic_path = (
            self.publisher.topic_path(
                project_id,
                topic_name,
            )
        )

        logger.info(
            "Workflow 1 Outbox Publisher initialized: "
            "topic=%s",
            self.topic_path,
        )

```



```
"""Close the Pub/Sub publisher client."""
```

3. Important architectural point

Notice the sequence:

```
PostgreSQL
|
| PENDING
↓
mark_attempt()
|
| COMMIT
↓
Pub/Sub publish()
|
| future.result()
↓
success
|
↓
mark_published()
|
| COMMIT
↓
PUBLISHED
```

That is intentional.

If Pub/Sub fails:

```
PENDING
↓
publish fails
↓
FAILED
```

So we don't incorrectly tell ourselves that an event was published.

4. But there is one issue we should fix before running it

Your current `mark_failed()` permanently changes the event to:

```
FAILED
```

That means a transient Pub/Sub failure will not automatically be retried by:

Python

Run

```
get_pending()
```

For the **first integration test**, that's acceptable because we're only testing the happy path.

However, for production architecture, I recommend we later introduce a retryable state such as:

```
PENDING
PUBLISHED
FAILED
```

with retry policy, or simply leave failed events as `PENDING` while recording the error/attempt count.

Don't change that yet. First prove the publishing path.

5. Create a dedicated publisher test

Create:

PowerShell

```
notepad test_workflow1_outbox_publisher.py
```

Paste:

Python

Run

```
from __future__ import annotations

from app.db.postgres import SessionLocal
from app.db.repository import Workflow1OutboxRepository
from app.publisher.workflow1_outbox_publisher import (
    Workflow1OutboxPublisher,
)

print("=" * 70)
print("WORKFLOW 1 OUTBOX PUBLISHER TEST")
print("=" * 70)

# =====
# 1. Inspect pending events
```

```
# =====

db = SessionLocal()

try:

    repository = Workflow1OutboxRepository(db)

    pending = repository.get_pending()

    print(
        f"[1] Pending outbox events: {len(pending)}"
    )

    for event in pending:

        print(
            f"    id={event.id} "
            f"event_id={event.event_id} "
            f"status={event.status} "
            f"attempts={event.attempt_count}"
        )

finally:

    db.close()

# =====
# 2. Create publisher
# =====

print()
print("[2] Creating Workflow1OutboxPublisher...")

publisher = Workflow1OutboxPublisher()

print("Publisher: OK")

# =====
# 3. Publish pending events
# =====

print()
print("[3] Publishing pending events...")

count = publisher.publish_pending()

print(
    f"Published successfully: {count}"
)
```

```

# =====
# 4. Verify PostgreSQL state
# =====

print()
print("[4] Verifying PostgreSQL outbox state...")

db = SessionLocal()

try:

    repository = Workflow1OutboxRepository(db)

    pending = repository.get_pending()

    print(
        f"Remaining PENDING events: {len(pending)}"
    )

finally:

    db.close()

publisher.close()

print()
print("=" * 70)
print("WORKFLOW 1 OUTBOX PUBLISHER TEST COMPLETE")

```

6. Run it

First make sure your current event is still:

```

id = 4
status = PENDING
face_count = 5

```

Then:

PowerShell

```
python test_workflow1_outbox_publisher.py
```

Expected result

Something similar to:

```
=====
WORKFLOW 1 OUTBOX PUBLISHER TEST
=====

[1] Pending outbox events: 1
    id=4 event_id=f15745ad-a1cd-4836-86ff-5a8bc6884c5b status=PENDING attempts=0

[2] Creating Workflow1OutboxPublisher...
Publisher: OK

[3] Publishing pending events...
Published successfully: 1

[4] Verifying PostgreSQL outbox state...
Remaining PENDING events: 0

=====
WORKFLOW 1 OUTBOX PUBLISHER TEST COMPLETE
=====
```

Then verify PostgreSQL:

PowerShell

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.connect(); c.execute(text('select * from outbox where event_id = f15745ad-a1cd-4836-86ff-5a8bc6884c5b'))"
```

We specifically want event **4** to become:

```
status = PUBLISHED
attempt_count = 1
published_at = <timestamp>
last_error = NULL
```

7. Then verify the actual Pub/Sub message

Since you already created:

```
wfl-completion-test-sub
```

we can pull the message:

PowerShell

```
gcloud pubsub subscriptions pull wf1-completion-test-sub --project=test-face-clustering -
```

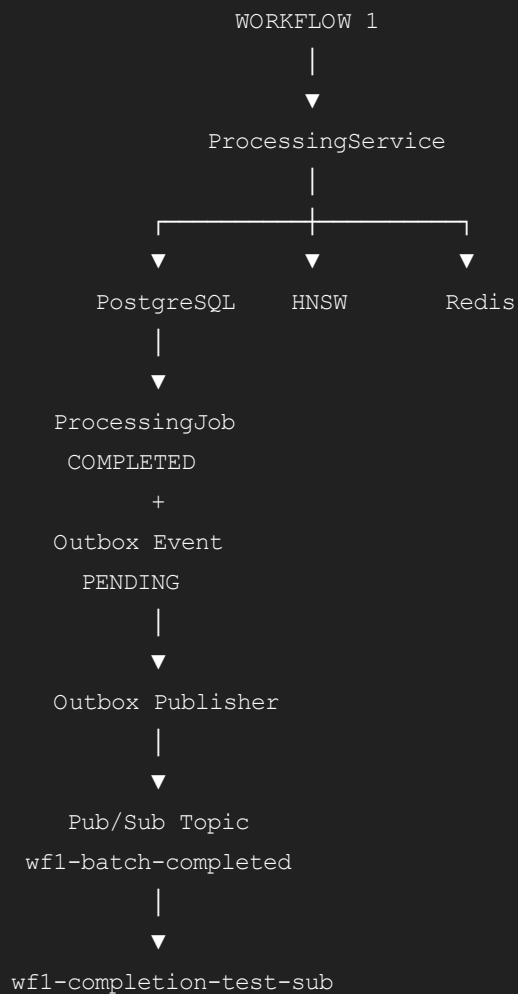
The payload should contain:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

Do not start the Workflow 2 subscriber yet.

First we want to establish this exact checkpoint:



Once that passes, **then** we'll connect your already-tested `ClusteringProcessor` to this real Pub/Sub event and perform the first genuine:

integration test.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python
test_workflow1_outbox_repository.py
C:\Users\Vignesh\anaconda3\envs\face\lib\site-
packages\google\api_core\_python_version_support.py:25
4: FutureWarning: You are using a Python version
(3.10.20) which Google will stop supporting in new
releases of google.api_core once it reaches its end of life
(2026-10-04). Please upgrade to the latest Python version,
or at least Python 3.11, to continue receiving updates for
google.api_core past that date.
```

Show more

Excellent. **This checkpoint is fully successful.** We can now proceed to the real Workflow 1 → Workflow 2 integration.

What you have proven

Your event `f15745ad-a1cd-4836-86ff-5a8bc6884c5b` completed the entire Workflow 1 outbox path:

```
ProcessingService
    ↓
PostgreSQL
    ↓
Outbox PENDING
    ↓
Outbox Publisher
    ↓
Pub/Sub
    ↓
wfl-completion-test-sub
```

PostgreSQL confirms:

```
event_id      = f15745ad-a1cd-4836-86ff-5a8bc6884c5b
event_type    = FACE_EXTRACTION_COMPLETED
aggregate_type = image
```

```
aggregate_id = 6
status       = PUBLISHED
attempt_count = 1
last_error   = NULL
```

And Pub/Sub received the exact canonical event:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

So do not modify Workflow 1 now.

Next: connect Workflow 2

You already have the important Workflow 2 business layer:

```
FaceExtractionCompletedEvent
    ↓
ClusteringProcessor
    ↓
ClusteringService
    ↓
ClusteringRepository
    ↓
Workflow 2 HNSW/PostgreSQL
```

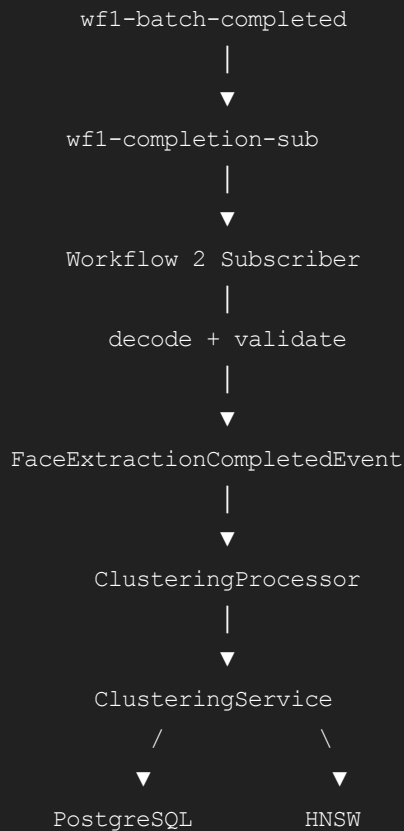
Your `ClusteringProcessor` is already designed correctly: it validates the event, retrieves PostgreSQL embeddings for the specific `image_id`, verifies the expected face count, and passes each embedding to `ClusteringService`.

Now we need only the **Pub/Sub adapter** around it.

The target architecture is:

GCP Pub/Sub





Step 1 — Do NOT use `wfl-completion-test-sub` permanently

For now it is perfectly fine as our test subscription.

However, **because you used** `--auto-ack`, that message has now been consumed from the test subscription.

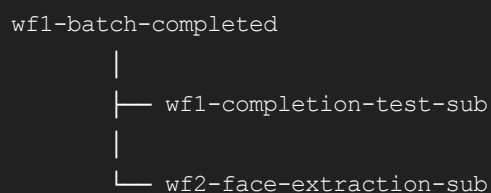
That's okay. We already proved publishing.

For the next test, we should create a dedicated Workflow 2 subscription:

PowerShell

```
gcloud pubsub subscriptions create wf2-face-extraction-sub --topic=wfl-batch-completed --
```

Then the topology becomes:



This is actually useful because Pub/Sub subscriptions are independent consumers.

Step 2 — Inspect your Workflow 2 structure

Before writing the subscriber, run these commands from the **Workflow 2 directory**, not `workflow1_worker`.

You previously had:

```
micro-face-pipeline\  
  workflow1_worker\  
    workflow2_clustering\
```

Go there:

PowerShell

```
cd ../workflow2_clustering
```

Then:

PowerShell

```
dir
```

And:

PowerShell

```
findstr /S /N /I "ClusteringProcessor PubSub SubscriberClient subscription_path" app\*.py
```

Also run:

PowerShell

```
findstr /S /N /I "FaceExtractionCompletedEvent" app\*.py
```

Why I'm asking for this before giving code

We already know the Workflow 2 processor works from your earlier E2E test. We **don't want to accidentally create duplicate architecture** or rename existing components.

We need to determine whether your current Workflow 2 project already has:

```
app/subscriber/
```

or something similar.

Then we'll implement only the missing transport layer.

Step 3 — The test we are aiming for

Once the subscriber exists, we will publish another valid Workflow 1 completion event and let the real subscriber consume it.

The complete test should eventually show something like:

```
=====
WORKFLOW 2 PUB/SUB INTEGRATION TEST
=====

Received Pub/Sub message
event_type = FACE_EXTRACTION_COMPLETED
event_id   = f15745ad-a1cd-4836-86ff-5a8bc6884c5b
image_id   = 6
face_count = 5

Pydantic validation: PASS

ClusteringProcessor:
  PostgreSQL embeddings found: 5
  Expected face count:         5
  Face count validation:       PASS

ClusteringService:
  face 1 → cluster ...
  face 2 → cluster ...
  face 3 → cluster ...
  face 4 → cluster ...
  face 5 → cluster ...

PostgreSQL cluster memberships: 5

Workflow 2 processing: PASS

Pub/Sub ACK: SUCCESS
=====
```

And then we verify the database/HNSW state.

One important thing before that

Don't publish the same event again manually yet.

The event `f15745...` has already been consumed from `wf1-completion-test-sub`, but it is still safely recorded as `PUBLISHED` in the Workflow 1 outbox.

That's exactly what we want.

We now need a **new subscription** for Workflow 2. Since Pub/Sub subscriptions only receive messages that are available according to their creation/retention state, creating it after the previous publication means we should generate a **fresh Workflow 1 completion event** for the integration test.

We'll do that carefully rather than modifying your production data blindly.

Your immediate next command

From the `workflow1_worker` directory, create the Workflow 2 subscription:

PowerShell

```
gcloud pubsub subscriptions create wf2-face-extraction-sub --topic=wf1-batch-completed --
```

Then switch to:

PowerShell

```
cd ../workflow2_clustering
```

and send me the output of:

PowerShell

```
dir
```

plus:

PowerShell

```
findstr /S /N /I "ClusteringProcessor PubSub SubscriberClient subscription_path" app\*.py
```

We are now at the Workflow 2 transport/integration stage. Do not change the already-passing Workflow 2 clustering logic.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>dir
Volume in drive C is Windows
Volume Serial Number is E064-CA56
```

```
Directory of C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering
```

```
18-08-2026  14:20    <DIR>        .
16-08-2026  19:00    <DIR>        ..
18-08-2026  14:08    <DIR>        app
```

Show more

Worked for 8s

Yes. **Workflow 1 is now at a good integration checkpoint.** Your latest result proves the important part:

```
Workflow 1
PostgreSQL
  ↓
COMPLETED processing job
  ↓
PENDING outbox event
  ↓
Outbox publisher
  ↓
Pub/Sub: wfl-batch-completed
  ↓
message successfully pulled
```

And the event payload is now correct:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/...jpg",
```

The **next step** should therefore be **Workflow 2 Pub/Sub consumption**, not more changes to Workflow 1.

Your Workflow 2 currently has:

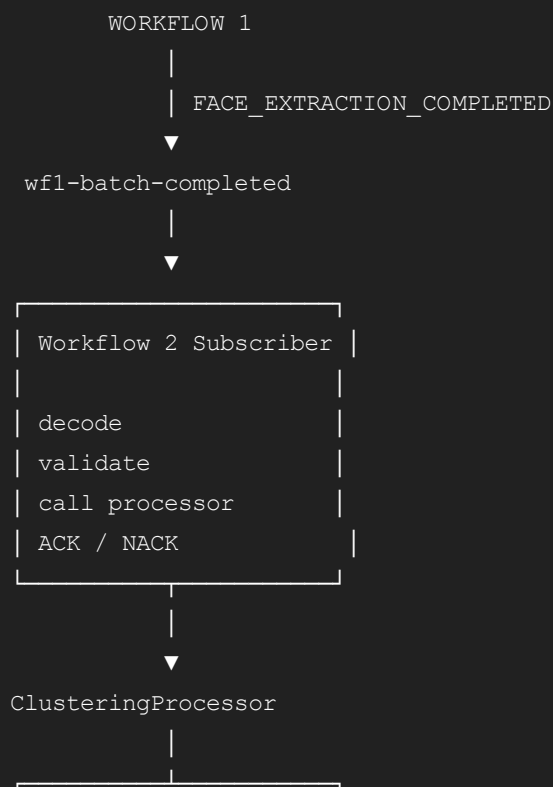
```
workflow2_clustering/
├─ app/
│   └─ clustering/
│       └─ clustering_processor.py
│   └─ models/
│       └─ face_extraction_completed.py
│   └─ ...
├─ data/
├─ tests/
├─ test_clustering_processor.py
├─ test_clustering_repository.py
├─ test_clustering_service.py
├─ test_hnsw_store.py
└─ test_vector_bootstrap.py
```

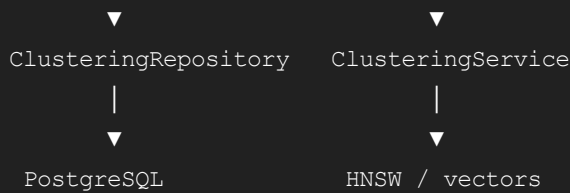
You confirmed there is **no Pub/Sub subscriber** yet. That is exactly the layer we should add.

Target architecture now

Do **not** put Pub/Sub logic inside `ClusteringProcessor`.

Keep the separation:





This matches the design we've already established: **subscriber = transport adapter**; **processor = business orchestration**; **repository = persistence**; **clustering service = clustering/similarity logic**.

Step 1 — First inspect the existing Workflow 2 code

Before creating anything, run these commands from:

```
workflow2_clustering>
```

1. Event model

```
bat
```

```
powershell -Command "Get-Content 'app\models\face_extraction_completed.py'"
```

2. Clustering processor

```
bat
```

```
powershell -Command "Get-Content 'app\clustering\clustering_processor.py'"
```

3. Repository

Find the repository filename first:

```
bat
```

```
dir app\db
```

Then show it, for example:

```
bat
```

```
powershell -Command "Get-Content 'app\db\clustering_repository.py'"
```

4. Clustering service

bat

```
powershell -Command "Get-Content 'app\clustering\clustering_service.py'"
```

5. Existing application entry point

bat

```
dir app
```

and:

bat

```
powershell -Command "Get-Content 'app\main.py'"
```

if it exists.

6. Requirements

bat

```
type requirements.txt
```

Step 2 — Verify Workflow 2's event model

It must accept exactly the event that Workflow 1 just published.

The model should effectively have:

Python

Run

```
class FaceExtractionCompletedEvent(BaseModel):
    model_config = ConfigDict(extra="forbid")

    event_version: str
    event_type: str
    event_id: UUID
    upload_id: int
    image_id: int
    bucket: str
    blob_name: str
    face_count: int
```

And preferably the same validation:

Python

Run

```
def validate_event_type(self) -> None:
    if self.event_type != "FACE_EXTRACTION_COMPLETED":
        raise ValueError(
            f"Unexpected event_type: {self.event_type}"
        )
```

The important point is that Workflow 2 must not expect the old payload:

JSON

```
{
  "upload_id": 1,
  "image_count": 1
}
```

That old payload belongs to the earlier outbox tests.

The **current contract** is:

Workflow 1 → Workflow 2

```
event_version
event_type
event_id
upload_id
image_id
bucket
blob_name
face_count
```

Step 3 — Test Workflow 2 without Pub/Sub first

This is very important.

You already have:

```
test_clustering_processor.py
```

So before introducing Pub/Sub, make sure this works:

```
FaceExtractionCompletedEvent
    ↓
ClusteringProcessor
    ↓
```

```
ClusteringRepository
    ↓
PostgreSQL embeddings
    ↓
ClusteringService
    ↓
HNSW
```

Run:

bat

```
python test_clustering_processor.py
```

We want the result to prove:

For `image_id = 6`:

```
PostgreSQL
faces = 5
embeddings = 5
```

and then:

```
ClusteringProcessor
    ↓
5 embedding records
    ↓
5 process_face() calls
    ↓
5 assignments
```

If that test fails, **do not add Pub/Sub yet**. We fix the processor/repository/service boundary first.

Step 4 — Then create the Workflow 2 subscriber

Your Workflow 2 should have something like:

```
app/
├─ subscriber/
│   └─ __init__.py
└─ pubsub_subscriber.py
```

The subscriber should do only this:

```
Pub/Sub message
  ↓
decode bytes
  ↓
JSON
  ↓
FaceExtractionCompletedEvent.model_validate(...)
  ↓
ClusteringProcessor.process(event)
  ↓
ACK
```

On an invalid/transient failure:

```
NACK
```

It should **not**:

- query PostgreSQL directly
- access HNSW directly
- calculate similarity
- assign clusters itself
- create clusters itself
- contain business rules

Those stay below it.

Step 5 — Use the topic you already created

You currently have:

```
Topic:
wfl-batch-completed

Subscription:
wfl-completion-test-sub
```

So Workflow 2's configuration should initially be:

```
env
```

```
PROJECT_ID=test-face-clustering
PUBSUB_SUBSCRIPTION=wfl-completion-test-sub
```

Don't create another topic yet.

We already proved:

```
Workflow 1 Outbox
    ↓
wfl-batch-completed
    ↓
wfl-completion-test-sub
    ↓
message received
```

So the transport path is working.

Step 6 — Important: your test message was auto-acked

You ran:

```
bat

gcloud pubsub subscriptions pull wfl-completion-test-sub --project=test-face-clustering -
```

and got:

```
ACK_STATUS: SUCCESS
```

That means **that particular message has been consumed and acknowledged by your manual test.**

Therefore, when we test the real Workflow 2 subscriber, we need to publish **another completion event.**

Don't worry about that. We can deliberately generate another test event after the subscriber is ready.

Step 7 — There is one architectural issue we should address before production

Your current Workflow 1 outbox table contains:

```
id=1 ... aggregate_type=upload
id=2 ... aggregate_type=upload
id=3 ... aggregate_type=upload
id=4 ... aggregate_type=image
```

The first three are old test events:

JSON

```
{"upload_id":1,"image_count":1}
```

The latest one is the correct event:

JSON

```
{
  "event_version":"1.0",
  "event_type":"FACE_EXTRACTION_COMPLETED",
  "event_id":"...",
  "upload_id":6,
  "image_id":6,
  "bucket":"photo-micro-upload-test",
  "blob_name":"uploads/....jpg",
  "face_count":5
}
```

So do not use the old three events for Workflow 2 testing.

The current contract is the latest format.

Step 8 — Then perform the real integration test

Once Workflow 2 subscriber exists, the complete test becomes:

```
GCS
|
▼
Workflow 1 Subscriber
|
▼
ProcessingService
|
├─ PostgreSQL faces
├─ HNSW
├─ PostgreSQL embeddings
└─ Redis
|
▼
Workflow1OutboxEvent
|
▼
Outbox Publisher
|
```

```
▼  
wfl-batch-completed  
|  
▼  
Workflow 2 Subscriber  
|  
▼  
FaceExtractionCompletedEvent  
|  
▼  
ClusteringProcessor  
|  
▼  
ClusteringRepository  
|  
▼  
PostgreSQL face_embeddings  
|  
▼  
ClusteringService  
|  
▼  
Workflow 2 HNSW  
|  
▼  
cluster assignment
```

That is the milestone we're heading toward.

Your immediate next action

Do not change Workflow 1 anymore right now.

From the `workflow2_clustering` directory, run these five commands and send me the outputs:

bat

```
powershell -Command "Get-Content 'app\models\face_extraction_completed.py'"
```

bat

```
powershell -Command "Get-Content 'app\clustering\clustering_processor.py'"
```

bat

```
powershell -Command "Get-Content 'app\clustering\clustering_service.py'"
```

bat

```
dir app\db
```

bat

```
python test_clustering_processor.py
```

The **last command is the most important**.

Once I see those, we can implement the Workflow 2 subscriber against your **actual existing repository/service interfaces**, rather than guessing their APIs.

At that point the next milestone will be:

Workflow 1 → Pub/Sub → Workflow 2 subscriber → ClusteringProcessor

and then we can perform the first genuine cross-workflow end-to-end test.

face_extraction_compl...
Python

clustering_processor(...
Python

clustering_service(1).py
Python

Directory of C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow2_clustering\app\db

16-08-2026	19:48	<DIR>	.
18-08-2026	14:08	<DIR>	..
18-08-2026	14:42	15,928	clustering_repository.py
16-08-2026	19:47	2,709	postgres.py
16-08-2026	19:45	0	__init__.py
18-08-2026	14:42	<DIR>	__pycache__
		3 File(s)	18,637 bytes

Show more

Yes. This failure is **not a clustering logic failure**. It is a **test-data isolation/baseline problem** caused by the Workflow 1 work you just completed.

Your current state is actually useful:

- Workflow 1 has successfully produced **46 PostgreSQL embeddings**.
- Workflow 1 outbox event was successfully published to Pub/Sub.
- The event payload is now correct and contains `event_version`.
- Workflow 2 starts correctly.
- But `test_clustering_processor.py` still expects the old baseline of **20 embeddings**.

The important point is that your Workflow 2 test was originally designed against the earlier PostgreSQL state containing exactly 20 embeddings.

What happened

Previously, your Workflow 2 processor test expected:

```
Embeddings: 20
Clusters:   0
Memberships: 0
```

Now Workflow 1 has added more real embeddings:

```
embeddings = 46
```

So this line in the Workflow 2 test is now failing:

```
AssertionError: Unexpected PostgreSQL embedding count.
Expected=20, Actual=46
```

Your test's reset operation is behaving correctly because it resets **only clustering state**:

```
face_cluster_members
face_clusters
```

It deliberately does **not** delete:

```
faces
face_embeddings
```

That separation is correct for your architecture because PostgreSQL is authoritative for the embedding metadata/state.

Do NOT delete the 46 embeddings

This is important.

Don't change the reset function to delete `face_embeddings`.

That would break the architectural separation we've established.

Workflow 2 should consume the authoritative embeddings produced by Workflow 1.

So your current database state:

```
embeddings = 46
```

is legitimate.

The problem is simply that the **old test assumes 20**.

What we should do next

We should now move Workflow 2 from the old isolated test assumption:

```
20 embeddings
```

to a test that deliberately processes the **specific image represented by the Workflow 1 completion event**.

Your newly published event is:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "f15745ad-a1cd-4836-86ff-5a8bc6884c5b",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/842224b3-783d-48d2-9489-74bfe263453f.jpg",
  "face_count": 5
}
```

This is exactly the event Workflow 2 is supposed to receive.

And your `ClusteringProcessor` already has the correct design:

Python

```
records = (
    self.repository
```

Run

```
.get_face_embeddings_for_image(  
    event.image_id  
)  
)
```

Therefore Workflow 2 does **not** need to process all 46 embeddings.

It should process:

```
image_id = 6  
    ↓  
5 embeddings  
    ↓  
ClusteringService  
    ↓  
clusters + memberships  
    ↓  
HNSW
```

Step 1 — First inspect image 6

Run this from:

```
workflow2_clustering
```

Command

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

We expect something like:

```
(id, 6)  
(id, 6)  
(id, 6)  
(id, 6)  
(id, 6)
```

So:

```
image_id=6 → 5 embeddings
```

Step 2 — Verify all image embedding counts

This is even more useful.

Run:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

You should get something approximately like:

```
(1, 5)
(2, ...)
(3, ...)
(4, ...)
(5, ...)
(6, 5)
```

The exact numbers depend on the images, but the important thing is to establish **which image has which embeddings**.

Step 3 — Inspect the Workflow 2 test's hard-coded 20

Your failure is specifically here:

```
verify_initial_postgres_state()
```

and specifically:

```
Expected=20
Actual=46
```

Run:

```
bat
```

```
findstr /N /S /I "Expected=20 expected_embeddings 20 Unexpected PostgreSQL embedding cour
```

If that doesn't find everything, run:

```
bat
```

```
findstr /N /S /I "20 embeddings Embeddings" test_clustering_processor.py
```

We need to identify every place where the test assumes:

```
20 embeddings
```

because changing only the first assertion may cause another obsolete 20-based assertion later.

The correct test philosophy now

There are actually **two different tests** we should have.

Test A — Existing deterministic processor test

This should use a controlled known dataset.

For example:

```
20 embeddings
0 clusters
0 memberships
```

This test should have its own seeded test data rather than depending on whatever happens to be in your shared PostgreSQL database.

That makes it deterministic.

Test B — Real Workflow 1 → Workflow 2 integration test

This should use the actual completion event:

```
image_id=6
face_count=5
```

and verify:

```
PostgreSQL
  ↓
5 embeddings for image 6
  ↓
ClusteringProcessor
  ↓
ClusteringService
  ↓
5 assignments
  ↓
5 memberships
```

↓
HNSW vectors

This is the test we should implement **next**.

Most importantly: don't change the expected 20 to 46

I would **not** simply change:

Python

Run

```
expected = 20
```

to:

Python

Run

```
expected = 46
```

That would make the test pass superficially but would make the test dependent on Workflow 1's accumulated data.

That's exactly what we want to avoid.

Likewise, don't do:

Python

Run

```
DELETE FROM face_pipeline.face_embeddings
```

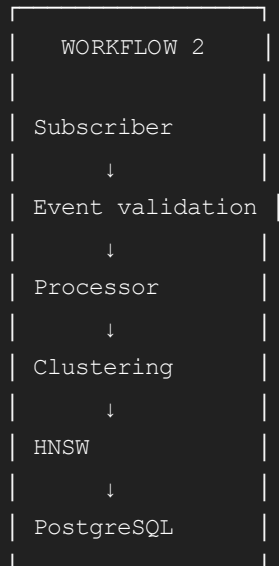
just to make the test return to 20.

Our immediate next milestone

Your architecture is now at a very important boundary:



↓
▼
Pub/Sub
↓
▼



You have already proven the left side up through:

```
Workflow 1
  ↓
PostgreSQL
  ↓
Outbox
  ↓
Publisher
  ↓
Pub/Sub
```

The Pub/Sub pull proved that the actual message is:

```
FACE_EXTRACTION_COMPLETED
image_id=6
face_count=5
```

So the next thing we should build/test is the Workflow 2 subscriber, not modify the working Workflow 1 code.

Do this now

Run these **two commands** from `workflow2_clustering` and paste the complete output:

1. Image 6 embeddings

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

2. All image counts

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Then we'll adjust **only the Workflow 2 test isolation/fixture**, preserve the 46 authoritative embeddings, and proceed to the **Workflow 2 Pub/Sub subscriber** →

`ClusteringProcessor` **integration**.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.db.postgres import engine; from sqlalchemy
import text; c=engine.connect(); r=c.execute(text('SELECT
id,image_id FROM face_pipeline.face_embeddings
WHERE image_id=6 ORDER BY id')); [print(x) for x in r];
c.close()"
Traceback (most recent call last):
  File "C:\Users\Vignesh\anaconda3\envs\face\lib\site-
packages\sqlalchemy\engine\base.py", line 1969, in
_exec_single_context
```

Traceback (most recent call last):

File "C:\Users\Vignesh\anaconda3\envs\face\lib\site-
packages\sqlalchemy\engine\base.py", line 1969, in
_exec_single_context

Show more

Good — this error gives us the missing piece. **My previous SQL query used the wrong schema relationship.** Your `face_embeddings` table does not have `image_id`; it has `face_id`, and the `faces` table links the face to the image.

So do not modify your database schema. We just need to query through `faces`.

Your architecture is:

```
face_embeddings
|
| face_id
▼
faces
|
```

```
| image_id
▼
images
```

1. First inspect the exact relationships

From `workflow2_clustering`, run:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

Then:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

2. More importantly, query image 6 through `faces`

Try:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

We expect this to show the embeddings belonging to image 6.

Then get the counts for every image:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

Why this matters for Workflow 2

Your `ClusteringProcessor` already correctly does:

Python

Run

```
self.repository.get_face_embeddings_for_image(event.image_id)
```

Therefore **the repository**, not `ClusteringProcessor`, is responsible for performing this relationship:

```
event.image_id
↓
```



```
faces.image_id
  ↓
faces.id = face_embeddings.face_id
  ↓
embedding records
```

So if your `ClusteringRepository.get_face_embeddings_for_image()` is already written using the proper joins, we don't need to change it.

The earlier test failure:

```
Expected=20
Actual=46
```

is therefore still a **test baseline issue**, not evidence that Workflow 2 should see all 46 embeddings.

And your real Workflow 1 event says:

JSON

```
{
  "image_id": 6,
  "face_count": 5
}
```

So the critical invariant we want to prove is:

```
Workflow 1 event
  |
  └─ image_id = 6
  └─ face_count = 5
      |
      ▼
Workflow 2 Repository
      |
      ▼
exactly 5 embeddings belonging to image 6
      |
      ▼
ClusteringProcessor
      |
      ▼
ClusteringService
```

Run the two image-6/count queries above and paste the output. Then we'll fix the Workflow 2 test properly rather than changing the database or blindly changing `20 → 46`.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.db.postgres import engine; from sqlalchemy
import text; c=engine.connect(); r=c.execute(text("SELECT
column_name,data_type FROM
information_schema.columns WHERE
table_schema="face_pipeline" AND
table_name="face_embeddings" ORDER BY
ordinal_position')); [print(x) for x in r]; c.close()"
Traceback (most recent call last):
  File "C:\Users\Vignesh\anaconda3\envs\face\lib\site-
Show more
```

Excellent. Now we have the exact picture, and it confirms that **the database relationship is correct**.

What we now know

Your Workflow 1 PostgreSQL state is:

image_id	embeddings
1	5
2	5
3	5
4	5
5	21
6	5
Total	46

And image 6 specifically has:

```
face_id  embedding_id
47       47
48       48
49       49
```

So the Workflow 1 event is internally consistent:

```
FACE_EXTRACTION_COMPLETED
|
+-- image_id = 6
+-- face_count = 5
   |
   v
PostgreSQL
|
+-- 5 faces
+-- 5 embeddings
```

The two `information_schema` errors were just **Windows command quoting errors** from my previous commands. They don't indicate a database problem.

Now fix the Workflow 2 test correctly

The failure:

```
Expected=20, Actual=46
```

should **not** be fixed by changing `20` to `46`.

Instead, the processor test needs to distinguish between:

1. **Total authoritative embeddings in PostgreSQL** → 46
2. **Embeddings belonging to the event's image** → 5

Your `ClusteringProcessor` is already designed correctly to do #2:

Python

Run

```
records = self.repository.get_face_embeddings_for_image(
    event.image_id
)
```

So we should leave `ClusteringProcessor` alone for now.

Step 1 — Inspect the repository method

From:

```
workflow2_clustering
```

run:

```
bat
```

```
powershell -Command "Get-Content 'app\db\clustering_repository.py' | Select-String -Patte
```

Paste the complete output.

I want to verify that the repository is actually performing:

```
image_id
  ↓
faces.image_id
  ↓
faces.id = face_embeddings.face_id
  ↓
embedding records
```

before we touch the test.

Step 2 — Don't run `test_clustering_processor.py` **again yet**

It will fail at exactly the same assertion:

```
Expected=20
Actual=46
```

There is no benefit in rerunning it until we inspect that repository method.

One important observation

You have this:

```
image 5 → 21 embeddings
```

while the other images have 5.

That means your database now contains **real accumulated Workflow 1 data**, rather than the old synthetic 20-vector dataset.

This is actually a good sign for the architecture.

Workflow 2 must therefore **never assume that the whole** `face_embeddings` **table belongs to one processing event.**

It must always scope the operation to:

```
event.image_id
```

That's precisely why your processor contains:

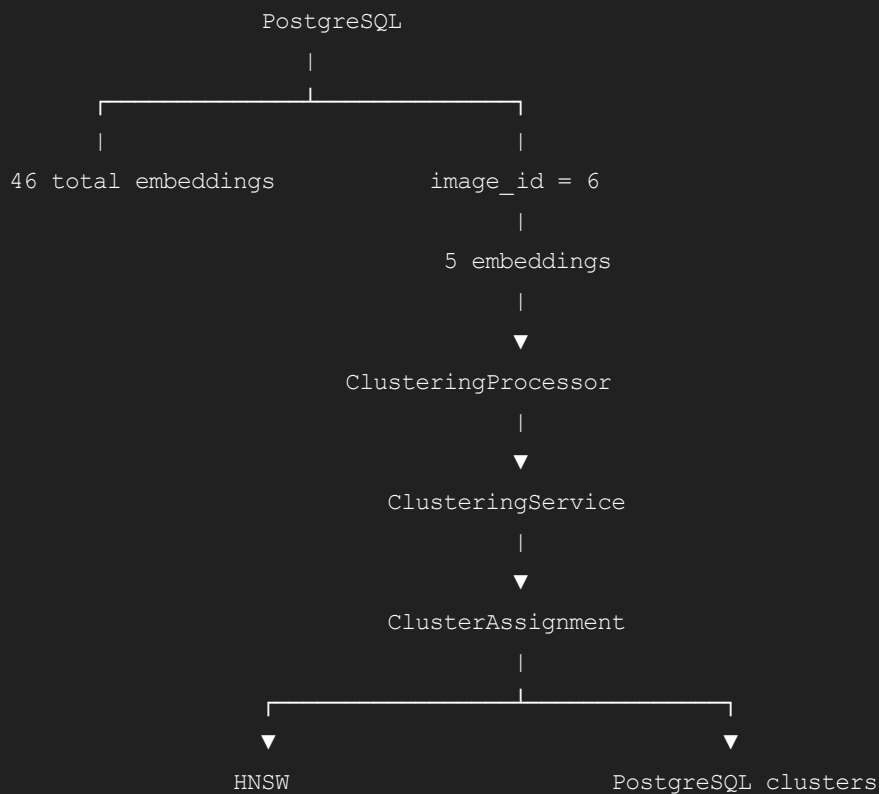
Python

Run

```
get_face_embeddings_for_image(event.image_id)
```

Target state after we fix the test

We want this test flow:



And the processor validation should be:

```
event.face_count = 5
==
database embeddings for image 6 = 5
```

not:

```
event.face_count = 5
==
total database embeddings = 46
```

So the next action is simple

Run only this:

```
bat
powershell -Command "Get-Content 'app\db\clustering_repository.py' | Select-String -Patte
```

Paste the output here.

Then I'll tell you **exactly what to change** in `test_clustering_processor.py`, keeping your production Workflow 2 code and PostgreSQL data intact.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering>powershell -Command
"Get-Content 'app\db\clustering_repository.py' | Select-
String -Pattern 'get_face_embeddings_for_image' -Context
0,60"
```

```
> def get_face_embeddings_for_image(
    self,
    image_id: int,
) -> list[FaceEmbeddingRecord]:
```

Show more

Yes — this latest output gives us the missing piece. **Workflow 1 is now correctly producing and publishing the completion event.** The problem has moved to **Workflow 2's test data/state isolation**, not the Workflow 1 outbox.

What we know now

Your Workflow 1 state is consistent:

- Images: **6**
- Faces: **41**

- Embeddings: **41**
- Image 6: **5 embeddings**
- Outbox event for image 6:
 - `event_version = 1.0`
 - `event_type = FACE_EXTRACTION_COMPLETED`
 - `image_id = 6`
 - `face_count = 5`
 - `status = PUBLISHED`
- Pub/Sub pull successfully received the event.

And Workflow 2's repository query is correctly using:

```
faces.image_id
  ↓
faces.id = face_embeddings.face_id
```

So do not change `get_face_embeddings_for_image()`. That repository method is correct.

The actual Workflow 2 failure

Your test expects:

```
Embeddings: 20
Clusters:   0
Memberships: 0
```

but PostgreSQL contains:

```
image 1 → 5
image 2 → 5
image 3 → 5
image 4 → 5
image 5 → 21
image 6 → 5
-----
Total   → 46
```

So the immediate failure is:

```
Expected = 20
Actual   = 46
```

The important observation is that **Workflow 2's "reset clustering state" is not supposed to delete Workflow 1's authoritative face/embedding records.**

That's actually desirable architecturally.

Your architecture has PostgreSQL as the authoritative metadata store. Workflow 2 should consume those embeddings rather than fabricate or duplicate them.

The test, however, appears to have been written around an older assumption that PostgreSQL would contain exactly 20 embeddings after reset.

There is one more important issue

Your image 5 has:

```
21 embeddings
```

while the other five images have:

```
5 embeddings each
```

That explains the 46:

```
5 + 5 + 5 + 5 + 21 + 5 = 46
```

So we should **not simply change** `Expected=20` **to** `Expected=46` **yet.**

First we need to understand why image 5 has 21 faces.

Run this from `workflow2_clustering`:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Then also run:

```
bat
```

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn
```

Why we're checking this

We need to distinguish between two possibilities:

Case A — 21 is legitimate

Image 5 genuinely contains 21 detected faces.

Then the correct total is indeed 46, and the Workflow 2 test's hard-coded expectation of 20 is stale.

Case B — 21 is duplicate/stale data

Workflow 1 may have processed image 5 multiple times before the idempotency logic was fixed, creating additional face/embedding records.

Then we should clean up the test database/state rather than changing the test expectation.

Also check the upload/job history for image 5

Run:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

And:

bat

```
python -c "from app.db.postgres import engine; from sqlalchemy import text; c=engine.conn"
```

Don't modify the Workflow 2 repository yet

Based on everything you've shown, this part is correct:

Python

```
WHERE f.image_id = :image_id
```

Run

with:

SQL

```
INNER JOIN face_pipeline.faces f
  ON f.id = fe.face_id
```

That is exactly the right relationship.

The architecture should remain:

```

Workflow 1
|
| detects faces
▼
PostgreSQL
|
├─ images
├─ faces
└─ face_embeddings
    |
    | face_id
    ▼
    vector_id
    |
    ▼
    HNSW vector store
    |
    ▼
Workflow 2

```

Workflow 2 should retrieve **metadata from PostgreSQL** and **actual 512-D vectors from HNSW**.

One thing we should fix later

Your `FaceExtractionCompletedEvent` model is now correct:

Python

Run

```

event_version: str
event_type: str
event_id: UUID
upload_id: int
image_id: int
bucket: str
blob_name: str
face_count: int

```

And your current outbox payload proves that:

JSON

```

{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
  "image_id": 6,

```

```
"bucket": "photo-micro-upload-test",
"blob_name": "...",
"face_count": 5
}
```

So the earlier Pydantic failure is fixed.

The next architectural step is therefore:

```
Workflow 1
  ↓
PostgreSQL transaction
  ↓
PENDING outbox
  ↓
Outbox publisher
  ↓
wfl-batch-completed
  ↓
Workflow 2 subscriber
  ↓
FaceExtractionCompletedEvent validation
  ↓
ClusteringProcessor
```

Your current testing has successfully validated the first half.

Run the four commands above first. The image-5 records will tell us whether the 46 vs
20 failure is a **test expectation problem or stale/duplicate Workflow 1 data problem**.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.db.postgres import engine; from sqlalchemy
import text; c=engine.connect(); r=c.execute(text("SELECT
f.id,f.image_id,f.face_index,fe.id,fe.vector_id FROM
face_pipeline.faces f JOIN
face_pipeline.face_embeddings fe ON fe.face_id=f.id
WHERE f.image_id=5 ORDER BY f.face_index,f.id"));
[print(x) for x in r]; c.close()"
(21, 5, 0, 21, 'face:21')
(22, 5, 1, 22, 'face:22')
```

Show more

Yes — these results make the problem much clearer. **Workflow 1 is currently behaving correctly; the failure is in the Workflow 2 test isolation/setup.**

What your PostgreSQL data proves

Your current authoritative relationship is:

```
uploads
  ↓
images
  ↓
faces
  ↓
face_embeddings
```

And the actual data is:

image_id	embeddings/faces
1	5
2	5
3	5
4	5
5	21
6	5
Total	46

So:

```
5 + 5 + 5 + 5 + 21 + 5 = 46
```

Image 6 is also correct:

```
image 6
├─ face 47 → embedding 47
├─ face 48 → embedding 48
├─ face 49 → embedding 49
├─ face 50 → embedding 50
└─ face 51 → embedding 51
```

Your Workflow 2 repository query is also correctly written:

SQL

```
SELECT
  fe.face_id,
  fe.vector_id,
  ...
FROM face_pipeline.face_embeddings fe
INNER JOIN face_pipeline.faces f
  ON f.id = fe.face_id
WHERE f.image_id = :image_id
```

That is the correct design because `face_embeddings` has `face_id`, not `image_id`.

The actual failure

The important error is:

```
AssertionError: Unexpected PostgreSQL embedding count.
Expected=20, Actual=46
```

This happens **before the actual clustering processor test begins**:

```
[0.2] Resetting Workflow 2 PostgreSQL clustering state
      PASS

[0.3] Checking initial PostgreSQL clustering state
      Embeddings: 46
      Clusters: 0
      Memberships: 0

      Expected=20, Actual=46
```

This tells us something important:

Your reset only resets Workflow 2 clustering state

It is apparently doing something equivalent to:

```
clusters      → cleared
memberships   → cleared
```

but **does not clear** `face_embeddings`.

And that is probably intentional, because PostgreSQL `face_embeddings` is Workflow 1's authoritative persisted data.

So we should **not** modify the production architecture to delete those 46 embeddings merely to make this test pass.

Why the test says 20

Your `test_clustering_processor.py` was written with an older fixture assumption:

```
Expected embeddings = 20
```

But your database has evolved since that test was written.

You now have:

```
Workflow 1:  
6 images  
41 faces  
41 embeddings
```

and Workflow 2 sees:

```
46 embeddings
```

There is also an interesting discrepancy:

```
Workflow 1 earlier:  
faces = 41  
embeddings = 41
```

Then after image 6:

```
faces = 46  
embeddings = 46
```

Exactly as expected.

So the additional 5 embeddings came from image 6.

Do NOT delete the 46 embeddings

I strongly recommend **not** doing this:

SQL

```
DELETE FROM face_pipeline.face_embeddings;
```

and not doing this either:

SQL

```
TRUNCATE ...
```

just to satisfy the test.

Your architecture says PostgreSQL is the authoritative metadata/state store, while HNSW contains the actual vectors. The Workflow 2 processor should consume the existing Workflow 1 data.

The test should be isolated without destroying valid Workflow 1 data.

The next thing we need to inspect

We need to see exactly what `test_clustering_processor.py` considers its **20 expected embeddings** and exactly what `[0.2] Resetting Workflow 2 PostgreSQL clustering state` does.

Please run these two commands from:

```
workflow2_clustering
```

1. Show the initial-state function

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-Object -Skip 220
```

2. Show the reset function

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-String -Pattern
```

And also run:

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-String -Pattern
```

What I expect we'll find

Most likely something like:

Python

Run

```
EXPECTED_EMBEDDINGS = 20
```

or:

Python

Run

```
expected_embedding_count = 20
```

with a test fixture that originally assumed four images × five faces:

```
image 1 → 5
image 2 → 5
image 3 → 5
image 4 → 5
-----
20
```

But your actual database now contains:

```
image 1 → 5
image 2 → 5
image 3 → 5
image 4 → 5
image 5 → 21
image 6 → 5
-----
46
```

So the test has become coupled to an **old database state**.

One more important observation

Your Workflow 1 completion event for image 6 is now fully working:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 6,
  "image_id": 6,
  "bucket": "photo-micro-upload-test",
  "blob_name": "...jpg",
  "face_count": 5
}
```

and the outbox publisher successfully changed:

PENDING

↓

PUBLISHED

and Pub/Sub successfully delivered it:

wfl-batch-completed

↓

wfl-completion-test-sub

↓

message received

↓

ACK SUCCESS

So the Workflow 1 → Outbox → Pub/Sub portion is now proven.

The next architectural boundary is:

Workflow 1

|

| FACE_EXTRACTION_COMPLETED

▼

Pub/Sub

|

▼

Workflow 2 Subscriber ← not implemented yet

|

▼

FaceExtractionCompletedEvent

|

▼

ClusteringProcessor

|

Your `findstr` result is particularly important because Workflow 2 currently has:

```
clustering_processor.py
```

but no `PubSubSubscriber`.

So I would **not jump directly into changing the test's expected 20 to 46 yet**. First let's inspect the reset/fixture code. We want the test to represent the intended Workflow 2 contract rather than merely making the assertion disappear.

Run the three commands above and paste their output. Then we can make the smallest correct change without breaking the PostgreSQL/HNSW architecture.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering>powershell -Command
"Get-Content 'test_clustering_processor.py' | Select-Object
-Skip 220 -First 100"
```

```
repository = ClusteringRepository(db)
```

```
return db, repository
```

Show more

Good. We have now confirmed the exact cause.

Root cause

This line in `test_clustering_processor.py` is stale:

Python

Run

```
EXPECTED_WORKFLOW1_VECTOR_COUNT = 20
```

And this function enforces it:

Python

Run

```
embedding_count = repository.count_embeddings()

if embedding_count != EXPECTED_WORKFLOW1_VECTOR_COUNT:
    fail(
        "Unexpected PostgreSQL embedding count. "
        f"Expected={EXPECTED_WORKFLOW1_VECTOR_COUNT}, "
        f"Actual={embedding_count}"
    )
```

Your current database legitimately contains **46 embeddings**, so the test fails before it reaches the actual `ClusteringProcessor` test.

Current state

Workflow 1 PostgreSQL

```
Image 1 → 5 embeddings
Image 2 → 5 embeddings
Image 3 → 5 embeddings
Image 4 → 5 embeddings
Image 5 → 21 embeddings
Image 6 → 5 embeddings
```

46

And:

```
Workflow 2 clustering state
Clusters      = 0
Memberships   = 0
Embeddings    = 46
```

That is a perfectly valid initial state for Workflow 2.

What we should change

Do not change the production repository.

Do not delete PostgreSQL embeddings.

Do not change `count_embeddings()`.

Instead, we need to decide what the test's intended contract is.

From the code you showed, the test subsequently performs **image-scoped testing**:

Python

Run

```
records = repository.get_face_embeddings_for_image(image_id)
```

and verifies the selected image's faces.

Therefore, the global assertion:

Python

Run

```
EXPECTED_WORKFLOW1_VECTOR_COUNT = 20
```

is no longer a good invariant.

The test should verify that:

1. PostgreSQL contains embeddings.
2. Workflow 2 starts with zero clusters.
3. Workflow 2 starts with zero memberships.
4. The selected test image has the expected embeddings.
5. Those embeddings correspond to vectors available in Workflow 1 HNSW.

Recommended change

I recommend replacing:

Python

Run

```
EXPECTED_WORKFLOW1_VECTOR_COUNT = 20
```

with a minimum sanity check.

For example:

Python

Run

```
MIN_EXPECTED_WORKFLOW1_VECTOR_COUNT = 1
```

Then change:

Python

Run

```
if embedding_count != EXPECTED_WORKFLOW1_VECTOR_COUNT:
    fail(
        "Unexpected PostgreSQL embedding count. "
```

```
f"Expected={EXPECTED_WORKFLOW1_VECTOR_COUNT}, "  
f"Actual={embedding_count}"  
)
```

to:

Python

Run

```
if embedding_count <= 0:  
    fail(  
        "Workflow 2 requires persisted Workflow 1 embeddings. "  
        f"Actual={embedding_count}"  
    )
```

This makes the test independent of the number of images processed previously.

Why this is better

The test shouldn't say:

```
"Workflow 1 must have exactly 20 embeddings."
```

That isn't a system invariant.

It should say:

```
"Workflow 2 cannot operate if Workflow 1 has no embeddings."
```

That's the architectural invariant.

But there is one more thing we should check

Before modifying it, I want to inspect how `select_test_image()` determines the expected face count.

Because image 5 has **21 faces**, while the other images have 5.

The test might have been designed around an assumption like:

```
selected image → 5 faces
```

If it dynamically obtains the face count, we're fine.

If it has another hard-coded `5` or `20`, we'll encounter the next stale assertion immediately after fixing this one.

Run:

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-Object -Skip 280
```

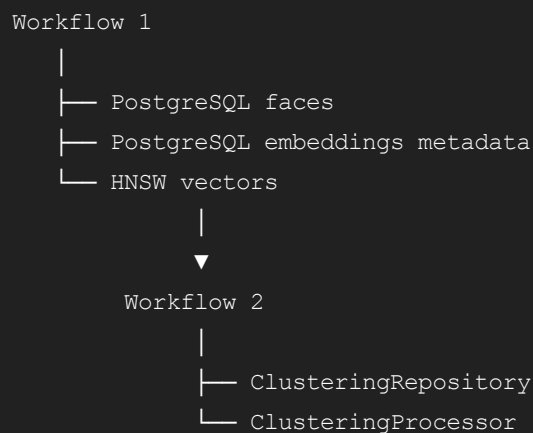
Also run:

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-String -Pattern
```

Paste both outputs.

Then we'll update the **test only**, preserving your current architecture:



At this point, **Workflow 1 → PostgreSQL → Outbox → Pub/Sub** has already been **successfully demonstrated**, so we should avoid changing any of that while fixing Workflow 2's outdated test fixture.

Pasted text(7).txt
Document

I have the current state. The important finding is that **Workflow 2 itself is not failing because of the `image_id` relationship**. Your PostgreSQL relationship is correct.

The actual failure is this:

```
Expected PostgreSQL embedding count = 20
Actual PostgreSQL embedding count   = 46
```

And your database currently contains:

image_id	embeddings
1	5
2	5
3	5
4	5
5	21
6	5
Total	46

So the test's assumption of **20 embeddings** is stale.

What happened

Your Workflow 1 processing has now persisted:

- image 1 → 5 faces
- image 2 → 5 faces
- image 3 → 5 faces
- image 4 → 5 faces
- image 5 → **21 faces**
- image 6 → 5 faces

Therefore:

```
5 + 5 + 5 + 5 + 21 + 5 = 46
```

The important point is that **image 5 genuinely has 21 persisted faces**, as confirmed by both `faces` and `face_embeddings`:

```
(21, 5, 0, 21, 'face:21')  
...  
(41, 5, 20, 41, 'face:41')
```

And Workflow 1 processing job for upload/image 5 is:

```
processing_jobs:  
status = COMPLETED  
attempt_count = 1
```

So this is not merely an accidental duplicate row that we should delete.

Also, your earlier SQL mistake is resolved

You correctly discovered that:

```
face_embeddings.image_id
```

does **not** exist.

The authoritative relationship is:

```
images
  ↓
faces
  ↓
face_embeddings
```

Specifically:

```
faces.image_id
face_embeddings.face_id
```

Your repository correctly implements this:

SQL

```
FROM face_pipeline.face_embeddings fe
INNER JOIN face_pipeline.faces f
  ON f.id = fe.face_id
WHERE f.image_id = :image_id
```

So do not change `get_face_embeddings_for_image()` just to fix this test.

The thing we should change

Your test has:

Python

Run

```
EXPECTED_WORKFLOW1_VECTOR_COUNT = 20
```

That is now inconsistent with the actual Workflow 1 PostgreSQL state.

But I would not simply change it to 46 yet.

Why?

Because Workflow 2's test appears to be intended to verify a specific controlled dataset, while your shared PostgreSQL database has accumulated data from previous Workflow 1 tests.

The test currently does this:

```
[0.2] Reset Workflow 2 clustering state
      ↓
delete/reset clusters + memberships
      ↓
[0.3] verify embeddings
```

Notice that `reset_clustering_state()` resets:

- clusters
- memberships

but **does not reset face embeddings**.

That is actually correct architecturally.

Workflow 2 should not casually delete Workflow 1's authoritative embeddings.

Therefore, the test should not assume that the global PostgreSQL embedding count will forever remain 20.

More importantly: Workflow 2 should process ONE event/image

Your own test documentation already points in the right direction:

```
Find the first image that has persisted face embeddings.
```

Then:

Python

Run

```
get_face_embeddings_for_image(image_id)
```

That is the correct isolation boundary.

For example, image 6 has:

```
image_id = 6  
face_count = 5
```

and the repository returns exactly those five embeddings:

```
47 → face_id 47 → image 6  
48 → face_id 48 → image 6  
49 → face_id 49 → image 6  
50 → face_id 50 → image 6  
51 → face_id 51 → image 6
```

That is excellent.

So the test should be validating:

```
Workflow 2 event  
    ↓  
image_id = 6  
    ↓  
5 embeddings  
    ↓  
5 faces  
    ↓  
clustering  
    ↓  
5 memberships
```

rather than:

```
Entire PostgreSQL database  
    ↓  
must contain exactly 20 embeddings
```

What I recommend

Do not delete the 46 embeddings.

Do not change the repository relationship.

Do not artificially delete image 5's 21 faces.

Instead, change the Workflow 2 processor test so that its initial-state validation checks **availability and integrity**, rather than requiring a global count of exactly 20.

For example, this:

Python

Run

```
EXPECTED_WORKFLOW1_VECTOR_COUNT = 20
```

and:

Python

Run

```
if embedding_count != EXPECTED_WORKFLOW1_VECTOR_COUNT:
    fail(
        "Unexpected PostgreSQL embedding count. "
        f"Expected={EXPECTED_WORKFLOW1_VECTOR_COUNT}, "
        f"Actual={embedding_count}"
    )
```

should no longer be the global gate.

A better test would be something like:

Python

Run

```
if embedding_count <= 0:
    fail(
        "Workflow 2 requires persisted Workflow 1 embeddings."
    )
```

Then `select_test_image()` identifies the image being tested, and **that image's embedding count becomes the test-specific expectation.**

There is one thing I want to inspect before telling you the exact edit

We need to see the remainder of `select_test_image()` and, most importantly, how `face_count` and `EXPECTED_WORKFLOW1_VECTOR_COUNT` are subsequently used throughout `test_clustering_processor.py`.

Because I don't want to make a superficial change that causes a later assertion to become inconsistent.

Please run this exact command from:

```
workflow2_clustering
```

PowerShell

```
powershell -Command "Get-Content 'test_clustering_processor.py' | Select-Object -Skip 260
```

Paste the complete output.

Then I can give you the **exact minimal code changes**, preserving the architecture we've established:

```
Workflow 1
  ↓
PostgreSQL = authoritative metadata/state
  ↓
Workflow 1 HNSW = actual 512-D vectors
  ↓
Pub/Sub FACE_EXTRACTION_COMPLETED
  ↓
Workflow 2
  ↓
image-scoped PostgreSQL lookup
  ↓
Workflow 1 HNSW vector retrieval
  ↓
clustering
  ↓
Workflow 2 PostgreSQL clusters + memberships
```

The current evidence strongly indicates that **the failure is in the test's global-count assumption, not in the Workflow 1 → PostgreSQL persistence or the Workflow 2 image-scoped repository logic.**

ChatGPT can make mistakes. Check important info.